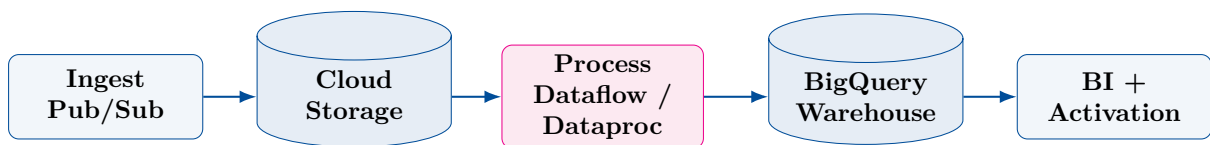


---

A COMPREHENSIVE, HANDS-ON ACADEMIC GUIDE

---

# GCP Master Data Engineer



*Inside this book*



*From durable object storage to governed, production-grade analytical platforms on Google Cloud.*

---

**PhD Aned Esquerra Arguelles**

A 24-week training guide for cloud data engineering practice

July 31, 2026

---

# Preface

Welcome to *GCP Master Data Engineer*. This book is a rigorous, hands-on academic guide for learning how modern data platforms are designed, implemented, secured, and operated on Google Cloud. It treats Google Cloud not as a menu of isolated products, but as an integrated data engineering system: storage, ingestion, processing, orchestration, warehousing, machine learning, governance, reliability, and cost control.

The training program behind this text is organized as twenty-four weeks of progressive practice. Each week introduces a cloud capability, a design lens, and a concrete implementation habit. The goal is not only to pass a certification exam or memorize product names; the goal is to reason like a professional data engineer who can defend architecture decisions under operational, financial, and security constraints.

## Who This Book Is For

This book is written for data engineers, analytics engineers, cloud engineers, software engineers, and technical leads who want to build production-grade data systems on Google Cloud. It is also useful for candidates preparing for the Professional Data Engineer exam and for architects who need a structured map from storage fundamentals to governed analytical platforms. Readers should be comfortable with SQL, Python, command-line tools, and basic distributed-systems vocabulary. The early chapters review key cloud concepts, but the exercises assume that you will use the Google Cloud CLI and read official documentation as part of the workflow.

## How This Book Is Organized

The book is divided into six parts, aligned to six training milestones and twenty-four weeks:

- **Part I: Core Data Infrastructure and Storage** establishes Cloud Storage, transactional sources, operational stores, identity, networking, and the durable landing zones that every data platform needs.
- **Part II: Data Warehousing and Analytics (BigQuery)** develops SQL, partitioning, clustering, dimensional modeling, performance tuning, and analytical cost governance.
- **Part III: Batch Processing and ETL Orchestration** covers Dataflow, Dataproc, Apache Beam, Spark migration, Cloud Composer, Dataform, and operational pipeline design.
- **Part IV: Streaming and Real-Time Analytics** focuses on Pub/Sub, event design, streaming windows, exactly-once trade-offs, real-time warehousing, observability, and resilience.
- **Part V: Machine Learning and MLOps** connects data engineering to Vertex AI, feature engineering, model pipelines, monitoring, and generative AI-assisted data work.

- **Part VI: Security, Governance, and Exam Prep** integrates IAM, CMEK, VPC Service Controls, Dataplex, lineage, data quality, FinOps, reliability reviews, and certification-style architecture decisions.

Appendices are intended to provide command references, exam checklists, solution sketches, and supplemental patterns as the course expands.

## Reading Paths

- **New to Google Cloud data engineering:** read each week in order and complete the hands-on project before moving on.
- **Experienced warehouse practitioner:** skim Part I, then spend extra time in BigQuery performance, Dataflow, Composer, and governance chapters.
- **Platform or cloud engineer:** focus on storage architecture, IAM, CMEK, networking boundaries, lifecycle policy, operational reliability, and cost controls.
- **Certification candidate:** read the key takeaways, architecture trade-off tables, interview questions, and exercises, then return to the official product documentation cited in each chapter.

## Conventions

Code identifiers, bucket names, CLI flags, and file names appear in `monospace`. Listings use the shared `pystyle` configuration from the preamble, with explicit languages on individual examples. Callout boxes flag **objectives**, **key terms**, **definitions**, **notes**, **tips**, and **pitfalls**. Citations point to primary papers and official Google Cloud documentation. Commands are written for PowerShell-friendly workflows where practical, but many Google Cloud CLI examples are portable shell commands that can be adapted to Cloud Shell.

# Contents

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| <b>0</b> | <b>Getting Started: Your Google Cloud Free Trial and Environment</b>     | <b>1</b>  |
| 0.1      | Why Chapter 0 Matters . . . . .                                          | 1         |
| 0.2      | Creating a Google Cloud Account . . . . .                                | 2         |
| 0.2.1    | Prerequisites . . . . .                                                  | 2         |
| 0.2.2    | Sign-Up Walkthrough . . . . .                                            | 2         |
| 0.2.3    | Resource Hierarchy: Organization, Folders, Projects, Resources . . . . . | 3         |
| 0.3      | Securing Identity Before Provisioning . . . . .                          | 4         |
| 0.3.1    | Cloud IAM Role Types . . . . .                                           | 4         |
| 0.3.2    | Why Not Grant Owner Broadly . . . . .                                    | 4         |
| 0.3.3    | Service Accounts and Keys . . . . .                                      | 4         |
| 0.3.4    | Enable 2-Step Verification . . . . .                                     | 5         |
| 0.4      | Tooling for Windows and PowerShell . . . . .                             | 5         |
| 0.4.1    | Install the Google Cloud CLI on Windows . . . . .                        | 5         |
| 0.4.2    | Initialize Authentication and Defaults . . . . .                         | 5         |
| 0.4.3    | Use <code>gcloud storage</code> and <code>gsutil</code> . . . . .        | 6         |
| 0.4.4    | Cloud Shell: No Local Install . . . . .                                  | 6         |
| 0.5      | Cost Guardrails Before Labs . . . . .                                    | 6         |
| 0.5.1    | Billing Account and Trial Protection . . . . .                           | 6         |
| 0.5.2    | Create a Budget with Alert Thresholds . . . . .                          | 7         |
| 0.5.3    | Billing Export and Quotas . . . . .                                      | 8         |
| 0.6      | Regions, Zones, Naming, and Labels . . . . .                             | 8         |
| 0.6.1    | Regions and Zones . . . . .                                              | 8         |
| 0.6.2    | Naming Conventions Used by the Book . . . . .                            | 8         |
| 0.7      | Hands-On Verification . . . . .                                          | 9         |
| 0.7.1    | Confirm Authentication and Configuration . . . . .                       | 9         |
| 0.7.2    | Create, List, and Delete a Bucket . . . . .                              | 10        |
| 0.8      | Cross-Cloud Setup Equivalence . . . . .                                  | 10        |
| 0.9      | Setup Checklist for the Rest of the Book . . . . .                       | 11        |
| 0.10     | Further Reading . . . . .                                                | 11        |
| 0.11     | Exercises . . . . .                                                      | 12        |
| <br>     |                                                                          |           |
| <b>I</b> | <b>Core Data Infrastructure and Storage</b>                              | <b>14</b> |
| <br>     |                                                                          |           |
| <b>1</b> | <b>Cloud Storage and Transfer Services</b>                               | <b>15</b> |
| 1.1      | Week 1 Roadmap . . . . .                                                 | 16        |
| 1.2      | Monday: Object Storage Fundamentals . . . . .                            | 17        |
| 1.2.1    | Objects, Buckets, and Namespaces . . . . .                               | 17        |
| 1.2.2    | Consistency, Durability, and Availability . . . . .                      | 17        |
| 1.2.3    | Storage Classes . . . . .                                                | 18        |

|          |                                                                                          |           |
|----------|------------------------------------------------------------------------------------------|-----------|
| 1.2.4    | Location Types . . . . .                                                                 | 18        |
| 1.2.5    | Bucket Architecture for Data Lakes . . . . .                                             | 19        |
| 1.3      | Tuesday: Lifecycle, Versioning, Retention, and Soft Delete . . . . .                     | 19        |
| 1.3.1    | Object Lifecycle Management . . . . .                                                    | 19        |
| 1.3.2    | Object Versioning . . . . .                                                              | 20        |
| 1.3.3    | Retention Policies and Bucket Lock . . . . .                                             | 20        |
| 1.3.4    | Soft Delete . . . . .                                                                    | 20        |
| 1.4      | Wednesday: Data Migration and Transfer Choices . . . . .                                 | 21        |
| 1.4.1    | Storage Transfer Service . . . . .                                                       | 21        |
| 1.4.2    | Transfer Appliance . . . . .                                                             | 21        |
| 1.4.3    | CLI Tools: <code>gcloud storage</code> and <code>gsutil</code> . . . . .                 | 21        |
| 1.4.4    | Migration Decision Matrix . . . . .                                                      | 22        |
| 1.5      | Thursday Hands-On Project: Bucket, Lifecycle, Transfer, and Client Code . . . . .        | 22        |
| 1.5.1    | PowerShell Preparation . . . . .                                                         | 22        |
| 1.5.2    | Lifecycle JSON . . . . .                                                                 | 23        |
| 1.5.3    | Storage Transfer Job from Public S3 . . . . .                                            | 24        |
| 1.5.4    | Python Client Snippet . . . . .                                                          | 24        |
| 1.5.5    | Validation Checklist . . . . .                                                           | 25        |
| 1.6      | Friday Use Case and Architecture: Multi-Region DR for Media Streaming . . . . .          | 25        |
| 1.6.1    | Scenario . . . . .                                                                       | 25        |
| 1.6.2    | Design . . . . .                                                                         | 25        |
| 1.6.3    | Cost and Operations . . . . .                                                            | 26        |
| 1.7      | Governance and Security Design Notes . . . . .                                           | 26        |
| 1.8      | Operational Runbook for Week 1 Buckets . . . . .                                         | 26        |
| 1.9      | Interview Q&A . . . . .                                                                  | 27        |
| 1.9.1    | Concepts and Trade-offs . . . . .                                                        | 27        |
| 1.10     | Further Reading . . . . .                                                                | 28        |
| 1.11     | Exercises . . . . .                                                                      | 28        |
| 1.12     | Capstone Project: A Resilient Global Media Asset Store on Cloud Storage . . . . .        | 29        |
| <b>2</b> | <b>Relational Databases: Cloud SQL and AlloyDB</b> . . . . .                             | <b>35</b> |
| 2.1      | Week 2 Roadmap . . . . .                                                                 | 36        |
| 2.2      | Monday: Cloud SQL Architecture, High Availability, and Read Replicas . . . . .           | 36        |
| 2.2.1    | Managed Architecture . . . . .                                                           | 36        |
| 2.2.2    | Regional High Availability . . . . .                                                     | 37        |
| 2.2.3    | Backups and Point-in-Time Recovery . . . . .                                             | 37        |
| 2.2.4    | Read Replicas . . . . .                                                                  | 38        |
| 2.3      | Tuesday: Database Migration Service and Private IP Connections . . . . .                 | 38        |
| 2.3.1    | Migration Phases . . . . .                                                               | 38        |
| 2.3.2    | Private IP and Private Services Access . . . . .                                         | 39        |
| 2.3.3    | DMS Design Checklist . . . . .                                                           | 39        |
| 2.4      | Wednesday: AlloyDB for PostgreSQL, Columnar Engine, and ML Integration . . . . .         | 39        |
| 2.4.1    | Why AlloyDB Exists . . . . .                                                             | 39        |
| 2.4.2    | Columnar Engine . . . . .                                                                | 40        |
| 2.4.3    | ML Integration . . . . .                                                                 | 40        |
| 2.5      | Thursday Hands-On Project: HA Cloud SQL PostgreSQL, Read Replica, and Failover . . . . . | 41        |
| 2.5.1    | PowerShell Preparation and HA Instance Creation . . . . .                                | 41        |
| 2.5.2    | Read Replica and Failover Drill . . . . .                                                | 42        |
| 2.5.3    | Python Connectivity Check . . . . .                                                      | 42        |
| 2.5.4    | Cleanup . . . . .                                                                        | 43        |

|          |                                                                               |           |
|----------|-------------------------------------------------------------------------------|-----------|
| 2.6      | Friday Use Case and Architecture: Zero-Downtime ERP Migration . . . . .       | 43        |
| 2.6.1    | Scenario . . . . .                                                            | 43        |
| 2.6.2    | Migration Architecture . . . . .                                              | 43        |
| 2.6.3    | Cutover and Validation . . . . .                                              | 43        |
| 2.7      | Security, Operations, and Cost Notes . . . . .                                | 44        |
| 2.8      | Interview Q&A . . . . .                                                       | 44        |
| 2.8.1    | Concepts and Trade-offs . . . . .                                             | 44        |
| 2.9      | Further Reading . . . . .                                                     | 44        |
| 2.10     | Exercises . . . . .                                                           | 45        |
| 2.11     | Capstone Project: HA Cloud SQL and AlloyDB Analytics for ERP Migration . .    | 45        |
| <b>3</b> | <b>Global Relational Scaling: Cloud Spanner</b>                               | <b>51</b> |
| 3.1      | Week 3 Roadmap . . . . .                                                      | 52        |
| 3.2      | Monday: Spanner Concepts and Strong Consistency . . . . .                     | 53        |
| 3.2.1    | Instances, Nodes, Processing Units, and Databases . . . . .                   | 53        |
| 3.2.2    | Replicas, Leaders, and Paxos Quorums . . . . .                                | 53        |
| 3.2.3    | TrueTime and External Consistency . . . . .                                   | 54        |
| 3.3      | Tuesday: Schema Design, Primary Keys, Interleaving, and Indexing . . . . .    | 54        |
| 3.3.1    | Primary Keys Define Physical Locality . . . . .                               | 54        |
| 3.3.2    | Interleaved Tables . . . . .                                                  | 55        |
| 3.3.3    | Secondary Indexes and Covering Reads . . . . .                                | 55        |
| 3.4      | Wednesday: Performance Tuning and Plans . . . . .                             | 56        |
| 3.4.1    | Reading Query Plans . . . . .                                                 | 56        |
| 3.4.2    | Hot Spots and Split Health . . . . .                                          | 56        |
| 3.4.3    | Transactions, Mutations, and Latency . . . . .                                | 56        |
| 3.5      | Thursday Hands-On Project: Multi-Region Cart Lab . . . . .                    | 57        |
| 3.5.1    | PowerShell Preparation and Instance Creation . . . . .                        | 57        |
| 3.5.2    | Interleaved Cart Schema . . . . .                                             | 57        |
| 3.5.3    | Python Synthetic Load . . . . .                                               | 58        |
| 3.5.4    | Query Plan Check and Cleanup . . . . .                                        | 59        |
| 3.6      | Friday Use Case: Global Payment Ledger . . . . .                              | 59        |
| 3.6.1    | Scenario . . . . .                                                            | 59        |
| 3.6.2    | Architecture Pattern . . . . .                                                | 60        |
| 3.6.3    | Correctness Controls . . . . .                                                | 60        |
| 3.7      | Security, Operations, and Cost Notes . . . . .                                | 60        |
| 3.8      | Interview Q&A . . . . .                                                       | 61        |
| 3.8.1    | Concepts and Trade-offs . . . . .                                             | 61        |
| 3.9      | Further Reading . . . . .                                                     | 61        |
| 3.10     | Exercises . . . . .                                                           | 62        |
| 3.11     | Capstone Project: Globally Consistent Payment Ledger on Cloud Spanner . . . . | 62        |
| <b>4</b> | <b>Wide-Column NoSQL: Cloud Bigtable</b>                                      | <b>68</b> |
| 4.1      | Week 4 Roadmap . . . . .                                                      | 69        |
| 4.2      | Monday: Bigtable Architecture, Tablets, Nodes, and App Profiles . . . . .     | 70        |
| 4.2.1    | Instances, Clusters, Nodes, Tables, and Column Families . . . . .             | 70        |
| 4.2.2    | Tablets and Node-Level Sharding . . . . .                                     | 70        |
| 4.2.3    | Replication and App Profiles . . . . .                                        | 71        |
| 4.3      | Tuesday: Row-Key Design, Salting, Hashing, and Hotspot Avoidance . . . . .    | 71        |
| 4.3.1    | Design from Access Patterns . . . . .                                         | 71        |
| 4.3.2    | Salting, Hashing, Bucketing, and Reverse Time . . . . .                       | 72        |
| 4.3.3    | Column Families and Cell Versions . . . . .                                   | 72        |
| 4.4      | Wednesday: Performance Optimization, Garbage Collection, and Key Visualizer   | 72        |

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| 4.4.1    | Throughput, Nodes, and Workload Isolation . . . . .                   | 72        |
| 4.4.2    | Garbage Collection Policies . . . . .                                 | 73        |
| 4.4.3    | Key Visualizer and Heatmap Evidence . . . . .                         | 73        |
| 4.5      | Thursday Hands-On Project: IoT Temperature Sensor Ingestion . . . . . | 74        |
| 4.5.1    | PowerShell Preparation and Cluster Creation . . . . .                 | 74        |
| 4.5.2    | Python Ingestion Script . . . . .                                     | 74        |
| 4.5.3    | Read Check and Cleanup . . . . .                                      | 75        |
| 4.6      | Friday Use Case: 100,000 Stock Ticks per Second . . . . .             | 76        |
| 4.6.1    | Scenario . . . . .                                                    | 76        |
| 4.6.2    | Architecture Pattern . . . . .                                        | 76        |
| 4.6.3    | Serving and Correctness Controls . . . . .                            | 76        |
| 4.7      | Security, Operations, and Cost Notes . . . . .                        | 76        |
| 4.8      | Interview Q&A . . . . .                                               | 77        |
| 4.8.1    | Concepts and Trade-offs . . . . .                                     | 77        |
| 4.9      | Further Reading . . . . .                                             | 77        |
| 4.10     | Exercises . . . . .                                                   | 78        |
| 4.11     | Milestone 1 Capstone: E-Commerce Multi-Tier Data Backend . . . . .    | 78        |
| <b>A</b> | <b>Cross-Cloud Service Equivalence</b>                                | <b>84</b> |
| A.1      | Object and Data-Lake Storage . . . . .                                | 84        |
| A.2      | Managed Relational Databases . . . . .                                | 85        |
| A.3      | Elastic Compute and Analytical Warehouses . . . . .                   | 85        |
| A.4      | Data Organization and the Medallion Pattern . . . . .                 | 85        |
| A.5      | Document Data Modeling . . . . .                                      | 86        |
| A.6      | Globally Distributed and NoSQL Databases . . . . .                    | 86        |
| A.7      | Managed Apache Spark . . . . .                                        | 87        |
| A.8      | Query Acceleration and Materialized Views . . . . .                   | 87        |
| A.9      | In-Memory Caching and Hybrid Ingestion . . . . .                      | 87        |
| A.10     | Wide-Column and Key-Value NoSQL . . . . .                             | 88        |
| A.11     | Data Protection: Cloning, Time Travel, and Migration . . . . .        | 88        |
|          | <b>Glossary</b>                                                       | <b>89</b> |

# List of Figures

|     |                                                                                                                                                                 |    |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1   | Google Cloud’s hierarchy places hands-on resources inside Projects, which may sit under Folders and an Organization. . . . .                                    | 3  |
| 1.1 | A storage-class lifecycle for a lakehouse-style Cloud Storage landing zone. . . . .                                                                             | 19 |
| 1.2 | A multi-region disaster recovery architecture for ten petabytes of media assets. . . . .                                                                        | 25 |
| 1.3 | Capstone reference architecture for a resilient global media asset store on Cloud Storage. . . . .                                                              | 30 |
| 2.1 | Cloud SQL regional HA places a standby in another zone and requires client reconnection after failover. . . . .                                                 | 37 |
| 2.2 | AlloyDB can serve transactions from PostgreSQL-compatible row storage while accelerating selected analytical scans with a columnar engine. . . . .              | 40 |
| 2.3 | Zero-downtime ERP migration using DMS, private connectivity, Cloud SQL HA, and AlloyDB analytics offload. . . . .                                               | 43 |
| 2.4 | Capstone reference architecture for HA Cloud SQL with AlloyDB analytics offload during ERP migration. . . . .                                                   | 47 |
| 3.1 | Spanner combines Paxos replication with TrueTime commit timestamps to provide globally ordered transactions. . . . .                                            | 54 |
| 3.2 | Interleaving stores child rows near parent rows while secondary indexes support alternate access paths. . . . .                                                 | 55 |
| 3.3 | A multi-continent payment ledger uses Spanner for globally consistent OLTP and streams committed facts to analytics systems. . . . .                            | 60 |
| 3.4 | Capstone reference architecture for a globally consistent payment ledger on multi-region Spanner. . . . .                                                       | 63 |
| 4.1 | Bigtable splits a table into tablets, then serves those row-key ranges across cluster nodes. . . . .                                                            | 71 |
| 4.2 | A timestamp-leading key hot spots current writes, while a salted reversed-timestamp key spreads ingestion and preserves recent-event scans. . . . .             | 72 |
| 4.3 | Trading ingestion uses Pub/Sub and Dataflow for burst absorption, Bigtable for low-latency serving, and analytical stores for archive and surveillance. . . . . | 76 |
| 4.4 | Milestone 1 reference architecture integrating Cloud Storage, Cloud SQL, Spanner, and Bigtable behind private application connectivity. . . . .                 | 80 |

# List of Tables

|     |                                                                 |    |
|-----|-----------------------------------------------------------------|----|
| 1   | Book naming and labeling conventions. . . . .                   | 9  |
| 1.1 | Cloud Storage classes and common data engineering uses. . . . . | 18 |
| 1.2 | Storage protection and governance controls. . . . .             | 21 |
| 1.3 | Choosing a Cloud Storage migration mechanism. . . . .           | 22 |
| 1.4 | Operational signals for Cloud Storage runbooks. . . . .         | 27 |
| 2.1 | Cloud SQL reliability and scale patterns. . . . .               | 38 |
| 2.2 | Database Migration Service readiness checklist. . . . .         | 39 |
| 3.1 | Spanner schema design trade-offs. . . . .                       | 56 |
| 4.1 | Bigtable performance diagnosis patterns. . . . .                | 73 |

# Getting Started: Your Google Cloud Free Trial and Environment

## Learning Objectives

- Create a Google Cloud account, start the \$300 / 90-day free trial, and explain how the Always Free tier differs from trial credits.
- Navigate the Organization, Folder, Project, and Resource hierarchy and use Projects as isolation, ownership, and billing boundaries.
- Secure a learner environment with Cloud IAM least privilege, service-account hygiene, and 2-Step Verification.
- Install and initialize the Google Cloud CLI on Windows, or use Cloud Shell when a browser-only environment is preferable.
- Configure cost guardrails with budgets, alert thresholds, labels, quotas, and billing export before creating data resources.
- Apply the book's region, zone, naming, and labeling conventions consistently across every hands-on chapter.
- Verify the setup by authenticating, selecting a project, creating a temporary Cloud Storage bucket, listing it, and deleting it.

## Key Terms

**Google Cloud account:** a billing-enabled cloud identity associated with a Google account and payment verification. **Free trial:** time-limited introductory credit, currently \$300 for 90 days for eligible new customers. **Always Free:** ongoing monthly no-cost usage limits for selected services in eligible regions. **Project:** the main Google Cloud isolation, API, quota, IAM, and billing unit used throughout this book. **Cloud IAM:** Google Cloud's identity and access management system for granting roles on resources. **Service account:** a non-human identity used by applications, pipelines, and managed services. **Budget:** a billing guardrail that sends alerts when actual or forecasted spend crosses thresholds.

## 0.1 Why Chapter 0 Matters

Chapter 1 begins with Cloud Storage, but successful cloud data engineering starts before the first bucket is created. A learner needs an account, a project, a billing boundary, a secure identity model, a working command line, and cost alerts. These foundations are not administrative trivia. They determine who can create resources, where data can live, how charges are attributed, and how quickly an experiment can be cleaned up.

This chapter is written for a Windows workstation using PowerShell. All commands are safe to run from a normal PowerShell session unless explicitly marked as an administrative install step. Cloud Shell is included as a browser-based alternative when local installation is unavailable. The goal is a small, well-labeled, disposable learning environment that can be used for every hands-on exercise in this book.

### Definition

**Google Cloud setup** is the combination of identity, billing, project selection, regional defaults, CLI authentication, labels, and cost controls required before provisioning technical services. A good setup makes later labs repeatable, auditable, and inexpensive.

## 0.2 Creating a Google Cloud Account

### 0.2.1 Prerequisites

To start the labs, you need three things. First, you need a Google account. This can be a personal Google account for individual study or an account supplied by an organization or school. Second, eligible new customers need a card or accepted payment method for identity and payment verification. Third, you need a browser, a phone or authenticator app for stronger sign-in security, and either a Windows PowerShell terminal or access to Cloud Shell.

Google Cloud's introductory offer gives eligible new customers a free trial with \$300 in credits for 90 days. The credits can be used across many Google Cloud services, subject to product limits and trial restrictions. Google also offers an Always Free tier for selected services, which provides limited no-cost usage each month in eligible regions. The trial and Always Free tier are related but not identical: trial credits expire after the trial period, while Always Free limits can continue after the trial if the account remains in good standing and usage stays within eligible limits [23].

### Note

Cloud offers change over time. Treat the exact trial amount, duration, eligible countries, and Always Free product limits as values to verify on the official free program page before beginning a cohort or workshop [23].

### 0.2.2 Sign-Up Walkthrough

Use the Google Cloud Console sign-up flow rather than an unofficial link. The process is normally:

1. Open the Google Cloud free program page and choose the option to start for free.
2. Sign in with the Google account you will use for the book.
3. Select country, account type, and required profile information.
4. Accept the terms after reading the trial and billing conditions.
5. Provide a card or accepted payment method for verification.
6. Create or confirm the first billing account.
7. Land in the Google Cloud Console and open the project selector.
8. Create a dedicated learning project instead of using a default or shared project.

The card protects Google from abuse and verifies payment readiness, but the trial is designed to prevent automatic charges during the free trial unless you explicitly upgrade to a paid account. When credits are exhausted or the trial ends, resources may stop, and you must choose whether to upgrade. You should still configure budgets and alerts immediately, because cost awareness is a professional habit, not only a payment-protection mechanism.

### 0.2.3 Resource Hierarchy: Organization, Folders, Projects, Resources

Google Cloud organizes resources hierarchically. At the top, an **Organization** represents a company, school, or domain-managed tenant. Under the Organization, **Folders** can group departments, environments, applications, or compliance zones. Under folders, **Projects** host resources. Resources are the services you create: buckets, BigQuery datasets, Pub/Sub topics, service accounts, Dataflow jobs, Dataproc clusters, and more.

Personal learners often start without an Organization resource. In that case, projects sit directly under the account's project list. Enterprise learners usually work inside an Organization with folder-level policies, shared networking, centralized billing, and security controls. The same project-centered mental model still applies.

#### Definition

A **Project** is the primary boundary for APIs, quotas, IAM grants, billing attribution, audit logs, and many operational policies. In this book, every hands-on resource belongs to one learner project unless a chapter explicitly says otherwise.

Projects are the unit of isolation because they collect related resources and make deletion practical. Deleting a lab project is safer than manually deleting dozens of scattered resources. Projects are also the billing attribution boundary. A billing account can fund many projects, but charges are reported by project, service, SKU, label, and time. Finally, projects are an IAM boundary. You can grant a user permission on one project without granting access to unrelated projects.

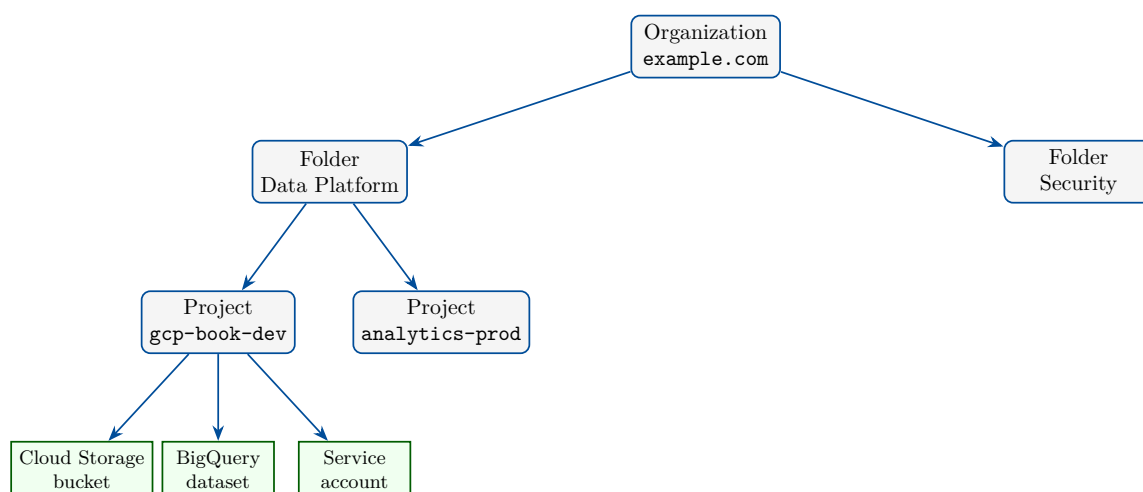


Figure 1: Google Cloud's hierarchy places hands-on resources inside Projects, which may sit under Folders and an Organization.

## 0.3 Securing Identity Before Provisioning

### 0.3.1 Cloud IAM Role Types

Cloud IAM grants permissions by binding a principal to a role on a resource. A principal can be a user, group, service account, domain, or workload identity. A role is a collection of permissions. A binding says, in effect, “this principal receives this role on this resource.” Permissions inherited from an Organization or Folder can flow down to projects, so always inspect the effective access path when troubleshooting.

Cloud IAM roles come in three major types [24]:

- **Basic roles:** Owner, Editor, and Viewer. These are broad legacy roles. They are easy to understand but too coarse for most production use.
- **Predefined roles:** Google-managed roles scoped to services and job functions, such as `roles/storage.admin`, `roles/bigquery.dataViewer`, or `roles/pubsub.publisher`.
- **Custom roles:** organization- or project-level roles containing a specific set of permissions selected by your team.

The principle of least privilege says each identity should receive only the permissions needed, at the narrowest practical scope, for the shortest practical time. In a learner project, you may temporarily act as an administrator because you own the environment. In a professional environment, day-to-day data engineers should not hold broad Owner access. They should use groups, predefined roles, approval workflows, break-glass procedures, and audited elevation.

### 0.3.2 Why Not Grant Owner Broadly

The Owner role can change IAM policy, enable services, manage resources, and link or modify billing settings. It is powerful enough to create security, cost, and data-loss incidents. Owner should be limited to a small number of accountable administrators and emergency paths. Avoid granting Owner to service accounts, lab partners, automation jobs, or broad groups. If a person only needs to create buckets, grant a storage-specific role. If a pipeline only needs to write objects to one bucket, grant an object-writing role on that bucket rather than Owner on the project.

#### Common Pitfall

Leaked service-account keys and Owner sprawl are two of the fastest ways to lose control of a cloud project. A JSON key committed to source control can be copied instantly, and an over-broad Owner grant can disable guardrails or create expensive resources. Prefer short-lived credentials, workload identity patterns, service-specific predefined roles, and routine access reviews.

### 0.3.3 Service Accounts and Keys

Service accounts are identities for workloads, not people. A Dataflow job, VM, Cloud Run service, Composer environment, or local automation script may run as a service account. Treat service accounts as production identities with owners, descriptions, minimal roles, and rotation or replacement plans.

Avoid user-managed service-account keys whenever possible. A downloaded key file is a long-lived bearer credential: whoever has the file can authenticate as the service account until the key is revoked or expires. Prefer managed service identities, Workload Identity Federation, attached service accounts on Google Cloud runtimes, and short-lived credentials. When a key

is unavoidable for a lab or legacy integration, store it outside source control, restrict the service account, set a rotation date, and delete the key as soon as the exercise ends [24].

### 0.3.4 Enable 2-Step Verification

Before creating resources, enable 2-Step Verification on the Google account used for the book. Use an authenticator app, security key, or another supported second factor. Store backup codes securely. If the account is managed by an organization, follow its identity policy and do not bypass centralized controls. Strong sign-in is especially important because the same account can access billing, IAM, audit logs, and project deletion.

## 0.4 Tooling for Windows and PowerShell

### 0.4.1 Install the Google Cloud CLI on Windows

The Google Cloud CLI provides the `gcloud`, `bq`, `gsutil`, and modern `gcloud storage` command groups used throughout this book [22]. On Windows, install it with the official Google Cloud CLI installer, Windows package management approved by your organization, or a managed software center. Use the official documentation rather than copying binaries from a third party.

After installation, open a new PowerShell window so that the updated `PATH` is loaded. Confirm that the command is available:

```
1 Get-Command gcloud
2
3 gcloud --version
```

Listing 1: Verifying the Google Cloud CLI from Windows PowerShell.

Expected output includes the path to `gcloud.cmd` and a version block similar to this:

```
1 Google Cloud SDK 528.0.0
2 bq 2.1.18
3 core 2026.07.15
4 gcloud-crc32c 1.0.0
5 gsutil 5.35
```

Listing 2: Representative Google Cloud CLI version output.

Version numbers will differ. The important result is that `gcloud` runs from PowerShell without requiring a special terminal.

### 0.4.2 Initialize Authentication and Defaults

Run `gcloud init` to authenticate, select or create a configuration, choose the default project, and optionally set a default Compute Engine region and zone. The command opens a browser login flow. Use the same Google account that owns or has access to the learner project.

```
1 gcloud init
2
3 gcloud auth login
4
5 gcloud config set project PROJECT_ID
6 gcloud config set compute/region us-central1
7 gcloud config set compute/zone us-central1-a
```

Listing 3: Initial Google Cloud CLI setup in PowerShell.

Replace `PROJECT_ID` with your real project ID. The project ID is globally unique, lower case, and different from the display name. A display name can be changed; the project ID is the stable identifier used by APIs, logs, and commands.

#### Note

`gcloud init` and `gcloud auth login` both authenticate a user account for CLI use. If you already initialized the CLI, `gcloud auth login` can refresh or add an account without redoing all configuration prompts.

### 0.4.3 Use `gcloud storage` and `gsutil`

Google now emphasizes `gcloud storage` for Cloud Storage command-line operations, while `gsutil` remains common in older scripts, documentation, and installed environments. This book prefers `gcloud storage` for new examples and mentions `gsutil` when it appears in production patterns or migration materials. Learn both names so that existing runbooks are readable.

For example, these commands express similar intent:

```
1 # Preferred for new examples in this book
2 gcloud storage buckets list
3
4 # Legacy command still seen in many environments
5 gsutil ls
```

Listing 4: Modern and legacy Cloud Storage command examples.

### 0.4.4 Cloud Shell: No Local Install

Cloud Shell is a browser-based shell in the Google Cloud Console with the Google Cloud CLI preinstalled, temporary compute, and persistent home storage within documented limits. It is useful for classrooms, locked-down laptops, and quick console-adjacent troubleshooting. Open it from the Cloud Shell icon in the Google Cloud Console. The shell already knows the console account, but you should still confirm the active project and configuration before running labs.

```
1 gcloud config get-value project
2 gcloud auth list
```

Listing 5: Checking the active project from Cloud Shell.

Cloud Shell is convenient, but local PowerShell remains valuable. Local development lets you use an editor, source control, local sample data, and the same scripts you will later automate. Use Cloud Shell when installation is blocked; use local PowerShell when building repeatable project workflows.

## 0.5 Cost Guardrails Before Labs

### 0.5.1 Billing Account and Trial Protection

A project must be linked to a billing account before most billable resources can run. The billing account is the payment and invoicing container. The project is where charges originate. The free trial credits apply to eligible usage while the trial is active. Google states that you are not automatically charged after the free trial ends unless you upgrade to a paid account [23]. This protection is helpful, but it does not replace cost design. If you upgrade, remove forgotten resources, or move to an organizational billing account, spend can become real.

**Tip**

**Measure it.** Every lab should have a way to answer three questions: what did we create, who owns it, and how much can it cost if forgotten for a week? Budgets, labels, quotas, and billing export turn those questions into observable signals.

### 0.5.2 Create a Budget with Alert Thresholds

Create a budget immediately after linking billing. In the Console, open **Billing** → **Budgets & alerts**, choose the billing account, create a budget scoped to the learner project, and set alert thresholds such as 50%, 90%, and 100% of a small monthly amount. For an individual learner, a budget of \$10 or \$25 is often enough to reveal mistakes. For a cohort, set budgets per project or per folder according to the teaching plan.

Budget alerts notify selected users or billing administrators. They do not automatically stop all resources. Think of a budget as an alarm, not a circuit breaker. Pair budgets with quotas, labels, lifecycle rules, and cleanup habits.

The exact CLI support for budgets can vary by installed components and API availability. The following PowerShell example shows a practical pattern: create a project, link billing, enable billing APIs, and create a budget from a JSON file through `gcloud billing budgets create`. If your organization restricts billing APIs, perform the budget step in the Console.

```

1 $ProjectId = "gcp-book-$(New-Guid).Guid.Substring(0,8)"
2 $BillingAccount = "BILLING_ACCOUNT_ID"
3 $Region = "us-central1"
4
5 # Create a dedicated project for the book.
6 gcloud projects create $ProjectId --name="GCP Book Labs"
7
8 # Link the project to a billing account.
9 gcloud billing projects link $ProjectId --billing-account=$BillingAccount
10
11 # Set defaults for later commands.
12 gcloud config set project $ProjectId
13 gcloud config set compute/region $Region
14 gcloud config set compute/zone "$Region-a"
15
16 # Enable the services used by setup and early chapters.
17 gcloud services enable cloudbilling.googleapis.com billingbudgets.googleapis.com storage.googleapis.com
18
19 # Create budget.json in the current directory, then create the budget.
20 @'
21 {
22   "displayName": "gcp-book-lab-budget",
23   "budgetFilter": {
24     "projects": ["projects/PROJECT_NUMBER"]
25   },
26   "amount": {
27     "specifiedAmount": {
28       "currencyCode": "USD",
29       "units": "25"
30     }
31   },
32   "thresholdRules": [
33     {"thresholdPercent": 0.5},
34     {"thresholdPercent": 0.9},
35     {"thresholdPercent": 1.0}
36   ]
37 }
21

```

```

37 }
38 '@ | Set-Content -Path .\budget.json -Encoding utf8
39
40 $ProjectNumber = gcloud projects describe $ProjectId --format="value(
    projectNumber)"
41 (Get-Content .\budget.json) -replace "PROJECT_NUMBER", $ProjectNumber |
42     Set-Content -Path .\budget.json -Encoding utf8
43
44 gcloud billing budgets create --billing-account=$BillingAccount --budget-from-
    file=.\budget.json
45 Remove-Item .\budget.json

```

Listing 6: Creating a learner project and budget guardrail from PowerShell.

### 0.5.3 Billing Export and Quotas

Billing export sends detailed cost and usage data to BigQuery. Enable it early in serious learning environments so that future chapters can query real spend patterns. In a single-person trial, billing export may feel unnecessary, but it teaches the same FinOps loop used in production: export, label, query, visualize, and review.

Quotas limit how much of a resource class a project can consume. They protect Google Cloud and can protect you from runaway experiments. Review quotas for expensive or scalable services before labs that create compute, streaming, or data processing resources. A quota error during a lab is not always bad; it may be a guardrail doing its job.

## 0.6 Regions, Zones, Naming, and Labels

### 0.6.1 Regions and Zones

A Google Cloud region is a specific geographic area, such as `us-central1`. A zone is an isolated deployment area within a region, such as `us-central1-a`. Some resources are zonal, such as many virtual machine placements. Some are regional, such as many managed service locations. Some are multi-region or global. Data engineering resources often have location compatibility requirements. For example, moving data between a Cloud Storage bucket and BigQuery dataset can involve location and egress considerations.

This book uses `us-central1` and `us-central1-a` as defaults unless a chapter explicitly chooses another region to demonstrate disaster recovery, data residency, or multi-region design. If you are outside the United States or under organizational policy, choose an approved region and use it consistently. Do not mix regions casually; inconsistent location choices complicate cost, latency, and governance.

### 0.6.2 Naming Conventions Used by the Book

Names should be readable, unique, and disposable. Use lower-case kebab-case for projects, buckets, topics, datasets when allowed, and service accounts where the product permits it. Include platform, purpose, environment, and a short random suffix when global uniqueness is required.

| Resource         | Pattern                                                   | Example                                 |
|------------------|-----------------------------------------------------------|-----------------------------------------|
| Project ID       | <code>gcp-book-&lt;name&gt;-&lt;suffix&gt;</code>         | <code>gcp-book-labs-a1b2c3d4</code>     |
| Bucket           | <code>&lt;project&gt;-&lt;zone&gt;-&lt;purpose&gt;</code> | <code>gcp-book-labs-a1b2c3d4-raw</code> |
| Service account  | <code>&lt;workload&gt;-sa</code>                          | <code>dataflow-lab-sa</code>            |
| BigQuery dataset | <code>&lt;domain&gt;_&lt;layer&gt;</code>                 | <code>retail_raw</code>                 |
| Pub/Sub topic    | <code>&lt;domain&gt;-&lt;event&gt;-topic</code>           | <code>orders-created-topic</code>       |
| Labels           | <code>owner, env, book, chapter</code>                    | <code>env=lab, chapter=00</code>        |

Table 1: Book naming and labeling conventions.

Labels are key-value metadata used for cost allocation, inventory, and operations. Apply labels at creation time whenever a service supports them. Recommended labels for this book are `book=gcp-data-engineer`, `env=lab`, `owner=<your-alias>`, and `chapter=00`. Avoid personal data in labels because labels may appear in billing exports, logs, and reports.

### Common Pitfall

A globally unique bucket name can accidentally reveal project names, customer names, or internal programs. Use neutral lab names and random suffixes. Never include secrets, email addresses, or regulated data categories in resource names.

## 0.7 Hands-On Verification

This section confirms that authentication, project selection, APIs, region defaults, and Cloud Storage access work. It creates one temporary bucket, lists it, and deletes it. The commands assume PowerShell and a configured project.

### 0.7.1 Confirm Authentication and Configuration

Run the following commands:

```
1 gcloud auth list
2 gcloud config list
```

Listing 7: Verifying authentication and active configuration.

Expected output resembles the following. Your account, project, and zone will differ.

```
1 Credentialed Accounts
2 ACTIVE ACCOUNT
3 * learner@example.com
4
5 [compute]
6 region = us-central1
7 zone = us-central1-a
8 [core]
9 account = learner@example.com
10 project = gcp-book-labs-a1b2c3d4
```

Listing 8: Representative authentication and configuration output.

If no active account appears, run `gcloud auth login`. If no project appears, run `gcloud config set project PROJECT_ID`. If the storage API is disabled, enable it with `gcloud services enable storage.googleapis.com`.

### 0.7.2 Create, List, and Delete a Bucket

Cloud Storage bucket names are globally unique. The following script builds a unique name from your project ID and a timestamp, creates the bucket in the default region, lists it, and then deletes it. The bucket is empty, so deletion should succeed immediately.

```

1 $ProjectId = gcloud config get-value project
2 $Region = gcloud config get-value compute/region
3 $Suffix = Get-Date -Format "yyyyMMddHHmmss"
4 $Bucket = "${ProjectId}-ch00-${Suffix}"
5
6 # Create a temporary bucket with labels.
7 gcloud storage buckets create "gs://$Bucket" --location=$Region --uniform-
   bucket-level-access --project=$ProjectId
8
9 gcloud storage buckets update "gs://$Bucket" --update-labels=book=gcp-data-
   engineer,env=lab,chapter=00
10
11 # List the bucket and inspect basic metadata.
12 gcloud storage buckets list --filter="name:$Bucket"
13 gcloud storage buckets describe "gs://$Bucket" --format="value(name,location,
   iamConfiguration.uniformBucketLevelAccess.enabled)"
14
15 # Clean up immediately.
16 gcloud storage rm --recursive "gs://$Bucket"

```

Listing 9: Creating, listing, and deleting a temporary verification bucket.

Representative output:

```

1 Creating gs://gcp-book-labs-a1b2c3d4-ch00-20260731010203/...
2 ---
3 creation_time: 2026-07-31T07:02:05+0000
4 name: gcp-book-labs-a1b2c3d4-ch00-20260731010203
5 storage_url: gs://gcp-book-labs-a1b2c3d4-ch00-20260731010203/
6
7 gcp-book-labs-a1b2c3d4-ch00-20260731010203 US-CENTRAL1 True
8 Removing gs://gcp-book-labs-a1b2c3d4-ch00-20260731010203/...

```

Listing 10: Representative output from the bucket verification script.

#### Note

A successful verification proves only that the basic learner path works. Later chapters may require additional APIs, roles, quotas, or service accounts. Return to this setup checklist whenever a lab fails before debugging the service itself.

## 0.8 Cross-Cloud Setup Equivalence

## Cross-Cloud Equivalence

| Setup step      | AWS                                       | Azure                                      | GCP                                                  |
|-----------------|-------------------------------------------|--------------------------------------------|------------------------------------------------------|
| Free offer      | Free Tier (12 mo + always free)           | Free account (\$200/30 days + always free) | Free trial (\$300/90 days + Always Free)             |
| Sign-up guard   | Card + root email                         | Card + Microsoft account                   | Card + Google account                                |
| Identity model  | IAM / IAM Identity Center                 | Microsoft Entra ID + RBAC                  | Cloud IAM                                            |
| Admin isolation | Never use root daily                      | Avoid Global Admin daily                   | Avoid Owner; least privilege                         |
| CLI             | AWS CLI v2 ( <code>aws configure</code> ) | Azure CLI ( <code>az login</code> )        | <code>gcloud CLI</code> ( <code>gcloud init</code> ) |
| Browser shell   | AWS CloudShell                            | Azure Cloud Shell                          | Cloud Shell                                          |
| Cost guardrail  | AWS Budgets + alerts                      | Cost Management budgets                    | Budgets & alerts                                     |
| Console         | AWS Management Console                    | Azure Portal                               | Google Cloud Console                                 |

Microsoft Fabric uses a Microsoft 365/Power BI work account plus a Fabric trial capacity (no card, org tenant); Snowflake uses a 30-day/\$400 trial (no card) guarded by Resource Monitors; MongoDB Atlas offers a forever-free M0 cluster (no card) secured by database users and an IP allowlist.

## 0.9 Setup Checklist for the Rest of the Book

Before starting Chapter 1, confirm the following:

1. The Google account used for labs has 2-Step Verification enabled.
2. The free trial is active, or the project is intentionally attached to an approved paid billing account.
3. A dedicated learner project exists and is selected in `gcloud config list`.
4. The project has a budget with alerts at 50%, 90%, and 100% of the chosen threshold.
5. The default region and zone are set to `us-central1` and `us-central1-a`, or to approved local alternatives.
6. The Cloud Storage API is enabled.
7. `gcloud auth list` shows the intended account as active.
8. The temporary bucket verification succeeded and the bucket was deleted.
9. You know how to open Cloud Shell if local PowerShell stops working.
10. You have recorded the project ID, billing account owner, region, and cleanup plan.

## 0.10 Further Reading

Read the Google Cloud free program page for the current free trial and Always Free terms [23]. Review the resource hierarchy and project documentation before designing multi-team environments [27]. Study Cloud IAM role types, service accounts, and least-privilege guidance before granting access [24]. Use the Google Cloud CLI documentation for installation, initialization, and command reference details [22]. Review Cloud Billing budgets, alerts, and billing

export before running long-lived labs [11]. The Google Cloud Architecture Framework provides broader operational, security, reliability, and cost review guidance [21].

## Key Takeaways

- Google Cloud’s free trial supplies temporary credits, while Always Free provides ongoing limited usage for eligible services.
- Projects are the practical unit of lab isolation, billing attribution, quota management, IAM scope, and cleanup.
- Least privilege, limited Owner grants, secure service accounts, and 2-Step Verification should be in place before data work begins.
- Windows learners can use the Google Cloud CLI from PowerShell; Cloud Shell is the no-install browser alternative.
- Budgets alert you, labels explain ownership, quotas constrain scale, and billing export makes costs queryable.
- Consistent regions, names, and labels prevent avoidable troubleshooting and cost-allocation problems.
- A setup is not complete until authentication, configuration, bucket creation, listing, and deletion have been verified.

## 0.11 Exercises

1. **(Checklist)** Complete every item in the setup checklist above. Record the project ID, active account, default region, and budget amount in your private lab notes.
2. **(Conceptual)** Explain the difference between a billing account and a project. Why can one billing account fund many projects while each project remains a useful isolation boundary?
3. **(Security)** Identify three reasons the Owner role should not be granted broadly. For each reason, name a safer predefined role or process.
4. **(Service accounts)** Describe why a downloaded JSON service-account key is riskier than an attached service account on a managed Google Cloud runtime.
5. **(PowerShell)** Run `gcloud auth list` and `gcloud config list`. Paste the output into private notes after removing email addresses if your notes may be shared.
6. **(Cost)** Create or verify a project-scoped budget with at least three alert thresholds. Explain why an alert is not the same as automatic shutdown.
7. **(Naming)** Propose names and labels for a raw bucket, a curated bucket, a Pub/Sub topic, and a Dataflow service account for a retail orders lab.
8. **(Hands-on)** Re-run the temporary bucket verification, then prove cleanup by listing buckets with the same prefix and confirming the verification bucket is gone.
9. **(Design)** Your organization requires data to remain in Canada. Rewrite this chapter’s region and naming assumptions for an approved Canadian region and explain which later labs may need location checks.

10. **(Reflection)** Write a five-sentence runbook for what you will do when a budget alert fires during the book.

## Part I

# Core Data Infrastructure and Storage

# Cloud Storage and Transfer Services

## Learning Objectives

- Explain the object storage model and why Cloud Storage is the default durable landing zone for many Google Cloud data platforms.
- Compare Standard, Nearline, Coldline, and Archive storage classes using access frequency, recovery expectations, and total cost.
- Choose among regional, dual-region, and multi-region bucket locations for analytics, availability, sovereignty, and disaster recovery requirements.
- Design lifecycle, versioning, soft delete, retention, and Bucket Lock controls that protect data without creating uncontrolled storage growth.
- Evaluate Storage Transfer Service, Transfer Appliance, `gcloud storage`, and `gsutil` for online and offline migrations.
- Implement a hands-on Cloud Storage bucket, lifecycle rule, transfer job, and Python client workflow.
- Design a multi-region disaster recovery architecture for ten petabytes of media assets with encryption, lifecycle policy, and cost governance.

## Cross-Cloud Equivalence

Although this book teaches **GCP**, the Week 1 storage foundations map almost one-to-one onto the other clouds—learn one mental model and carry it everywhere.

| Capability          | AWS                                     | Azure                                     | GCP                                       |
|---------------------|-----------------------------------------|-------------------------------------------|-------------------------------------------|
| Object storage      | Amazon S3                               | ADLS Gen2 / Blob                          | Cloud Storage                             |
| Hot tier            | S3 Standard                             | Hot                                       | Standard                                  |
| Infrequent access   | S3 Standard-IA                          | Cool / Cold                               | Nearline                                  |
| Archive (instant)   | Glacier Instant Retrieval               | Cold                                      | Coldline                                  |
| Deep archive        | Glacier Deep Archive                    | Archive                                   | Archive                                   |
| Lifecycle rules     | Lifecycle policies                      | Lifecycle management                      | Object Lifecycle Mgmt                     |
| Versioning          | S3 Versioning                           | Blob versioning                           | Object Versioning                         |
| Immutability (WORM) | Object Lock                             | Immutable blob storage                    | Bucket Lock                               |
| Cross-region copy   | CRR / SRR                               | Object replication (GRS)                  | Dual/multi-region + Transfer              |
| Online transfer     | DataSync                                | AzCopy                                    | Storage Transfer Service                  |
| Offline transfer    | Snow Family                             | Data Box                                  | Transfer Appliance                        |
| Design durability   | 11 nines                                | 11 nines (LRS) / 16 (GRS)                 | 11 nines                                  |
| Availability SLA    | 99.9% (Standard)                        | 99.9% (RA-GRS read 99.99%)                | 99.95% (multi-region)                     |
| Encryption at rest  | SSE-S3 / SSE-KMS / SSE-C                | SSE + CMK via Key Vault                   | Google-managed / CMEK / CSEK              |
| Access control      | IAM & bucket policies, ACLs             | Azure RBAC, SAS, POSIX ACLs               | IAM, ACLs, signed URLs                    |
| Pricing levers      | Storage + requests + egress + retrieval | Storage + operations + egress + retrieval | Storage + operations + egress + retrieval |
| Consistency         | Strong read-after-write                 | Strong read-after-write                   | Strong (global)                           |

Beyond raw object storage, the same layer reappears as a managed abstraction: **Fabric OneLake** is a tenant-wide lake built on ADLS Gen2 (Delta–Parquet), **Snowflake** persists data as immutable micro-partitions on the provider’s object store, and **MongoDB’s** WiredTiger is a node-local engine rather than object storage—the contrast is itself instructive.

### Key Terms

**Object:** immutable byte payload plus metadata addressed by bucket and object name. **Bucket:** globally unique namespace, location boundary, and policy container for objects. **Storage class:** pricing and retrieval behavior profile such as Standard, Nearline, Coldline, or Archive. **Lifecycle rule:** policy that transitions, deletes, or otherwise manages objects based on age, version, prefix, or state. **Retention policy:** minimum retention period that prevents deletion or replacement before a compliance deadline. **Storage Transfer Service:** managed service for scheduled, online transfers among clouds, URLs, file systems, and Cloud Storage.

## 1.1 Week 1 Roadmap

This chapter is the first week of the GCP Master Data Engineer program. It begins with object storage fundamentals, then adds governance and protection controls, then compares migration mechanisms, and closes with hands-on deployment and an architecture case. Cloud Storage appears simple because its core API is small: create a bucket, write objects, read objects, list objects, apply metadata, and delete objects. In production data engineering, however, that small API becomes the root of ingestion, data lake zoning, recovery, retention, compliance, and cost management. A poorly designed bucket hierarchy can leak data, strand objects in

expensive classes, slow downstream pipelines, or make disaster recovery impossible to test.

Cloud Storage is also the service where many engineers first encounter the design style of Google Cloud: global control plane, strong durability claims, location choices, uniform IAM, customer-managed encryption keys, event integration, and policy automation. The same architectural concerns reappear later in BigQuery, Dataflow, Dataproc, Pub/Sub, Dataplex, and Vertex AI. Therefore, the storage decisions made in Week 1 are not introductory trivia; they are the foundation of the rest of the book.

#### Definition

**Cloud Storage** is Google Cloud’s managed object storage service for durable, highly available storage of immutable objects. It is commonly used for data lakes, backups, media assets, application artifacts, analytical staging, model artifacts, and cross-system exchange [17].

## 1.2 Monday: Object Storage Fundamentals

### 1.2.1 Objects, Buckets, and Namespaces

Object storage differs from block storage and file storage. A block device gives an operating system addressable blocks. A file system gives applications hierarchical directories, permissions, and partial file operations. Object storage gives applications objects in buckets. Each object has a name, bytes, metadata, generation, checksum information, and access controls inherited or assigned through the surrounding policy model. The name may contain slashes, but those slashes are part of a flat object key rather than true directories.

This distinction matters in data engineering. Data lake paths such as `raw/events/date=2026-07-30/hour=21/` are conventions over object names. Query engines and pipeline code interpret prefixes as partitions, but Cloud Storage itself stores named objects. A rename is usually implemented as copy plus delete. A large recursive move is not a cheap metadata operation; it can be a large data movement.

#### Note

A bucket is both a namespace and an administrative boundary. Location, uniform bucket-level access, public access prevention, retention policy, lifecycle policy, default encryption behavior, and labels are bucket-level design choices. Object prefixes can organize data, but they do not replace careful bucket design.

The object model is well suited to analytical systems because files are normally written once and read many times. Pipeline tasks write immutable outputs, downstream engines scan them, and governance services catalog them. The model is less suited to workloads that require low-latency random updates to small records. Later chapters use Spanner and Bigtable for those patterns, reflecting lessons from Google’s distributed storage systems [2, 1]. The broader history of distributed storage includes the Google File System, which framed large-file storage for data processing at scale [3].

### 1.2.2 Consistency, Durability, and Availability

Cloud Storage is designed for very high durability through redundant storage of object data and metadata. Durability is the probability that stored data will not be lost. Availability is the probability that the service can successfully serve requests when clients need it. A system can

be durable but temporarily unavailable; data still exists, but an outage or network problem can prevent access. A data engineer must design for both.

#### Definition

**Durability** protects against data loss. **Availability** protects access to data during a time window. **Recovery time objective** (RTO) describes how quickly service must be restored; **recovery point objective** (RPO) describes how much data loss is acceptable.

Cloud Storage supports strong read-after-write and list consistency for object operations. That consistency simplifies ingestion workflows: once a writer receives success for a new object, downstream processes can discover and read it. Still, correctness requires idempotent pipeline design. A producer can retry after a network timeout and create duplicate names if names are not deterministic. A consumer can process the same object twice if its checkpointing is weak. Storage guarantees are necessary but not sufficient for exactly-once data engineering.

### 1.2.3 Storage Classes

Cloud Storage classes express access frequency and recovery economics. They are not merely labels. A class controls storage price, retrieval cost, minimum storage duration, and the operational expectation for how often objects are accessed. The four general-purpose classes are Standard, Nearline, Coldline, and Archive [17].

| Class    | Typical access          | Design intent                                   | Data engineering examples                                  |
|----------|-------------------------|-------------------------------------------------|------------------------------------------------------------|
| Standard | Frequent or low-latency | Active data and hot analytical staging          | Current raw zone, active feature data, recent media assets |
| Nearline | About monthly           | Infrequently accessed but occasionally restored | Monthly audit extracts, older raw partitions, warm backups |
| Coldline | About quarterly         | Disaster recovery and rarely read archives      | Quarterly compliance snapshots, DR copies                  |
| Archive  | About yearly            | Lowest-cost long-term preservation              | Legal hold exports, historical video masters, deep archive |

Table 1.1: Cloud Storage classes and common data engineering uses.

#### Common Pitfall

Choosing a colder class too early can increase total cost. Storage price may fall, but retrieval charges, operation charges, minimum storage duration, and pipeline delay can dominate if data is unexpectedly reprocessed. Model total cost using realistic replay scenarios, not only steady-state storage volume.

### 1.2.4 Location Types

A bucket location controls where object data is stored. Region buckets keep data in one Google Cloud region. Dual-region buckets store data in two specific regions. Multi-region buckets place data across a broad geographic area. The correct choice depends on the consuming compute location, regulatory requirements, disaster recovery targets, and data egress patterns.

Regional buckets are often best for pipelines whose compute is regional: a Dataflow job in `us-central1`, a Dataproc cluster in `us-central1`, or a BigQuery dataset in a compatible loca-

tion. Dual-region buckets are useful when an organization wants explicit geographic redundancy and control over the participating regions. Multi-region buckets simplify global availability for content distribution and cross-region analytics, but they can be the wrong fit when strict locality or predictable co-location with compute matters.

#### Tip

Pick the narrowest location that meets durability, availability, sovereignty, and recovery requirements. Wider geography can improve resilience and read locality for global consumers, but it may complicate compliance, downstream dataset location choices, and cost attribution.

### 1.2.5 Bucket Architecture for Data Lakes

A common pattern separates zones by trust and processing state: **raw**, **validated**, **curated**, and **sandbox**. The raw zone preserves source fidelity. The validated zone enforces schema and quality checks. The curated zone exposes analytics-ready data. The sandbox zone supports experiments with stricter expiration. In small projects, these zones can be prefixes in a bucket. In regulated or multi-team platforms, they often become separate buckets with separate IAM, retention, encryption, and lifecycle controls.

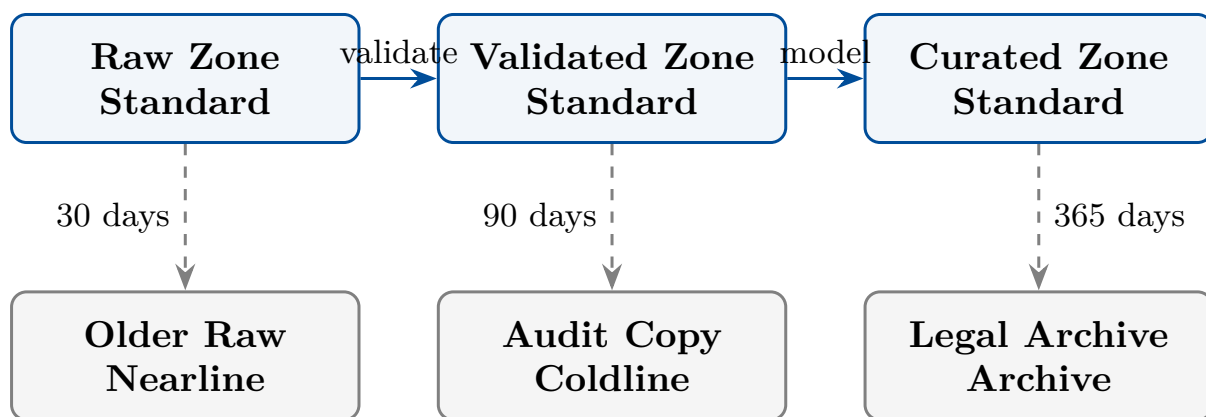


Figure 1.1: A storage-class lifecycle for a lakehouse-style Cloud Storage landing zone.

## 1.3 Tuesday: Lifecycle, Versioning, Retention, and Soft Delete

### 1.3.1 Object Lifecycle Management

Object Lifecycle Management automates storage hygiene. A lifecycle policy contains rules. Each rule has a condition and an action. Conditions can match age, creation date, storage class, object prefix or suffix, number of newer versions, and live or noncurrent state. Actions can change storage class or delete objects. Lifecycle policies are essential because manual cleanup does not scale to billions of objects.

For data platforms, lifecycle rules should be tied to data semantics. Raw event files may transition to Archive after they are outside the normal replay window. Temporary Dataflow staging files may be deleted after a few days. Curated analytical extracts may remain Standard for a quarter because business users frequently run year-to-date reports. A single global rule such as “archive everything after thirty days” is rarely correct for all prefixes.

**Note**

Lifecycle management is policy-as-code for storage economics. Review lifecycle rules with the same rigor as pipeline code: version them, test them in a non-production bucket, and connect each rule to a retention, recovery, or cost requirement.

### 1.3.2 Object Versioning

Object Versioning keeps noncurrent generations when a live object is overwritten or deleted. This protects against accidental replacement and application bugs, but it also increases storage consumption. Versioning is especially valuable for configuration files, small reference datasets, model artifacts, and high-value manifests. It is less attractive for high-volume append-only event files where object names are unique and overwrites should never happen.

**Common Pitfall**

Versioning without lifecycle controls can create hidden cost growth. Noncurrent versions are easy to forget because normal list operations emphasize live objects. Pair versioning with lifecycle rules that delete old noncurrent versions after the recovery window.

### 1.3.3 Retention Policies and Bucket Lock

A retention policy sets a minimum age before objects can be deleted or replaced. It can support compliance rules, audit preservation, and ransomware-resistant backups. Bucket Lock makes a retention policy permanent for the bucket. Once locked, the policy cannot be shortened or removed. This is intentionally irreversible because it is designed for compliance use cases.

**Definition**

**Bucket Lock** is an irreversible action that locks a retention policy on a bucket. It should be used only after legal, compliance, platform, and operations teams agree on the retention period and test the policy in a non-production environment.

Retention and lifecycle policies can interact. A lifecycle rule may attempt to delete an object, but retention can prevent deletion until the minimum period has passed. This is desirable when retention expresses a higher-priority compliance control. Engineers should document the expected precedence so that cost forecasts and cleanup jobs do not assume deletion will occur earlier.

### 1.3.4 Soft Delete

Soft delete provides a recovery window for deleted Cloud Storage objects. It is a safety mechanism against accidental or malicious deletion, distinct from object versioning and retention. Versioning preserves generations when objects change. Retention prevents deletion before a deadline. Soft delete allows recovery of deleted objects within a configured interval. The right combination depends on threat model and operational goals.

| Control           | Protects against                                    | Primary caution                                                         |
|-------------------|-----------------------------------------------------|-------------------------------------------------------------------------|
| Lifecycle policy  | Cost drift and unmanaged old data                   | Incorrect rules can move or delete data before business users expect it |
| Object Versioning | Accidental overwrite or delete of important objects | Noncurrent versions create hidden storage growth                        |
| Retention policy  | Premature deletion before compliance deadline       | Can block cleanup and increase storage cost                             |
| Bucket Lock       | Tamper-resistant compliance retention               | Irreversible; test before locking                                       |
| Soft delete       | Recent accidental or malicious deletion             | Recovery window is not a substitute for full backup and DR design       |

Table 1.2: Storage protection and governance controls.

## 1.4 Wednesday: Data Migration and Transfer Choices

### 1.4.1 Storage Transfer Service

Storage Transfer Service is a managed service for online transfers. It supports scheduled and one-time movement from sources such as Amazon S3, Azure Blob Storage, HTTP or HTTPS lists, POSIX file systems through agents, and Cloud Storage buckets [28]. It is preferable when data can move over the network within the migration window, when repeat synchronization is needed, or when the team wants managed retries, scheduling, logging, and incremental behavior.

For a data engineer, the important design questions are not only source and destination. Ask whether object metadata must be preserved, whether deletes should propagate, whether prefixes must be remapped, how credentials are scoped, how transfer logs are monitored, and what happens if the source changes during migration. A transfer job should have a reconciliation plan: expected object count, expected byte count, sample checksums, and a sign-off method.

### 1.4.2 Transfer Appliance

Transfer Appliance is for offline migration of large data volumes when network transfer is too slow, too costly, or operationally impossible [29]. Google ships a secure appliance, the customer loads data locally, and the appliance is shipped back for ingestion. Offline transfer changes the risk profile. It reduces dependence on wide-area bandwidth, but it introduces logistics, physical chain of custody, export scheduling, and a cutover plan for changed data.

#### Definition

**Online transfer** moves data over a network connection. **Offline transfer** moves data by physically shipping encrypted storage hardware. Online transfer favors repeatability and lower logistics; offline transfer favors very large initial loads when network time is unacceptable.

### 1.4.3 CLI Tools: `gcloud storage` and `gsutil`

Google Cloud provides both `gcloud storage` and `gsutil` workflows. `gcloud storage` is the modern Google Cloud CLI surface for storage operations. `gsutil` remains widely used in scripts and documentation, especially for advanced or legacy workflows. A professional data engineer should be able to read both, but new automation should prefer the supported direction for the organization and be explicit about tool versions.

**Tip**

For migrations, do not benchmark with a tiny file. Use representative object sizes, compression, count distribution, and network path. Millions of small objects often stress listing and per-object overhead more than raw bytes.

**1.4.4 Migration Decision Matrix**

| Option                   | Best fit                                                                | Watch for                                                      |
|--------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------|
| gcloud storage cp        | Small to medium ad hoc copies, scripted deployment, developer workflows | Local bandwidth, retries, local credentials, shell portability |
| gsutil rsync             | Legacy sync scripts and operational parity checks                       | Semantics of delete propagation and metadata differences       |
| Storage Transfer Service | Managed cloud-to-cloud or scheduled transfers                           | Credential scope, source mutation during transfer, monitoring  |
| Transfer Appliance       | Very large initial loads with poor network feasibility                  | Logistics, chain of custody, final delta sync                  |
| Partner tools            | Specialized enterprise migrations                                       | Licensing, support boundaries, auditability                    |

Table 1.3: Choosing a Cloud Storage migration mechanism.

**1.5 Thursday Hands-On Project: Bucket, Lifecycle, Transfer, and Client Code**

This project creates a Cloud Storage bucket, applies a lifecycle policy that transitions objects to Archive after thirty days, and demonstrates a Storage Transfer Service job from a public Amazon S3 bucket. The commands are realistic, but names and project identifiers must be changed before execution. Use a sandbox project, confirm billing expectations, and delete test resources after the lab.

**Note**

The placeholders `PROJECT_ID`, `BUCKET_NAME`, `REGION`, and `TRANSFER_JOB_NAME` are intentionally explicit. Replace them with values that satisfy your organization's naming and location standards.

**1.5.1 PowerShell Preparation**

The following commands use environment variables in a PowerShell-friendly form. The `gcloud storage` commands themselves can also be run in Cloud Shell with minor syntax changes.

```

1 # Replace these values before running.
2 $env:PROJECT_ID = "my-gcp-data-engineering-lab"
3 $env:BUCKET_NAME = "my-gcp-data-engineering-lab-raw"
4 $env:REGION = "US"
5
6 gcloud config set project $env:PROJECT_ID
7
8 gcloud storage buckets create gs://$env:BUCKET_NAME '
9   --location=$env:REGION '
10  --uniform-bucket-level-access
11
12 "event_id,event_ts,customer_id" | Set-Content -Path .\sample-events.csv

```

```

13 "1,2026-07-30T21:35:44Z,c001" | Add-Content -Path .\sample-events.csv
14
15 gcloud storage cp .\sample-events.csv gs://$env:BUCKET_NAME/raw/events/sample-
   events.csv
16
17 gcloud storage ls --long gs://$env:BUCKET_NAME/raw/events/

```

Listing 1.1: Creating a bucket and uploading a sample object with the Google Cloud CLI.

### 1.5.2 Lifecycle JSON

The lifecycle rule below transitions objects under selected prefixes to Archive after thirty days. In real platforms, use separate rules for raw, curated, temporary, and compliance prefixes.

```

1 {
2   "rule": [
3     {
4       "action": {
5         "type": "SetStorageClass",
6         "storageClass": "ARCHIVE"
7       },
8       "condition": {
9         "age": 30,
10        "matchesPrefix": [
11          "raw/",
12          "landing/"
13        ]
14      }
15    }
16  ]
17 }

```

Listing 1.2: Lifecycle policy that transitions matching objects to Archive after thirty days.

```

1 @'
2 {
3   "rule": [
4     {
5       "action": {
6         "type": "SetStorageClass",
7         "storageClass": "ARCHIVE"
8       },
9       "condition": {
10        "age": 30,
11        "matchesPrefix": ["raw/", "landing/"]
12      }
13    }
14  ]
15 }
16 '@ | Set-Content -Path .\lifecycle-archive-30-days.json
17
18 gcloud storage buckets update gs://$env:BUCKET_NAME '
19   --lifecycle-file=.\lifecycle-archive-30-days.json
20
21 gcloud storage buckets describe gs://$env:BUCKET_NAME '
22   --format="yaml(location,storageClass,uniformBucketLevelAccess,
   lifecycle_config)"

```

Listing 1.3: Applying and verifying the lifecycle policy.

### 1.5.3 Storage Transfer Job from Public S3

Public S3 sources change over time. Treat the bucket name and prefix below as lab placeholders and confirm that the source permits public reads before running. In production, use scoped credentials and a dedicated service account for the transfer job.

```

1 $env:TRANSFER_JOB_NAME = "public-s3-to-gcs-lab"
2 $env:SOURCE_S3_BUCKET = "commoncrawl"
3 $env:SOURCE_PREFIX = "crawl-data/CC-MAIN-2024-10/warc.paths.gz"
4
5 gcloud transfer jobs create '
6   s3://$env:SOURCE_S3_BUCKET/$env:SOURCE_PREFIX '
7   gs://$env:BUCKET_NAME/transfers/commoncrawl/ '
8   --name=$env:TRANSFER_JOB_NAME '
9   --description="Lab transfer from public S3 object to Cloud Storage" '
10  --source-creds-file="" '
11  --schedule-starts=$(Get-Date -Format yyyy-MM-dd) '
12  --schedule-repeats-every=1d
13
14 gcloud transfer jobs list --filter="name:$env:TRANSFER_JOB_NAME"

```

Listing 1.4: Creating a Storage Transfer Service job from a public Amazon S3 bucket.

If the CLI version in your environment does not allow an empty credentials file for a public S3 source, create the job through the Google Cloud console or use the Storage Transfer Service API with an empty AWS credentials block according to the current documentation. The architectural lesson is the same: use a managed transfer job when the source, schedule, retry behavior, and monitoring should be controlled by a service rather than a laptop shell.

### 1.5.4 Python Client Snippet

The Python client library is useful for applications and tests that need to create objects, read metadata, or assert bucket policy. Install `google-cloud-storage` in an isolated environment and authenticate with Application Default Credentials.

```

1 from google.cloud import storage
2
3
4 def upload_text(project_id: str, bucket_name: str, object_name: str, text: str)
5     -> None:
6     client = storage.Client(project=project_id)
7     bucket = client.bucket(bucket_name)
8     blob = bucket.blob(object_name)
9     blob.upload_from_string(text, content_type="text/csv")
10    blob.reload()
11    print({
12        "bucket": bucket.name,
13        "object": blob.name,
14        "generation": blob.generation,
15        "storage_class": blob.storage_class,
16        "size": blob.size,
17    })
18
19 if __name__ == "__main__":
20     upload_text(
21         project_id="my-gcp-data-engineering-lab",
22         bucket_name="my-gcp-data-engineering-lab-raw",
23         object_name="raw/events/python-client-sample.csv",
24         text="event_id,event_ts\n2,2026-07-30T21:40:00Z\n",
25     )

```

Listing 1.5: Uploading an object and reading metadata with the Python Cloud Storage client.

### 1.5.5 Validation Checklist

A completed lab should show the bucket exists, uniform bucket-level access is enabled, the lifecycle configuration is attached, the sample object exists, the transfer job is visible, and the Python client can upload or inspect an object. For production change control, add labels such as `env=lab`, `owner=data-engineering`, and `cost-center=training`. Labels make cost reports and cleanup safer.

## 1.6 Friday Use Case and Architecture: Multi-Region DR for Media Streaming

### 1.6.1 Scenario

A media streaming company stores ten petabytes of video assets. The library contains raw mezzanine files, encoded renditions, thumbnails, subtitles, manifests, and analytical logs. Customers stream globally. Editors and encoding pipelines are regionally concentrated. The company wants low risk of data loss, fast recovery from regional disruption, encryption with customer-managed keys, lifecycle-based cost control, and evidence that disaster recovery works.

The architecture must separate content serving from archival preservation. Hot renditions used by active customers should remain in Standard class and close to content delivery paths. Original masters may transition to colder classes after quality control and contractual windows. Temporary transcoding outputs should expire quickly. Compliance and rights-management records may require retention policies. Analytics logs should land in Cloud Storage and flow to BigQuery, whose Dremel roots explain its analytical execution model [30, 7].

### 1.6.2 Design

Use a multi-region or dual-region bucket strategy for serving assets, with explicit replication requirements for masters. For example, active assets can live in a US multi-region bucket when customer demand is broad and regulatory constraints permit it. A dual-region bucket pairing two selected regions can provide clearer DR placement. Use customer-managed encryption keys (CMEK) with Cloud KMS keys in appropriate locations, and define key rotation and incident procedures. Use separate buckets for masters, encoded renditions, temporary work, logs, and public delivery integration rather than one universal bucket.

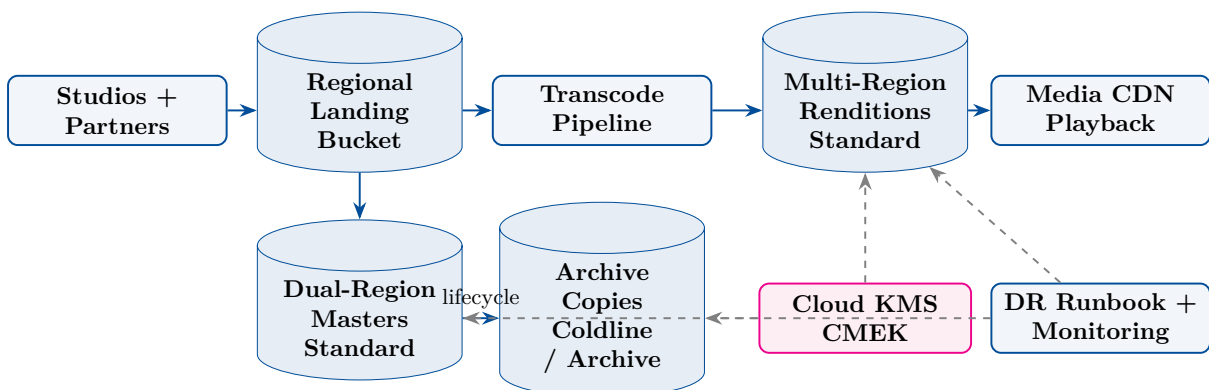


Figure 1.2: A multi-region disaster recovery architecture for ten petabytes of media assets.

### 1.6.3 Cost and Operations

Ten petabytes changes every decision. A one percent mistake is one hundred terabytes. Lifecycle rules should be simulated against inventory reports before rollout. Archive transitions should consider contractual replay requirements, re-encoding probability, content popularity, and legal hold. For active renditions, cache and CDN strategy may reduce object read pressure, but it does not remove the need for source-of-truth storage durability. For masters, DR tests should restore a statistically meaningful sample and verify checksums, metadata, KMS permissions, and downstream playback or transcoding compatibility.

#### Tip

Use labels, bucket inventory, access logs, and billing export to BigQuery for storage FinOps. A weekly report should identify top buckets by bytes, operation count, retrieval fees, old temporary prefixes, noncurrent versions, and lifecycle exceptions.

#### Common Pitfall

A multi-region bucket is not a complete disaster recovery plan. DR also requires IAM continuity, key availability, tested restore procedures, pipeline redeployment, DNS or delivery failover, monitoring, and people who know the runbook.

## 1.7 Governance and Security Design Notes

Cloud Storage security begins with identity and access management. Prefer uniform bucket-level access for most data engineering buckets so that IAM is the policy model. Avoid object ACLs unless a legacy integration requires them. Enable public access prevention for private data. Use service accounts with least privilege for Dataflow jobs, Dataproc clusters, Composer environments, transfer jobs, and application writers. Separate writer, reader, operator, and auditor roles. For sensitive data, combine CMEK, restricted networks, audit logs, and governance with Dataplex where appropriate [20].

Data classification should be visible in bucket names, labels, and catalog metadata. A bucket named for a raw ingestion zone should not also host public assets. A bucket containing regulated data should have explicit retention, access review cadence, and incident response owner. The Google Cloud Architecture Framework provides a useful structure for reviewing reliability, security, cost, performance, and operations across these decisions [21].

#### Note

Storage architecture is a socio-technical system. The bucket policy may be correct, but if naming, ownership, documentation, and monitoring are unclear, teams will create unsafe workarounds.

## 1.8 Operational Runbook for Week 1 Buckets

Every production bucket should have an operational runbook before it receives critical data. The runbook does not need to be long, but it must answer who owns the bucket, what data classification it contains, which systems write to it, which systems read from it, which lifecycle rules are expected to fire, and how incidents are handled. A bucket with no owner becomes a landfill. A bucket with no deletion policy becomes a cost risk. A bucket with no restore test becomes a false sense of safety.

**Tip**

For each important bucket, keep a one-page record: purpose, owner, location, storage class strategy, CMEK status, lifecycle rules, retention controls, service accounts, expected monthly growth, recovery procedure, and last restore-test date.

The simplest useful monitoring pattern exports Cloud Audit Logs and billing data to BigQuery, then builds a weekly review. The review should flag sudden growth, unusual write principals, public-access changes, lifecycle failures, objects stuck in temporary prefixes, high retrieval fees from cold classes, and buckets with no labels. Later chapters will connect these signals to Cloud Monitoring, Dataplex data quality, and FinOps dashboards.

| Signal                     | Question it answers                             | Possible response                                               |
|----------------------------|-------------------------------------------------|-----------------------------------------------------------------|
| Bytes by bucket and prefix | Is storage growing as forecast?                 | Investigate source volume, lifecycle rules, or duplicate writes |
| Operations by principal    | Who is reading or writing data?                 | Review IAM, service accounts, and unexpected clients            |
| Cold-class retrieval fees  | Are archived objects being re-stored too often? | Reconsider class transition timing or replay design             |
| Lifecycle action counts    | Are policies firing as expected?                | Test rule conditions and retention interactions                 |
| Unlabeled buckets          | Can costs and ownership be assigned?            | Enforce labels through provisioning policy                      |

Table 1.4: Operational signals for Cloud Storage runbooks.

## 1.9 Interview Q&A

### 1.9.1 Concepts and Trade-offs

1. **Q: When would you choose a regional bucket over a multi-region bucket?**

A: Choose a regional bucket when compute and analytics are regional, data residency requires a specific region, or lower locality complexity matters more than broad geographic availability. Choose wider geography only when it satisfies a concrete availability or access requirement.

2. **Q: Why can Archive storage be more expensive than Standard for some workloads?**

A: Archive reduces storage price but adds retrieval and minimum-duration considerations. If data is frequently restored, reprocessed, or migrated, the total cost can exceed Standard or Nearline.

3. **Q: How do versioning, retention, and soft delete differ?**

A: Versioning preserves older generations, retention prevents premature deletion or overwrite, and soft delete provides a recovery window after deletion. They solve related but distinct failure modes.

4. **Q: How would you validate a petabyte-scale transfer?**

A: Compare expected and actual bytes, object counts, sampled checksums, metadata, prefix coverage, transfer logs, and downstream readability. Also test incremental synchronization for changed source data.

## 1.10 Further Reading

Read the Cloud Storage overview for the current behavior of buckets, locations, classes, lifecycle, retention, and soft delete [17]. Review the Google Cloud Architecture Framework for reliability, security, cost, performance, and operations review prompts [21]. Study Storage Transfer Service and Transfer Appliance documentation before designing migrations [28, 29]. For distributed-storage context, read the Google File System paper [3]. For the analytical systems that often consume Cloud Storage data, read Dremel and BigQuery documentation [30, 7]. For governance direction, consult Dataplex documentation [20]. Dataflow documentation is useful when object storage becomes the source or sink for Apache Beam pipelines [19].

## Key Takeaways

- Cloud Storage is the durable object foundation for many GCP data platforms, but bucket design is an architectural decision, not a default checkbox.
- Storage class selection must account for retrieval, operations, minimum duration, and replay probability, not only monthly storage price.
- Location type should be chosen from compute locality, sovereignty, availability, and disaster recovery requirements.
- Lifecycle policies, versioning, retention, Bucket Lock, and soft delete each protect against different risks and can interact.
- Storage Transfer Service is the default managed option for online transfer; Transfer Appliance is for large offline migrations where network transfer is not feasible.
- A working DR architecture includes IAM, KMS, lifecycle rules, monitoring, restore tests, and runbooks in addition to replicated storage.

## 1.11 Exercises

1. **(Conceptual)** Explain the difference between durability and availability. Give one example of a system that is durable but temporarily unavailable.
2. **(Conceptual)** A team wants to store all raw data in Archive immediately after ingestion. Identify three questions you would ask before approving the design.
3. **(Hands-on)** Create a lab bucket with uniform bucket-level access and upload three sample objects under `raw/`, `validated/`, and `tmp/`. Add labels for owner, environment, and cost center.
4. **(Hands-on)** Write a lifecycle policy that deletes `tmp/` objects after seven days, transitions `raw/` objects to Nearline after thirty days, and deletes noncurrent versions after fourteen days.
5. **(Hands-on)** Use `gcloud storage` or `gsutil` to compare object listings before and after enabling versioning. Document what changes in the output.
6. **(Design)** Choose regional, dual-region, or multi-region locations for a healthcare analytics platform with strict residency requirements and explain the trade-off.
7. **(Design)** Create a migration plan for two hundred terabytes in S3 and ten petabytes in an on-premises NAS. Decide which transfer mechanisms you would use and how you would validate completion.

8. **(Architecture)** Extend the media streaming DR design with BigQuery billing export, alerting, and a quarterly restore test. Define the evidence you would present in an architecture review.

## 1.12 Capstone Project: A Resilient Global Media Asset Store on Cloud Storage

### Capstone Project

**Mission:** Design and prototype a resilient Cloud Storage foundation for a global media-streaming company. Your solution must ingest source assets from another cloud, protect irreplaceable video masters, serve globally distributed consumers, and control the cost of a ten-petabyte library without weakening disaster recovery.

**Estimated time:** 4–6 hours.

**What you'll practice:**

- Choosing a multi-region location and storage classes for hot, warm, cold, and archive media assets.
- Implementing lifecycle transitions from Standard to Nearline, Coldline, and Archive.
- Enabling Object Versioning, a retention policy, and Bucket Lock for WORM-style protection.
- Applying CMEK encryption and least-privilege IAM for operators, transfer jobs, and viewers.
- Creating a Storage Transfer Service design that imports from a public Amazon S3 source into Cloud Storage.
- Producing an architecture diagram, runbook evidence, and acceptance criteria that an architecture review board can evaluate.

### Scenario

StreamForge Media operates a global streaming platform with roughly 10 PB of video assets. The library includes mezzanine source files from studios, encoded renditions for adaptive bitrate playback, captions, thumbnails, manifests, quality-control reports, and rights-management metadata. Customer playback is global, while encoding teams and editorial workflows are concentrated in North America. The company wants a storage design that keeps popular assets close to serving paths, preserves masters against accidental deletion and ransomware, and provides a tested disaster recovery story for regional disruption.

The business problem is not simply “store 10 PB cheaply.” The active catalog must remain available during high-traffic premieres. Older content must transition to colder classes only when retrieval cost and restore time remain acceptable. Legal and licensing records must be immutable for the required period. Source assets arrive from partners, including a public AWS S3 bucket used for this lab. Security leadership also requires customer-managed encryption keys, public access prevention, auditability, and service accounts with narrow permissions.

### Requirements

Your capstone design must include the following controls and explain the trade-offs behind each one:

- Create a Cloud Storage bucket in a multi-region location for globally resilient media assets.
- Use a lifecycle policy that transitions objects from Standard to Nearline, then Coldline, then Archive based on age and prefix.

- Enable Object Versioning so accidental overwrites and deletes can be recovered within an operational window.
- Configure a retention policy and lock it with Bucket Lock after validation, creating a WORM control for protected masters.
- Use CMEK encryption with a Cloud KMS key whose location is compatible with the bucket location.
- Define a Storage Transfer Service job that imports selected objects from a public AWS S3 bucket into the destination bucket.
- Apply least-privilege IAM: separate transfer, media-operator, auditor, and serving identities rather than granting broad project roles.
- Document cost controls for 10 PB, including labels, lifecycle review, noncurrent-version cleanup, and cold-class retrieval assumptions.

## Reference Architecture

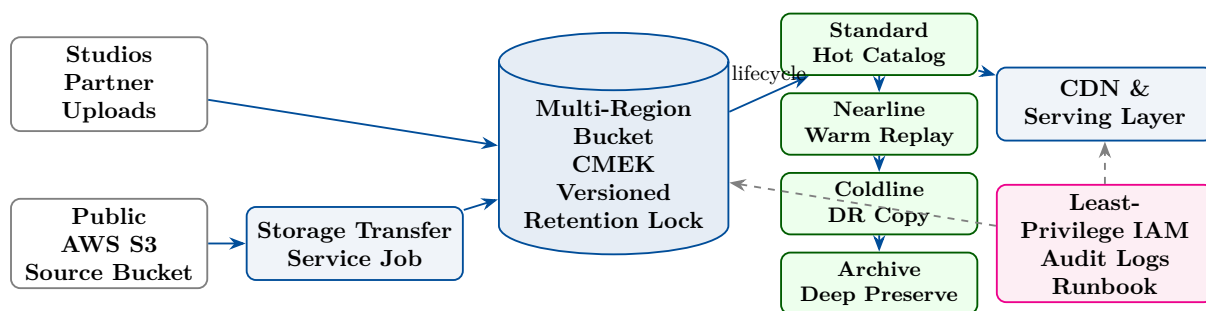


Figure 1.3: Capstone reference architecture for a resilient global media asset store on Cloud Storage.

## Implementation Steps

1. **Prepare project variables and create the bucket.** Use a sandbox project first. The examples are PowerShell-friendly and use a multi-region location. Replace every placeholder before execution.

```

1 $env:PROJECT_ID = "my-gcp-media-capstone"
2 $env:BUCKET_NAME = "my-gcp-media-capstone-assets"
3 $env:LOCATION = "US"
4 $env:KMS_RING = "media-assets-ring"
5 $env:KMS_KEY = "media-assets-key"
6 $env:KMS_LOCATION = "us"
7 $env:TRANSFER_SA = "media-transfer-sa"
8 $env:OPS_GROUP = "gcs-media-operators@example.com"
9 $env:AUDIT_GROUP = "gcs-media-auditors@example.com"
10
11 gcloud config set project $env:PROJECT_ID
12
13 gcloud kms keyrings create $env:KMS_RING '
14   --location=$env:KMS_LOCATION
15
16 gcloud kms keys create $env:KMS_KEY '
17   --location=$env:KMS_LOCATION '
18   --keyring=$env:KMS_RING '

```

```

19  --purpose=encryption
20
21  gcloud storage buckets create gs://$env:BUCKET_NAME '
22    --location=$env:LOCATION '
23    --uniform-bucket-level-access '
24    --public-access-prevention '
25    --default-encryption-key=projects/$env:PROJECT_ID/locations/$env:
      KMS_LOCATION/keyRings/$env:KMS_RING/cryptoKeys/$env:KMS_KEY '
26    --labels=env=capstone,workload=media-assets,owner=data-engineering
27
28  gcloud iam service-accounts create $env:TRANSFER_SA '
29    --display-name="Media transfer service account"
30
31  gcloud storage buckets add-iam-policy-binding gs://$env:BUCKET_NAME '
32    --member="serviceAccount:$env:TRANSFER_SA@$env:PROJECT_ID.iam.
      gserviceaccount.com" '
33    --role="roles/storage.objectCreator"
34
35  gcloud storage buckets add-iam-policy-binding gs://$env:BUCKET_NAME '
36    --member="group:$env:OPS_GROUP" '
37    --role="roles/storage.objectAdmin"
38
39  gcloud storage buckets add-iam-policy-binding gs://$env:BUCKET_NAME '
40    --member="group:$env:AUDIT_GROUP" '
41    --role="roles/storage.objectViewer"

```

Listing 1.6: PowerShell setup, CMEK key, bucket creation, and least-privilege IAM.

2. **Attach lifecycle policy and enable versioning.** Keep current active renditions in Standard, move warm objects to Nearline after 30 days, cold DR copies to Coldline after 90 days, and preserved masters to Archive after 365 days. Delete old noncurrent versions after the recovery period so versioning does not create unbounded cost.

```

1  {
2    "rule": [
3      {
4        "action": {"type": "SetStorageClass", "storageClass": "NEARLINE"},
5        "condition": {"age": 30, "matchesPrefix": ["renditions/", "masters/"]}
6      },
7      {
8        "action": {"type": "SetStorageClass", "storageClass": "COLDLINE"},
9        "condition": {"age": 90, "matchesPrefix": ["masters/", "dr/"]}
10     },
11     {
12       "action": {"type": "SetStorageClass", "storageClass": "ARCHIVE"},
13       "condition": {"age": 365, "matchesPrefix": ["masters/", "legal/"]}
14     },
15     {
16       "action": {"type": "Delete"},
17       "condition": {"isLive": false, "numNewerVersions": 3, "age": 45}
18     },
19     {
20       "action": {"type": "Delete"},
21       "condition": {"age": 7, "matchesPrefix": ["tmp/", "transcode-staging/"]}
22     }
23   ]
24 }

```

Listing 1.7: Lifecycle policy for hot, warm, cold, archive, and noncurrent media versions.

```

2 {
3   "rule": [
4     {
5       "action": {"type": "SetStorageClass", "storageClass": "NEARLINE"},
6       "condition": {"age": 30, "matchesPrefix": ["renditions/", "masters
7     ]},
8     {
9       "action": {"type": "SetStorageClass", "storageClass": "COLDLINE"},
10      "condition": {"age": 90, "matchesPrefix": ["masters/", "dr/"]}
11    },
12    {
13      "action": {"type": "SetStorageClass", "storageClass": "ARCHIVE"},
14      "condition": {"age": 365, "matchesPrefix": ["masters/", "legal/"]}
15    },
16    {
17      "action": {"type": "Delete"},
18      "condition": {"isLive": false, "numNewerVersions": 3, "age": 45}
19    },
20    {
21      "action": {"type": "Delete"},
22      "condition": {"age": 7, "matchesPrefix": ["tmp/", "transcode-staging
23    ]}
24  ]
25 }
26 '@ | Set-Content -Path .\media-lifecycle.json
27
28 gcloud storage buckets update gs://$env:BUCKET_NAME '
29   --lifecycle-file=.\media-lifecycle.json
30
31 gsutil versioning set on gs://$env:BUCKET_NAME
32
33 gsutil retention set 365d gs://$env:BUCKET_NAME
34
35 gsutil retention get gs://$env:BUCKET_NAME
36
37 # Irreversible: run only after policy review and non-production testing.
38 gsutil retention lock gs://$env:BUCKET_NAME
39
40 gcloud storage buckets describe gs://$env:BUCKET_NAME '
41   --format="yaml(location,defaultKmsKeyName,versioning,retentionPolicy,
42     lifecycle_config)"

```

Listing 1.8: Applying lifecycle policy, versioning, retention policy, and Bucket Lock.

3. **Define the transfer job from public AWS S3.** In a production migration, credentials should be scoped to the minimum readable prefix. This lab models a public source, imports to a restricted prefix, and avoids delete propagation from the source.

```

1 from google.cloud import storage_transfer
2
3 project_id = "my-gcp-media-capstone"
4 destination_bucket = "my-gcp-media-capstone-assets"
5
6 client = storage_transfer.StorageTransferServiceClient()
7
8 transfer_job = {
9   "description": "Capstone public S3 media seed import",
10  "project_id": project_id,
11  "status": storage_transfer.TransferJob.Status.ENABLED,
12  "transfer_spec": {
13    "aws_s3_data_source": {

```

```

14         "bucket_name": "commoncrawl",
15         "path": "crawl-data/CC-MAIN-2024-10/"
16     },
17     "gcs_data_sink": {
18         "bucket_name": destination_bucket,
19         "path": "imports/public-s3/commoncrawl/"
20     },
21     "transfer_options": {
22         "overwrite_objects_already_existing_in_sink": False,
23         "delete_objects_unique_in_sink": False
24     }
25 },
26 "schedule": {
27     "schedule_start_date": {"year": 2026, "month": 7, "day": 31},
28     "schedule_end_date": {"year": 2026, "month": 7, "day": 31}
29 }
30 }
31
32 response = client.create_transfer_job(request={"transfer_job": transfer_job
33                                     })
34 print(response.name)

```

Listing 1.9: Storage Transfer Service job body for a public S3 source.

4. **Validate and document operations.** Upload a small marker object under `masters/`, overwrite it once to prove versioning, verify retention status, inspect lifecycle configuration, and record the transfer job name. Do not test deletion in the locked production bucket; use a disposable bucket for destructive validation.

## Deliverables

- A one-page architecture summary explaining the bucket location, lifecycle design, retention period, CMEK key location, and IAM model.
- The completed reference diagram or an equivalent diagram with sources, transfer service, Cloud Storage bucket controls, lifecycle classes, and CDN or serving layer.
- Command transcript or screenshots proving bucket creation, versioning, lifecycle policy, retention policy, CMEK, IAM bindings, and transfer job configuration.
- A 10 PB cost model with assumptions for active catalog size, monthly growth, retrieval frequency, noncurrent versions, and Archive restore scenarios.
- A DR runbook excerpt that states RTO, RPO, restore test cadence, sample-checksum process, KMS dependency, and escalation owners.

## Acceptance Criteria

- The design uses a multi-region Cloud Storage bucket and explains why it is preferred over regional or dual-region for this scenario.
- Lifecycle rules cover Standard to Nearline to Coldline to Archive transitions and include cleanup for temporary prefixes or noncurrent versions.
- Object Versioning is enabled and paired with a bounded noncurrent-version policy.
- The retention policy and Bucket Lock decision are documented as irreversible, reviewed, and tested before production use.
- CMEK is configured with a compatible Cloud KMS location, and the operational impact of key disablement is documented.

- Storage Transfer Service imports from a public AWS S3 source or equivalent lab source and writes to a controlled destination prefix.
- IAM grants are least privilege and avoid broad project Owner, Editor, or Viewer roles for routine storage operations.
- The cost model addresses 10 PB scale, retrieval charges, lifecycle minimum durations, and at least one DR restore event.
- The runbook includes validation evidence: object counts, byte counts, sampled checksums, transfer logs, audit logs, and restore-test results.

### Stretch Goals

- Evaluate Autoclass for prefixes where access patterns are unpredictable, and compare it with explicit lifecycle rules.
- Replace the multi-region bucket with a dual-region design that uses turbo replication, then compare RPO, cost, and sovereignty trade-offs.
- Build a BigQuery billing-export dashboard that estimates monthly storage, operation, retrieval, and replication costs for 10 PB.
- Add a Cloud DLP or Sensitive Data Protection scan for captions, metadata, and support files that might contain regulated information.
- Create an inventory-based reconciliation job that compares expected and actual object counts, byte totals, generations, and storage classes.
- Add a quarterly game-day plan that disables a non-production KMS key, restores sample media from Archive, and verifies playback through the serving layer.

# Relational Databases: Cloud SQL and AlloyDB

## Learning Objectives

- Explain Cloud SQL architecture for managed PostgreSQL, MySQL, and SQL Server workloads in Google Cloud.
- Design regional high availability, automatic failover, backups, point-in-time recovery, and read replicas for relational systems.
- Use Database Migration Service and private IP connectivity to move source databases into Cloud SQL with minimal downtime.
- Describe AlloyDB for PostgreSQL, including its disaggregated storage, columnar engine, and machine learning integrations.
- Implement an HA Cloud SQL for PostgreSQL instance, create a read replica, and simulate failover using PowerShell-friendly `gcloud` commands.
- Build Python client checks that validate primary and replica connectivity during a hands-on relational database lab.
- Design a zero-downtime ERP migration to Cloud SQL with an AlloyDB analytics replica for operational reporting.

## Cross-Cloud Equivalence

Managed relational services use different product names, but the architectural vocabulary is portable: engine compatibility, HA placement, replicas, migration tooling, and analytics acceleration.

| Capability               | AWS                           | Azure                                     | GCP                                          |
|--------------------------|-------------------------------|-------------------------------------------|----------------------------------------------|
| Managed relational       | RDS / Aurora                  | Azure SQL / Azure Database for PostgreSQL | Cloud SQL / AlloyDB                          |
| Scale-out or accelerated | Aurora                        | Hyperscale                                | AlloyDB columnar engine                      |
| High availability        | Multi-AZ                      | Zone-redundant HA                         | Regional HA                                  |
| Read replicas            | RDS replicas / Aurora readers | Read replicas / geo-replicas              | Cloud SQL read replicas / AlloyDB read pools |
| Migration                | DMS + SCT                     | Azure DMS                                 | Database Migration Service                   |

MongoDB Atlas, Microsoft Fabric, and Snowflake are useful analogs but not direct replacements. MongoDB Atlas targets document workloads, Fabric combines analytics services

over OneLake, and Snowflake separates compute from storage for warehousing. Cloud SQL and AlloyDB remain the default GCP choices when the source of truth is relational and transactional.

### Key Terms

**Cloud SQL:** Google Cloud's managed relational database service for PostgreSQL, MySQL, and SQL Server. **Regional HA:** Cloud SQL configuration with a standby instance in another zone of the same region. **Read replica:** asynchronously replicated instance used for read scaling, reporting, or regional read locality. **Database Migration Service:** managed service for migrating databases to Cloud SQL and AlloyDB with continuous replication. **Private IP:** connectivity pattern that keeps database traffic on private Google Cloud networking rather than public addresses. **AlloyDB:** PostgreSQL-compatible managed database designed for high performance, availability, and analytics acceleration.

## 2.1 Week 2 Roadmap

Week 2 moves from object storage into the transactional systems that feed analytical platforms. Most data engineering projects begin with operational data in relational databases: customers, orders, invoices, inventory, account balances, workflow states, and audit events. A professional data engineer must understand how those systems stay available, how replicas lag, how migrations preserve consistency, and how analytical offloading prevents reporting from hurting online transactions.

This chapter uses Cloud SQL for managed relational operations and AlloyDB for PostgreSQL-compatible workloads that need higher throughput or embedded analytical acceleration. Monday establishes Cloud SQL architecture, high availability, and replicas. Tuesday focuses on Database Migration Service and private IP networking. Wednesday introduces AlloyDB, its columnar engine, and machine learning integration. Thursday builds an HA PostgreSQL lab with a replica and failover drill. Friday applies the patterns to a zero-downtime ERP migration.

### Definition

**Cloud SQL** is a fully managed relational database service that handles provisioning, maintenance, backups, replication, and high availability for supported database engines. It reduces undifferentiated administration, but schema design, transaction boundaries, connection management, and migration correctness remain engineering responsibilities [15].

## 2.2 Monday: Cloud SQL Architecture, High Availability, and Read Replicas

### 2.2.1 Managed Architecture

Cloud SQL wraps familiar relational engines with a managed control plane. You still choose an engine version, machine shape, storage size, region, network access, backup window, maintenance window, and database flags. Google Cloud manages host provisioning, patch orchestration, storage growth when configured, monitoring integration, backups, and failover mechanics. Applications connect through standard database protocols, the Cloud SQL Auth Proxy, private IP, public IP with authorized networks, or language connectors.

**Note**

Managed does not mean ownerless. Application teams still own schema migrations, query efficiency, index strategy, connection pooling, transaction isolation choices, failover testing, and business recovery procedures.

For data engineers, Cloud SQL often appears in three roles. First, it is an operational source that exports change data into pipelines. Second, it is a serving store for metadata, workflow state, and small application datasets. Third, it is a migration target when an on-premises database must move quickly without replatforming. In all three roles, reliability comes from combining database features with network design, IAM, monitoring, and runbooks.

### 2.2.2 Regional High Availability

Cloud SQL regional high availability creates a primary instance and a standby instance in different zones of the same region. The standby is not a read target. It exists to take over when the primary zone or host becomes unhealthy. Persistent disk replication and managed failover are designed to reduce recovery time compared with restoring from backup, but applications must still reconnect and retry safely.

**Definition**

**High availability** improves service continuity after infrastructure failure. It is not the same as disaster recovery. Regional HA protects against zonal failure inside one region; cross-region DR requires backups, replicas, export plans, or a separate architecture.

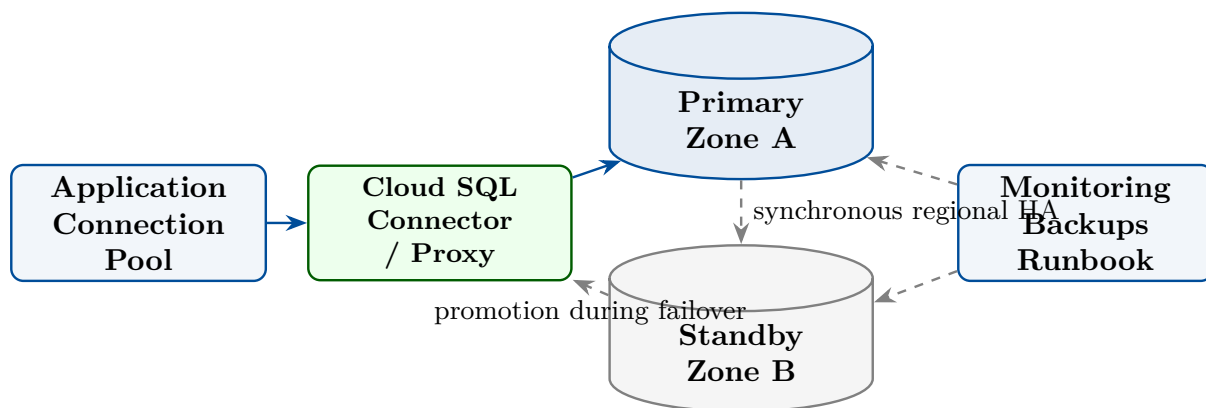


Figure 2.1: Cloud SQL regional HA places a standby in another zone and requires client reconnection after failover.

**Common Pitfall**

Failover testing often reveals application bugs, not database bugs. Hard-coded IP addresses, missing retry logic, oversized connection pools, long transactions, and non-idempotent writes can turn a managed failover into an application outage.

### 2.2.3 Backups and Point-in-Time Recovery

Backups protect against logical failure: accidental deletes, destructive deployments, corrupt data loads, or ransomware. High availability keeps the service running after infrastructure failure, but it faithfully replicates bad transactions. Point-in-time recovery uses write-ahead

logs or binary logs to restore to a selected moment inside the retention window. Production designs should document backup retention, restore targets, restore testing cadence, encryption, and who can initiate recovery.

#### Tip

Treat restore tests as a production control. A backup that has never been restored is an assumption. Schedule a non-production restore, run data quality checks, and record elapsed time against the recovery time objective.

### 2.2.4 Read Replicas

Cloud SQL read replicas use asynchronous replication. They improve read scale, isolate analytical queries, and can provide regional read locality. They do not guarantee zero-lag reads. A reporting dashboard that reads from a replica may show slightly stale results, which is usually acceptable for analytics but dangerous for payment confirmation, inventory decrement, or idempotency checks.

| Pattern              |      | Best fit                                      | Caution                                              |
|----------------------|------|-----------------------------------------------|------------------------------------------------------|
| Regional standby     | HA   | Zonal failure protection for primary database | Standby is not readable and does not replace backups |
| Same-region replica  | read | Reporting, read scale, operational dashboards | Replica lag and extra cost                           |
| Cross-region replica | read | Geographic read locality or DR seed           | Network cost, lag, and promotion runbook             |
| Backup restore       |      | Logical recovery or environment cloning       | Restore time and validation effort                   |

Table 2.1: Cloud SQL reliability and scale patterns.

## 2.3 Tuesday: Database Migration Service and Private IP Connections

### 2.3.1 Migration Phases

Database Migration Service (DMS) supports migrations into Cloud SQL and AlloyDB by combining initial load with change replication for compatible sources [18]. A typical migration has five phases: assessment, connectivity, full load, continuous replication, and cutover. Assessment verifies engine version, extensions, character sets, unsupported objects, stored procedures, large tables, and downtime constraints. Connectivity establishes secure paths from the source to Google Cloud. Full load copies existing data. Continuous replication applies changes while applications still write to the source. Cutover stops writes, drains replication lag, validates data, and redirects applications.

#### Note

Migration success is measured by business continuity, not only bytes copied. The cutover plan must include freeze windows, rollback criteria, DNS or configuration changes, application smoke tests, and stakeholder sign-off.

### 2.3.2 Private IP and Private Services Access

Private IP connections keep database traffic inside private networking. For Cloud SQL, private IP requires VPC configuration and private services access. Applications running on GCE, GKE, Cloud Run with appropriate VPC access, Dataflow, or Composer can connect without exposing the database through a public address. Private connectivity reduces attack surface and simplifies compliance narratives, but it introduces routing, DNS, firewall, and peering considerations.

#### Definition

**Private services access** connects a consumer VPC to Google-managed services by using allocated internal address ranges. Cloud SQL private IP depends on this pattern so that instances receive private addresses reachable from selected VPC clients [16].

#### Common Pitfall

A private IP database is not automatically secure. Poor IAM, broad network reachability, weak passwords, missing SSL requirements, or overprivileged service accounts can still expose data. Private IP is one layer in a defense-in-depth design.

### 2.3.3 DMS Design Checklist

Before creating a migration job, document source credentials, network path, replication user privileges, schema conversion needs, table exclusions, large object handling, trigger behavior, sequence synchronization, replication slot cleanup, and performance limits. For PostgreSQL migrations, verify logical replication settings and extensions. For MySQL, verify binary logging and row-based replication settings. For SQL Server, confirm the supported path and downtime expectation.

| Area          | Question                                                      | Evidence                                         |
|---------------|---------------------------------------------------------------|--------------------------------------------------|
| Compatibility | Are engine version, extensions, and schema objects supported? | Assessment report and remediation list           |
| Connectivity  | Can DMS reach the source and destination privately?           | VPC, VPN, firewall, and connection profile tests |
| Replication   | Can changes be captured continuously?                         | Source replication settings and lag dashboard    |
| Validation    | Do row counts, checksums, and application tests pass?         | Reconciliation report and sign-off               |
| Cutover       | How are writes frozen and clients redirected?                 | Runbook with rollback criteria                   |

Table 2.2: Database Migration Service readiness checklist.

## 2.4 Wednesday: AlloyDB for PostgreSQL, Columnar Engine, and ML Integration

### 2.4.1 Why AlloyDB Exists

AlloyDB is a PostgreSQL-compatible managed database designed for demanding enterprise workloads [6]. It separates compute from a distributed storage layer, supports automated backups and high availability, and adds Google-engineered performance features. For data engineers, AlloyDB is attractive when teams want PostgreSQL compatibility but need stronger

performance, read scaling, or mixed operational and analytical capability than a conventional managed instance can provide.

#### Tip

Use Cloud SQL when the workload needs managed relational simplicity and broad engine options. Evaluate AlloyDB when PostgreSQL compatibility is required and the workload is performance-sensitive, analytically active, or expected to grow beyond a conventional single-instance pattern.

### 2.4.2 Columnar Engine

Traditional row storage is efficient for transactional lookups and updates because all columns of a row are near each other. Analytical queries often scan a few columns across many rows. AlloyDB's columnar engine accelerates suitable analytical queries by storing and processing selected data in a column-oriented format while preserving PostgreSQL compatibility for applications. This can reduce the need to offload every operational report into a separate warehouse immediately, though BigQuery remains the preferred platform for large-scale analytical warehousing.

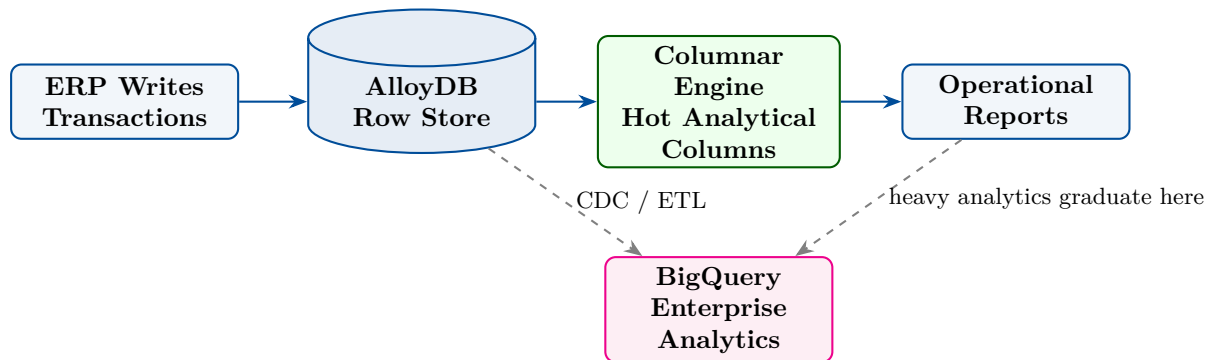


Figure 2.2: AlloyDB can serve transactions from PostgreSQL-compatible row storage while accelerating selected analytical scans with a columnar engine.

### 2.4.3 ML Integration

AlloyDB supports integrations that bring machine learning closer to PostgreSQL data, including vector search and model invocation patterns documented by Google Cloud [4]. These capabilities are useful when an application needs low-latency retrieval, semantic search, or lightweight predictive features near operational data. They should not turn the transactional database into an uncontrolled feature store. Training pipelines, large-scale feature computation, and governed model deployment still belong in services such as BigQuery, Dataflow, and Vertex AI.

#### Common Pitfall

Do not run every dashboard, vector search, and batch feature job against the primary transactional database. Even accelerated systems have concurrency, cost, and operational limits. Isolate workloads with read pools, replicas, BigQuery exports, and clear service-level objectives.

## 2.5 Thursday Hands-On Project: HA Cloud SQL PostgreSQL, Read Replica, and Failover

This lab deploys a regional HA Cloud SQL for PostgreSQL instance, creates a read replica, runs a simple Python connectivity test, and invokes a managed failover. Use a sandbox project, replace every placeholder, and delete resources afterward to avoid charges. Commands are written for Windows PowerShell.

### Note

Cloud SQL HA instances and replicas incur cost. Use a small machine type in a training project, choose a short lab window, and record cleanup commands before creating resources.

### 2.5.1 PowerShell Preparation and HA Instance Creation

```

1 $env:PROJECT_ID = "my-gcp-db-lab"
2 $env:REGION = "us-central1"
3 $env:NETWORK = "default"
4 $env:INSTANCE = "erp-pg-ha-lab"
5 $env:REPLICA = "erp-pg-ha-lab-replica"
6 $env:DB_NAME = "erp"
7 $env:DB_USER = "erp_app"
8 $env:DB_PASSWORD = "ChangeMe-Use-Secret-Manager"
9
10 gcloud config set project $env:PROJECT_ID
11
12 gcloud services enable sqladmin.googleapis.com servicenetworking.googleapis.com
13
14 gcloud compute addresses create google-managed-services-$env:NETWORK '
15   --global '
16   --purpose=VPC_PEERING '
17   --prefix-length=16 '
18   --network=$env:NETWORK
19
20 gcloud services vpc-peerings connect '
21   --service=servicenetworking.googleapis.com '
22   --ranges=google-managed-services-$env:NETWORK '
23   --network=$env:NETWORK
24
25 gcloud sql instances create $env:INSTANCE '
26   --database-version=POSTGRES_15 '
27   --region=$env:REGION '
28   --availability-type=REGIONAL '
29   --tier=db-custom-2-7680 '
30   --storage-type=SSD '
31   --storage-size=20GB '
32   --network=$env:NETWORK '
33   --no-assign-ip '
34   --backup-start-time=03:00 '
35   --enable-point-in-time-recovery '
36   --maintenance-window-day=SUN '
37   --maintenance-window-hour=4
38
39 gcloud sql databases create $env:DB_NAME --instance=$env:INSTANCE
40
41 gcloud sql users create $env:DB_USER '
42   --instance=$env:INSTANCE '
43   --password=$env:DB_PASSWORD

```

Listing 2.1: Creating an HA Cloud SQL for PostgreSQL instance with private IP placeholders.

### 2.5.2 Read Replica and Failover Drill

```

1 gcloud sql instances create $env:REPLICA '
2   --master-instance-name=$env:INSTANCE '
3   --region=$env:REGION '
4   --tier=db-custom-2-7680
5
6 gcloud sql instances describe $env:INSTANCE '
7   --format="yaml(name,region,gceZone,secondaryGceZone,availabilityType,state,
8     ipAddresses)"
9
10 gcloud sql instances describe $env:REPLICA '
11   --format="yaml(name,masterInstanceName,replicaNames,state,ipAddresses)"
12
13 # Run during an approved lab window. Applications must reconnect after failover
14
15 gcloud sql instances failover $env:INSTANCE
16
17 gcloud sql operations list '
18   --instance=$env:INSTANCE '
19   --limit=5

```

Listing 2.2: Creating a read replica, checking status, and simulating failover.

### 2.5.3 Python Connectivity Check

The following snippet assumes the Cloud SQL Auth Proxy or connector path provides a local PostgreSQL endpoint. In production, store credentials in Secret Manager and use IAM-authenticated connectors where possible.

```

1 import os
2 import time
3
4 import psycopg
5
6
7 def connect_once(label: str) -> None:
8     conninfo = {
9         "host": os.environ.get("PGHOST", "127.0.0.1"),
10        "port": os.environ.get("PGPORT", "5432"),
11        "dbname": os.environ.get("PGDATABASE", "erp"),
12        "user": os.environ.get("PGUSER", "erp_app"),
13        "password": os.environ.get("PGPASSWORD"),
14        "connect_timeout": 10,
15    }
16    with psycopg.connect(**conninfo) as conn:
17        with conn.cursor() as cur:
18            cur.execute("select current_database(), now()")
19            database_name, current_time = cur.fetchone()
20            print({"label": label, "database": database_name, "time": str(
21                current_time)})
22
23 if __name__ == "__main__":
24     for attempt in range(1, 6):
25         try:
26             connect_once(label=f"attempt-{attempt}")
27             break
28         except psycopg.OperationalError as exc:
29             print({"attempt": attempt, "status": "retrying", "error": str(exc)})
30             time.sleep(10)

```

Listing 2.3: Validating PostgreSQL connectivity before and after a Cloud SQL failover.

### 2.5.4 Cleanup

```
1 gcloud sql instances delete $env:REPLICA --quiet
2 gcloud sql instances delete $env:INSTANCE --quiet
```

Listing 2.4: Deleting lab Cloud SQL resources after validation.

## 2.6 Friday Use Case and Architecture: Zero-Downtime ERP Migration

### 2.6.1 Scenario

A manufacturing company runs a monolithic on-premises ERP on PostgreSQL. The ERP manages purchasing, inventory, production orders, finance, and customer shipments. Executives want the platform moved to Google Cloud without a disruptive weekend outage. Reporting queries currently run on the same database as transactions and regularly slow month-end close. The target architecture must use Cloud SQL for transactional continuity, Database Migration Service for continuous replication, private connectivity for security, and AlloyDB for accelerated operational analytics.

### 2.6.2 Migration Architecture

The smallest safe plan is not a big-bang rewrite. Keep the ERP application behavior stable, migrate the database first, and offload heavy reads. Establish VPN or Interconnect from the data center to Google Cloud, create DMS connection profiles, perform initial load, enable continuous replication, validate row counts and checksums, rehearse cutover, freeze writes, drain lag, promote Cloud SQL, and redirect the ERP application. After stabilization, replicate selected tables or change streams into AlloyDB and BigQuery for analytics.

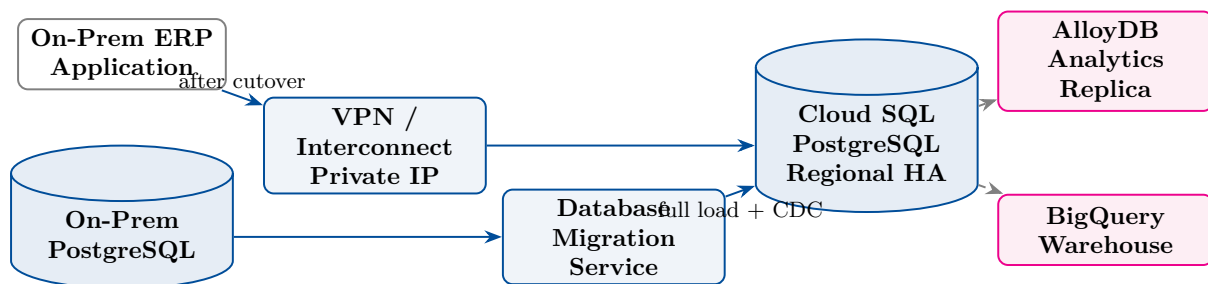


Figure 2.3: Zero-downtime ERP migration using DMS, private connectivity, Cloud SQL HA, and AlloyDB analytics offload.

### 2.6.3 Cutover and Validation

Zero downtime is an aspiration that must be translated into measurable tolerances. The ERP may tolerate a short write freeze while replication lag drains and the application restarts. Define acceptable lag, maximum cutover duration, rollback point, and business validation. Validate table counts, financial totals, open order counts, sequence values, permissions, stored procedure behavior, and representative application transactions. Keep the source database read-only until the rollback window closes.

**Tip**

Use business invariants for validation: trial balance totals, inventory quantities by warehouse, open invoice counts, and shipment status counts. Row counts are necessary but not sufficient.

## 2.7 Security, Operations, and Cost Notes

Relational databases concentrate high-value data. Use private IP, least-privilege IAM, strong database roles, Secret Manager, audit logging, and SSL where required. Avoid embedding passwords in scripts or application configuration. Use connection pooling because serverless and autoscaled clients can overwhelm PostgreSQL with thousands of short-lived connections. Monitor CPU, memory, storage growth, lock waits, slow queries, replication lag, connection count, backup status, and failover events.

**Note**

Database cost is driven by always-on compute, storage, backups, replicas, HA standby capacity, network traffic, and operational mistakes such as unbounded reporting queries. Label instances and export billing data so database spend is visible by workload.

## 2.8 Interview Q&A

### 2.8.1 Concepts and Trade-offs

1. **Q: How does Cloud SQL regional HA differ from a read replica?**

A: Regional HA uses a standby in another zone for failover and is not a read endpoint. A read replica is asynchronously replicated and can serve reads, but it may lag and does not replace HA.

2. **Q: Why should migrations use private IP when possible?**

A: Private IP keeps database traffic off public addresses, reduces exposed attack surface, and fits compliance expectations, while still requiring IAM, firewall, credential, and database-role controls.

3. **Q: What is the main risk of reading from replicas?**

A: Replica lag. Reporting can usually tolerate slight staleness, but workflows that require read-after-write consistency should use the primary or design explicit consistency checks.

4. **Q: When would you choose AlloyDB over Cloud SQL for PostgreSQL?**

A: Choose AlloyDB when PostgreSQL compatibility is required and the workload needs higher performance, read pools, improved availability patterns, or columnar acceleration for operational analytics.

## 2.9 Further Reading

Read the Cloud SQL documentation for current PostgreSQL, MySQL, SQL Server, HA, backup, replica, and connectivity behavior [15]. Review the Cloud SQL private IP guidance before designing production networks [16]. Study Database Migration Service documentation for supported sources, continuous replication, connection profiles, and cutover practices [18]. Review AlloyDB documentation for clusters, instances, high availability, read pools, columnar engine behavior, and AI integrations [6, 5, 4]. Use the Google Cloud Architecture Framework to evaluate reliability, security, cost, performance, and operations [21].

## Key Takeaways

- Cloud SQL simplifies database administration, but engineers still own schema quality, connection behavior, migration correctness, and recovery testing.
- Regional HA protects against zonal infrastructure failure; backups and point-in-time recovery protect against logical failure.
- Read replicas scale reads and isolate reporting, but asynchronous lag must be visible and acceptable.
- Database Migration Service migrations need assessment, private connectivity, continuous replication, validation, and a rehearsed cutover runbook.
- AlloyDB extends PostgreSQL-compatible operations with performance features, columnar acceleration, and AI-oriented integrations.
- ERP migrations should stabilize the transactional target first, then offload analytics to replicas, AlloyDB, or BigQuery.

## 2.10 Exercises

1. **(Conceptual)** Explain why high availability does not protect against an accidental `delete from orders` statement.
2. **(Conceptual)** Compare a Cloud SQL regional HA standby, same-region read replica, and cross-region read replica.
3. **(Hands-on)** Create a Cloud SQL PostgreSQL instance in a sandbox project with backups and point-in-time recovery enabled.
4. **(Hands-on)** Create a read replica and measure replication lag after inserting test rows into the primary.
5. **(Hands-on)** Run a failover drill and record application reconnect time using a small Python script.
6. **(Design)** Draft a DMS migration checklist for a 2 TB on-premises PostgreSQL database with a one-hour cutover window.
7. **(Design)** Choose private IP, public IP with authorized networks, or connector-based access for Cloud Run, GKE, and on-premises clients.
8. **(Architecture)** Extend the ERP migration design with monitoring, rollback criteria, business validation metrics, and an AlloyDB reporting workload.

## 2.11 Capstone Project: HA Cloud SQL and AlloyDB Analytics for ERP Migration

### Capstone Project

**Mission:** Design and prototype a zero-downtime ERP database migration from on-premises PostgreSQL to HA Cloud SQL, then offload operational analytics to AlloyDB. Your solution must preserve transactional correctness, use private connectivity, provide a tested failover plan, and give reporting users a path that does not overload the ERP primary.

**Estimated time:** 5–7 hours.

**What you'll practice:**

- Creating a regional HA Cloud SQL for PostgreSQL target with backups and point-in-time recovery.
- Designing DMS connection profiles, migration jobs, validation checks, and cutover steps.
- Using private IP and least-privilege access for application and migration traffic.
- Creating a read replica and defining promotion or failover runbooks.
- Adding AlloyDB as an analytics-serving replica pattern for operational reporting.
- Producing a reference architecture, implementation transcript, acceptance criteria, and rollback plan.

## Scenario

Northwind Precision Manufacturing runs a monolithic ERP backed by a 4 TB on-premises PostgreSQL database. The database receives order, procurement, inventory, finance, and shipping transactions around the clock. Month-end reporting joins large invoice, payment, and inventory tables and regularly slows application users. Leadership wants the database moved to Google Cloud with no data loss, only a brief planned write freeze, private connectivity, and a clear path for faster reporting.

## Requirements

- Deploy a Cloud SQL for PostgreSQL target with regional HA, automated backups, and point-in-time recovery.
- Use private IP connectivity through a VPC design appropriate for application clients and DMS.
- Define DMS source and destination connection profiles, full load, continuous replication, and cutover criteria.
- Create at least one Cloud SQL read replica for reporting isolation or DR rehearsal.
- Design an AlloyDB analytics replica pattern for month-end reporting tables and selected operational dashboards.
- Document database roles, Secret Manager usage, audit logging, monitoring, and alert thresholds.
- Provide validation evidence for row counts, checksums, sequence values, business totals, and application smoke tests.
- Define rollback criteria and source database handling during the post-cutover stabilization window.

## Reference Architecture

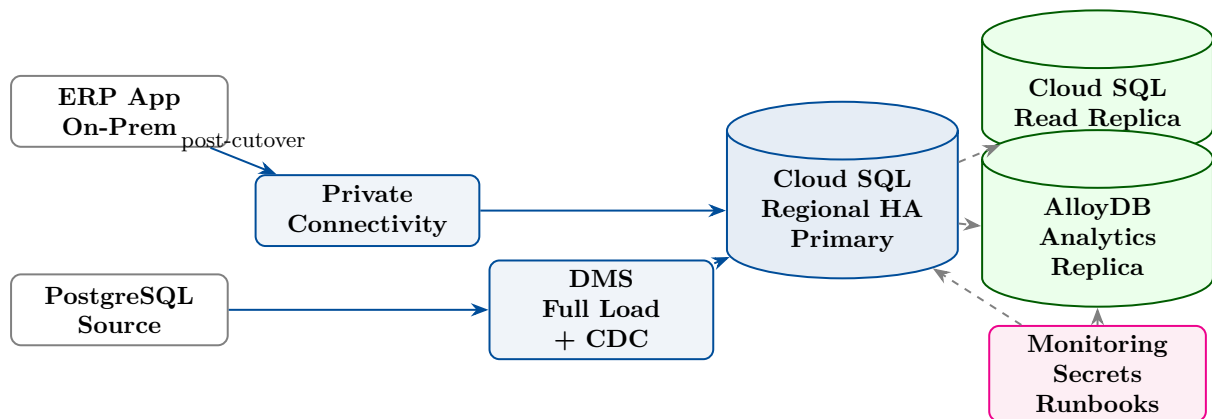


Figure 2.4: Capstone reference architecture for HA Cloud SQL with AlloyDB analytics offload during ERP migration.

## Implementation Steps

1. **Create the HA Cloud SQL target and read replica.** Replace all placeholders, use a sandbox project first, and do not store production passwords in shell history.

```

1 $env:PROJECT_ID = "my-gcp-erp-capstone"
2 $env:REGION = "us-central1"
3 $env:NETWORK = "default"
4 $env:INSTANCE = "erp-prod-pg-ha"
5 $env:REPLICA = "erp-prod-pg-reporting"
6 $env:DB_NAME = "erp"
7 $env:DB_USER = "erp_app"
8 $env:DB_PASSWORD = "Replace-With-Secret-Manager-Value"
9
10 gcloud config set project $env:PROJECT_ID
11
12 gcloud sql instances create $env:INSTANCE '
13   --database-version=POSTGRES_15 '
14   --region=$env:REGION '
15   --availability-type=REGIONAL '
16   --tier=db-custom-4-15360 '
17   --storage-type=SSD '
18   --storage-size=500GB '
19   --storage-auto-increase '
20   --network=$env:NETWORK '
21   --no-assign-ip '
22   --backup-start-time=02:00 '
23   --enable-point-in-time-recovery
24
25 gcloud sql databases create $env:DB_NAME --instance=$env:INSTANCE
26
27 gcloud sql users create $env:DB_USER '
28   --instance=$env:INSTANCE '
29   --password=$env:DB_PASSWORD
30
31 gcloud sql instances create $env:REPLICA '
32   --master-instance-name=$env:INSTANCE '
33   --region=$env:REGION '
34   --tier=db-custom-4-15360
  
```

Listing 2.5: Capstone Cloud SQL target, database, user, and replica creation.

2. **Create DMS placeholders and validation artifacts.** The exact flags vary by source engine and network path, so the listing emphasizes the repeatable evidence each migration must capture.

```

1 $env:MIGRATION_JOB = "erp-postgres-to-cloudsql"
2 $env:SOURCE_PROFILE = "erp-onprem-postgres"
3 $env:DEST_PROFILE = "erp-cloudsql-postgres"
4
5 gcloud database-migration connection-profiles list --region=$env:REGION
6
7 gcloud database-migration migration-jobs list --region=$env:REGION
8
9 gcloud database-migration migration-jobs describe $env:MIGRATION_JOB '
10 --region=$env:REGION '
11 --format="yaml(state,phase,source,destination,dumpPath,duration)"
12
13 @'
14 validation_item,status,evidence
15 row_counts,pending,Compare source and target counts for critical tables
16 checksums,pending,Sample deterministic hashes by primary key range
17 sequences,pending,Verify next values after cutover rehearsal
18 business_totals,pending,Compare trial balance and inventory by warehouse
19 smoke_tests,pending,Run login order invoice and shipment workflows
20 '@ | Set-Content -Path .\erp-migration-validation.csv

```

Listing 2.6: DMS planning commands and migration validation evidence placeholders.

3. **Prototype application retry validation.** Run this check before and after a planned failover or cutover rehearsal.

```

1 import os
2 import time
3
4 import psycopg
5
6
7 SQL = """
8 select
9     count(*) as open_orders
10 from erp_orders
11 where status in ('OPEN', 'ALLOCATED')
12 """
13
14
15 def query_open_orders() -> int:
16     with psycopg.connect(
17         host=os.environ["PGHOST"],
18         port=os.environ.get("PGPORT", "5432"),
19         dbname=os.environ["PGDATABASE"],
20         user=os.environ["PGUSER"],
21         password=os.environ["PGPASSWORD"],
22         connect_timeout=10,
23     ) as conn:
24         with conn.cursor() as cur:
25             cur.execute(SQL)
26             return cur.fetchone()[0]
27
28
29 for attempt in range(1, 8):
30     try:
31         print({"attempt": attempt, "open_orders": query_open_orders()})
32         break
33     except psycopg.OperationalError as exc:
34         print({"attempt": attempt, "retry": True, "error": str(exc)})

```

```
35      time.sleep(15)
```

Listing 2.7: Capstone retry loop for ERP database smoke testing.

## Deliverables

- A one-page architecture summary explaining HA Cloud SQL, private IP, DMS, read replica, AlloyDB analytics offload, and rollback boundaries.
- The completed reference diagram or an equivalent diagram showing on-premises ERP, DMS, private connectivity, Cloud SQL HA, read replica, AlloyDB, monitoring, and secrets.
- Command transcript or screenshots proving Cloud SQL creation, backup settings, PITR, private IP, read replica, and failover rehearsal.
- A migration runbook with assessment findings, replication setup, validation queries, cutover timeline, rollback triggers, and owner assignments.
- A reporting offload plan that identifies which ERP reports stay on Cloud SQL replicas, which move to AlloyDB, and which graduate to BigQuery.

## Acceptance Criteria

- The target Cloud SQL instance uses regional HA, backups, point-in-time recovery, private IP, and least-privilege database roles.
- DMS migration phases are documented from source assessment through continuous replication and cutover.
- Validation includes row counts, sampled checksums, sequence values, business invariants, and application smoke tests.
- The cutover plan defines write freeze, lag drain, DNS or configuration switch, rollback criteria, and source read-only handling.
- A read replica or AlloyDB offload path protects the transactional primary from heavy reporting.
- Monitoring covers CPU, memory, connections, storage growth, slow queries, locks, replica lag, backup status, and failover events.
- Secrets are not hardcoded in application code, Terraform variables, notebooks, or command transcripts.
- The design states residual risks, including replica lag, extension compatibility, stored procedure behavior, and application reconnect behavior.

## Stretch Goals

- Add a Cloud SQL Auth Proxy or language connector deployment pattern for Cloud Run, GKE, and on-premises migration workers.
- Build a BigQuery dashboard from exported validation CSV files that tracks migration readiness by table and business process.
- Add Datastream or another CDC path from Cloud SQL into BigQuery for long-term analytical history.

- Prototype AlloyDB vector search for ERP support-ticket retrieval using sanitized, non-sensitive text.
- Create a failover game day that measures application reconnect time, replica lag, alert delivery, and runbook accuracy.
- Estimate monthly cost for Cloud SQL HA, replica, backups, AlloyDB, network transfer, and operational monitoring.

# Global Relational Scaling: Cloud Spanner

## Learning Objectives

- Explain Cloud Spanner architecture, including instances, nodes, processing units, splits, replicas, leaders, Paxos, and TrueTime.
- Describe how Cloud Spanner provides strong global consistency and external consistency for relational transactions.
- Design schemas with primary keys, interleaved tables, secondary indexes, and foreign keys while avoiding hot spots.
- Use query execution plans, Key Visualizer, statistics, and indexing strategy to tune Spanner performance.
- Create a multi-region Spanner instance, build an interleaved e-commerce cart schema, and load 10,000 synthetic records.
- Evaluate a globally distributed payment-processing ledger that requires 99.999% availability and zero data loss across continents.
- Build a capstone architecture for a globally consistent payment ledger on multi-region Spanner with operations, validation, and recovery evidence.

## Cross-Cloud Equivalence

Globally distributed transactional databases are marketed differently across clouds, but the design questions are portable: consistency model, write placement, replica quorum, horizontal scale, latency, and relational compatibility.

| Capability                  | GCP                                                                            | AWS                                                                           | Azure                                                            | Traditional sharded RDBMS                                               |
|-----------------------------|--------------------------------------------------------------------------------|-------------------------------------------------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------------------|
| Globally consistent OLTP    | Cloud Spanner with external consistency and SQL                                | Aurora Database for low-latency global reads; Aurora DSQL for distributed SQL | Cosmos DB with strong consistency for NoSQL APIs                 | Usually application-managed two-phase commit or eventual reconciliation |
| Horizontal scale            | Automatic splits across nodes and replicas                                     | Cluster and regional replicas; DSQL partitions distributed data               | Partitioned containers and global distribution                   | Manual shards, re-sharding, and routing layer                           |
| Synchronous replication     | Paxos quorum across replicas                                                   | Region-aware replication varies by product and mode                           | Configurable consistency and multi-region writes                 | Often limited to one region or expensive cross-shard coordination       |
| External consistency / time | TrueTime commit timestamps order transactions globally                         | Service-specific transaction ordering; no direct TrueTime equivalent          | Strong consistency can linearize reads within configured regions | Application timestamps rarely prove external consistency                |
| Schema modeling             | Relational tables, primary keys, secondary indexes, interleaving, foreign keys | Relational models in Aurora; distributed SQL model in DSQL                    | NoSQL containers, partition keys, and consistency levels         | Normalized tables inside shards; cross-shard foreign keys are hard      |

MongoDB sharded clusters can distribute document workloads and Snowflake can replicate analytical data, but neither is a direct drop-in for globally consistent relational OLTP. MongoDB usually trades relational constraints for document flexibility, while Snowflake is analytical rather than a low-latency transaction processor.

### Key Terms

**Cloud Spanner:** Google Cloud’s fully managed, horizontally scalable relational database with strong consistency and high availability. **TrueTime:** Google’s bounded-time API that lets Spanner assign commit timestamps with known uncertainty. **External consistency:** guarantee that if transaction A commits before transaction B starts, B receives a later timestamp and observes A where appropriate. **Split:** contiguous key range that Spanner can move or replicate independently for scale. **Interleaved table:** child table whose rows are physically colocated with a parent row by primary key prefix. **Leader replica:** replica that coordinates writes for a Paxos group.

## 3.1 Week 3 Roadmap

Week 3 moves from regional relational services to a database built for global transactional scale. Cloud Spanner is the GCP service to evaluate when the business needs relational semantics, SQL, horizontal scaling, synchronous replication, and high availability beyond a single region [12]. It is not merely a bigger Cloud SQL instance. It changes how engineers think about primary keys, transactions, locality, indexes, and operational evidence.

Monday introduces nodes, processing units, splits, replicas, Paxos, TrueTime, and strong global consistency. Tuesday focuses on schema design with primary keys, interleaved tables, foreign keys, and secondary indexes. Wednesday studies performance tuning, query plans, leader placement, statistics, and hot-spot avoidance. Thursday builds a multi-region Spanner lab with an

interleaved shopping-cart schema and 10,000 synthetic rows. Friday applies the patterns to a payment-processing ledger that requires 99.999% availability and zero data loss across continents.

#### Definition

**Cloud Spanner** is a managed relational database that combines SQL, transactions, schema, and strong consistency with horizontal scaling and synchronous replication. It was designed from the Spanner research system described by Google, where TrueTime and Paxos make globally ordered commits practical at large scale [2].

## 3.2 Monday: Spanner Concepts and Strong Consistency

### 3.2.1 Instances, Nodes, Processing Units, and Databases

A Spanner instance is the administrative container for one or more databases. The instance configuration chooses a regional or multi-region topology, which determines where replicas live and which regions can serve reads and writes. Capacity is allocated in nodes or processing units. A node is a larger capacity unit; processing units allow finer increments for smaller workloads. Capacity affects CPU, storage throughput, and the amount of work Spanner can process concurrently, but it does not change the fundamental consistency model.

Spanner stores data in splits. A split is a contiguous range of primary keys that can be moved, replicated, and served independently. As data grows or traffic patterns change, Spanner can split ranges and distribute them across servers. This is why primary-key design matters: the primary key defines the physical ordering of rows, and the ordering determines whether writes spread evenly or concentrate on a narrow hot range.

#### Note

Spanner removes many database administration tasks, but it does not remove data modeling. Poor primary keys, unbounded transactions, and inefficient secondary indexes can still create latency and cost problems.

### 3.2.2 Replicas, Leaders, and Paxos Quorums

Every split is replicated. For each Paxos group, one replica acts as leader and coordinates writes. A write commits only after a quorum agrees, which protects against losing committed data when a replica or zone fails. Regional configurations keep replicas inside one region for lower write latency. Multi-region configurations place replicas across multiple regions to improve availability and global read locality, at the cost of higher write latency because consensus crosses longer network distances.

#### Definition

**Paxos** is a consensus protocol family that lets distributed replicas agree on a value even when some replicas fail. In Spanner, Paxos helps ensure that committed writes survive replica failures and that a majority quorum decides the ordered history for each key range.

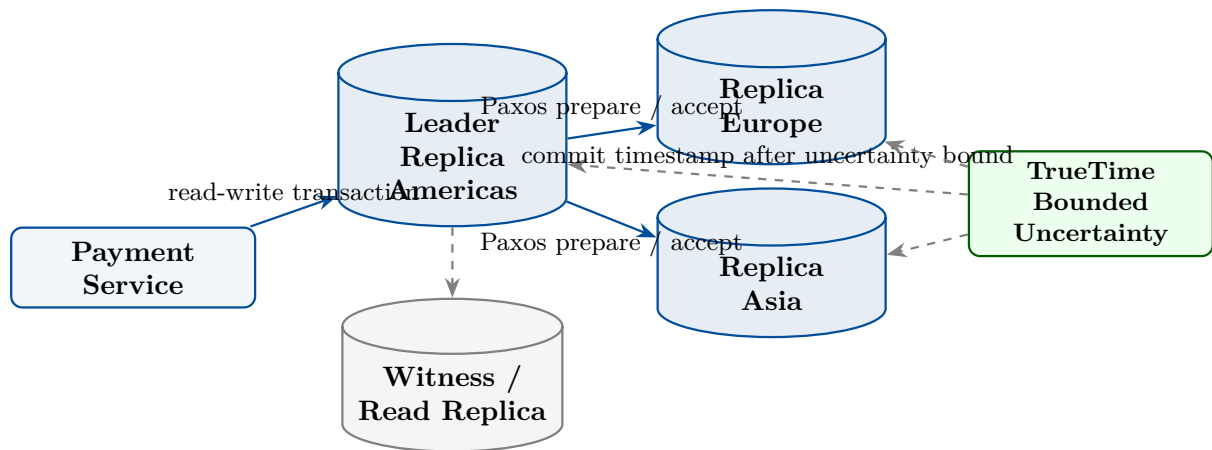


Figure 3.1: Spanner combines Paxos replication with TrueTime commit timestamps to provide globally ordered transactions.

### 3.2.3 TrueTime and External Consistency

TrueTime exposes a time interval rather than a single perfect clock reading. The interval contains the real current time with bounded uncertainty. Spanner uses this bound when assigning commit timestamps. Before a transaction is externally visible, Spanner waits long enough to ensure that later transactions receive later timestamps. This commit-wait technique is one reason Spanner can provide external consistency across distributed replicas.

#### Definition

**External consistency** is stronger than ordinary serializability. It preserves real-time ordering: if one transaction commits before another transaction begins, the second transaction is ordered after the first.

Strong global consistency is especially valuable for ledgers, entitlements, inventory allocation, identity state, and idempotency records. Without it, engineers often add application-level locks, reconciliation jobs, and regional conflict resolution. Spanner does not make latency disappear, but it gives a precise consistency contract that simplifies correctness-sensitive designs.

#### Common Pitfall

Do not choose Spanner only because a database is large. Choose it when the workload needs horizontal relational scale, high availability, strong consistency, or global distribution. For one-region departmental databases, Cloud SQL or AlloyDB may be simpler and cheaper.

## 3.3 Tuesday: Schema Design, Primary Keys, Interleaving, and Indexing

### 3.3.1 Primary Keys Define Physical Locality

In Spanner, primary-key order is physical design. Rows with nearby primary-key values tend to live near each other until the range splits. This helps range scans and parent-child access patterns, but it can hurt write scalability when the leading key is monotonically increasing. For example, using a timestamp as the first primary-key column can concentrate all new writes into the newest range. Better designs place a high-cardinality business identifier, hash bucket,

tenant identifier, or region-aware prefix before time when write distribution matters.

#### Tip

Start primary-key design from access patterns and write distribution. Ask: Which queries must be fast? Which rows are read together? Which column changes monotonically? Which tenant, account, or merchant could become hot?

### 3.3.2 Interleaved Tables

Interleaved tables physically colocate child rows with a parent row by sharing the parent primary-key prefix. An order table can be interleaved under a customer table; order lines can be interleaved under orders. This is useful when child rows are usually read with their parent and when the child lifecycle follows the parent lifecycle. It is less useful for independent child records that are frequently scanned by a different access path.

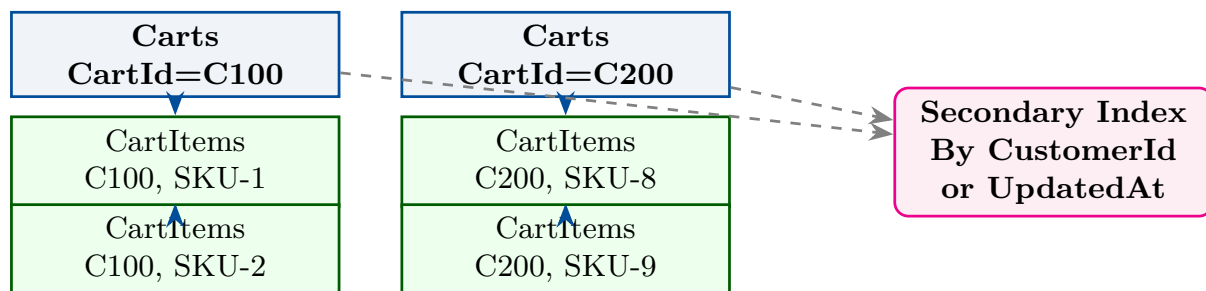


Figure 3.2: Interleaving stores child rows near parent rows while secondary indexes support alternate access paths.

#### Note

Interleaving is not the same as a foreign key. Foreign keys enforce referential integrity. Interleaving influences physical locality and optional cascade behavior. A design can use one, both, or neither depending on access and integrity requirements.

### 3.3.3 Secondary Indexes and Covering Reads

Secondary indexes provide alternate sort orders and lookup paths. They are essential when queries filter by columns that are not the leading primary-key columns. However, every index is another structure that must be maintained synchronously during writes. Too few indexes cause full scans; too many indexes increase write cost and storage. A covering index can store additional columns so Spanner can answer a query from the index without joining back to the base table.

| Design choice                | Best fit                                              | Caution                                                    |
|------------------------------|-------------------------------------------------------|------------------------------------------------------------|
| Random or hashed leading key | High write distribution for large tenants or accounts | Range queries may need a secondary index or bucket fan-out |
| Tenant then entity key       | Multi-tenant isolation and tenant-scoped reads        | One very large tenant can still become hot                 |
| Timestamp suffix             | Recent-history scans inside a parent or tenant        | Timestamp as leading key can hot spot writes               |
| Interleaved child table      | Parent-child reads and lifecycle coupling             | Deep hierarchies and large parents can concentrate traffic |
| Covering secondary index     | Low-latency reads by alternate predicate              | Extra write latency, storage, and consistency cost         |

Table 3.1: Spanner schema design trade-offs.

## 3.4 Wednesday: Performance Tuning and Plans

### 3.4.1 Reading Query Plans

Spanner query plans show scans, joins, distributed unions, filters, sorting, and row counts. A healthy point lookup uses the primary key or a selective index. A suspicious plan scans far more rows than it returns, sorts large intermediate results, or performs distributed work for a query that should be tenant-local. Use the Google Cloud console, query statistics tables, and the command-line tools to compare the plan with the intended access path [13].

#### Definition

**Selectivity** describes how much a predicate narrows the candidate rows. A highly selective predicate, such as a unique payment identifier, should use a key or index. A low-selectivity predicate, such as status equals open across all merchants, may still scan many rows.

### 3.4.2 Hot Spots and Split Health

Hot spots happen when a small key range receives a disproportionate share of reads or writes. Common causes include monotonically increasing primary keys, a single global counter row, a popular tenant with no distribution strategy, or an index whose leading column has low cardinality. Key Visualizer helps detect hot key ranges, while metrics expose CPU, latency, lock waits, and transaction aborts [26].

#### Common Pitfall

A secondary index can hot spot even when the base table is well distributed. If every new transaction writes an index entry with the same leading status or current timestamp range, the index becomes the bottleneck.

### 3.4.3 Transactions, Mutations, and Latency

Read-write transactions acquire locks and coordinate commits through the leader. Keep them short, deterministic, and limited to rows that must be atomically changed together. Large analytical reads, export jobs, and backfills should use partitioned reads, Dataflow connectors, or BigQuery exports rather than long transactions. For idempotent ingestion, store request identifiers and business keys so retries can safely detect already committed work.

**Tip**

Tune from evidence. Capture the SQL text, parameters, query plan, latency percentile, rows scanned, rows returned, CPU, lock wait, and index choices before changing schema. Then compare the same evidence after the change.

## 3.5 Thursday Hands-On Project: Multi-Region Cart Lab

This lab creates a multi-region Spanner instance, defines an interleaved cart schema, and loads 10,000 synthetic cart-item records. Use a sandbox project, replace placeholders, and delete resources when finished. Commands are written for Windows PowerShell.

**Note**

Multi-region Spanner instances can incur meaningful cost. Use the smallest capacity suitable for the lab, set a cleanup reminder, and verify that no production project or billing account is used.

### 3.5.1 PowerShell Preparation and Instance Creation

```

1 $env:PROJECT_ID = "my-gcp-spanner-lab"
2 $env:INSTANCE_ID = "cart-global-lab"
3 $env:DATABASE_ID = "cartdb"
4 $env:INSTANCE_CONFIG = "nam3"
5 $env:PROCESSING_UNITS = "1000"
6
7 gcloud config set project $env:PROJECT_ID
8
9 gcloud services enable spanner.googleapis.com
10
11 gcloud spanner instances create $env:INSTANCE_ID `
12   --config=$env:INSTANCE_CONFIG `
13   --description="Global cart lab" `
14   --processing-units=$env:PROCESSING_UNITS
15
16 gcloud spanner databases create $env:DATABASE_ID `
17   --instance=$env:INSTANCE_ID

```

Listing 3.1: Creating a multi-region Cloud Spanner instance from PowerShell.

### 3.5.2 Interleaved Cart Schema

```

1 $ddl = @'
2 CREATE TABLE Carts (
3   CartId STRING(36) NOT NULL,
4   CustomerId STRING(36) NOT NULL,
5   HomeRegion STRING(16) NOT NULL,
6   Status STRING(16) NOT NULL,
7   CreatedAt TIMESTAMP NOT NULL OPTIONS (allow_commit_timestamp=true),
8   UpdatedAt TIMESTAMP NOT NULL OPTIONS (allow_commit_timestamp=true),
9 ) PRIMARY KEY (CartId);
10
11 CREATE TABLE CartItems (
12   CartId STRING(36) NOT NULL,
13   LineId STRING(36) NOT NULL,
14   Sku STRING(64) NOT NULL,
15   Quantity INT64 NOT NULL,
16   UnitPrice NUMERIC NOT NULL,
17   AddedAt TIMESTAMP NOT NULL OPTIONS (allow_commit_timestamp=true),

```

```

18 ) PRIMARY KEY (CartId, LineId),
19     INTERLEAVE IN PARENT Carts ON DELETE CASCADE;
20
21 CREATE INDEX CartsByCustomerUpdated
22 ON Carts(CustomerId, UpdatedAt DESC)
23 STORING (Status, HomeRegion);
24 '@
25
26 $ddl | gcloud spanner databases ddl update $env:DATABASE_ID '
27     --instance=$env:INSTANCE_ID '
28     --ddl-file=-

```

Listing 3.2: Creating an interleaved e-commerce cart schema with DDL.

### 3.5.3 Python Synthetic Load

The Python client uses batches to insert 1,000 carts and 10,000 cart items. In production, prefer service-account credentials, workload identity, and controlled batch sizes matched to latency and mutation limits.

```

1 import os
2 import random
3 import uuid
4 from decimal import Decimal
5
6 from google.cloud import spanner
7 from google.cloud.spanner_v1 import COMMIT_TIMESTAMP
8
9
10 PROJECT_ID = os.environ["PROJECT_ID"]
11 INSTANCE_ID = os.environ.get("INSTANCE_ID", "cart-global-lab")
12 DATABASE_ID = os.environ.get("DATABASE_ID", "cartdb")
13 CART_COUNT = 1000
14 ITEMS_PER_CART = 10
15 REGIONS = ["americas", "europe", "asia"]
16 STATUSES = ["ACTIVE", "ABANDONED", "CHECKED_OUT"]
17
18
19 def chunked(values, size):
20     for start in range(0, len(values), size):
21         yield values[start:start + size]
22
23
24 def main() -> None:
25     client = spanner.Client(project=PROJECT_ID)
26     instance = client.instance(INSTANCE_ID)
27     database = instance.database(DATABASE_ID)
28
29     carts = []
30     items = []
31     for cart_number in range(CART_COUNT):
32         cart_id = str(uuid.uuid4())
33         customer_id = str(uuid.uuid4())
34         carts.append((
35             cart_id,
36             customer_id,
37             random.choice(REGIONS),
38             random.choice(STATUSES),
39             COMMIT_TIMESTAMP,
40             COMMIT_TIMESTAMP,
41         ))
42         for line_number in range(ITEMS_PER_CART):
43             items.append((

```

```

44         cart_id,
45         f"line-{{line_number:02d}}",
46         f"SKU-{{random.randint(1, 5000):05d}}",
47         random.randint(1, 5),
48         Decimal(random.randrange(100, 25000)) / Decimal("100"),
49         COMMIT_TIMESTAMP,
50     ))
51
52     with database.batch() as batch:
53         batch.insert(
54             table="Carts",
55             columns=("CartId", "CustomerId", "HomeRegion", "Status", "CreatedAt",
56                     "UpdatedAt"),
57             values=carts,
58         )
59
60     for item_chunk in chunked(items, 1000):
61         with database.batch() as batch:
62             batch.insert(
63                 table="CartItems",
64                 columns=("CartId", "LineId", "Sku", "Quantity", "UnitPrice", "
65                         AddedAt"),
66                 values=item_chunk,
67             )
68
69     print({"carts": len(carts), "cart_items": len(items)})
70
71 if __name__ == "__main__":
72     main()

```

Listing 3.3: Loading 10,000 synthetic records into Cloud Spanner with google-cloud-spanner.

### 3.5.4 Query Plan Check and Cleanup

```

1 gcloud spanner databases execute-sql $env:DATABASE_ID '
2     --instance=$env:INSTANCE_ID '
3     --sql="SELECT CartId, Status FROM Carts WHERE CustomerId = 'replace-customer-
4         id'"
5
6 gcloud spanner databases execute-sql $env:DATABASE_ID '
7     --instance=$env:INSTANCE_ID '
8     --query-mode=PLAN '
9     --sql="SELECT c.CartId, i.Sku, i.Quantity FROM Carts c JOIN CartItems i USING
10         (CartId) WHERE c.CartId = 'replace-cart-id'"
11
12 gcloud spanner instances delete $env:INSTANCE_ID --quiet

```

Listing 3.4: Checking sample query plans and deleting lab resources.

## 3.6 Friday Use Case: Global Payment Ledger

### 3.6.1 Scenario

A payment processor authorizes purchases in North America, Europe, and Asia. Merchants require low-latency authorization, regulators require immutable ledger evidence, and the business requires 99.999% availability with zero data loss across continents. A regional outage must not lose committed debits, credits, holds, refunds, or idempotency records. The ledger must support strict account balance correctness, globally unique transaction identifiers, and audit reads by compliance teams.

### 3.6.2 Architecture Pattern

The core ledger belongs in a multi-region Spanner instance. Payment APIs write idempotency records, ledger entries, and account-balance deltas in read-write transactions. Regional service tiers route requests to nearby endpoints, but the database maintains one globally consistent transaction history. Pub/Sub and Dataflow can stream committed ledger facts to BigQuery for analytics, while Cloud Monitoring and audit logs provide operational evidence.

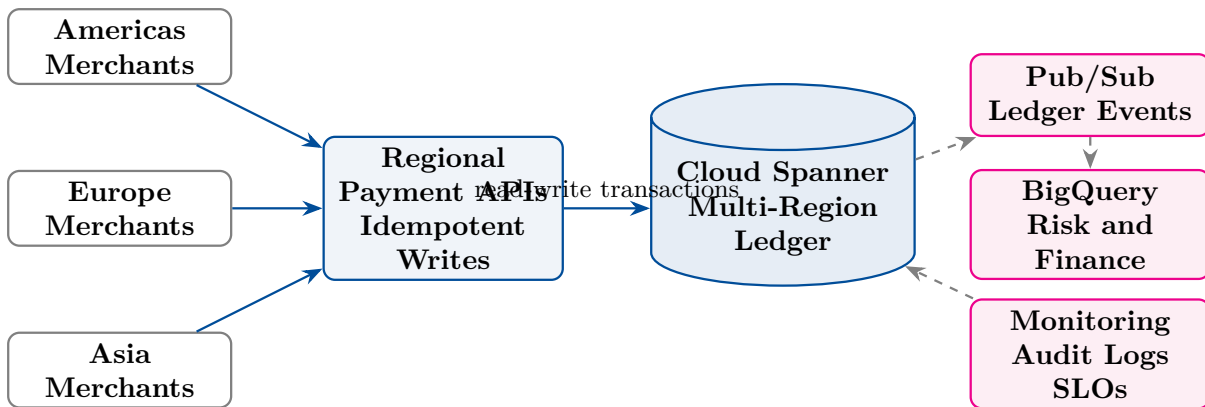


Figure 3.3: A multi-continent payment ledger uses Spanner for globally consistent OLTP and streams committed facts to analytics systems.

### 3.6.3 Correctness Controls

Ledger transactions should be append-only where possible. Instead of updating historical entries, write reversing entries and preserve immutable facts. Use a unique payment request identifier for idempotency, a ledger-entry table keyed by account and commit timestamp or sequence, and balance rows updated transactionally only when the invariant must be enforced immediately. For analytics, replicate committed facts downstream rather than running heavy reports against the OLTP leader.

#### Tip

For payment systems, availability is not enough. The design must prove no committed transaction is lost, no duplicate request is double-applied, and every balance can be reconstructed from the ledger.

## 3.7 Security, Operations, and Cost Notes

Use least-privilege IAM roles, database roles, audit logging, CMEK where required, VPC Service Controls for sensitive perimeters, and Secret Manager for application configuration. Monitor CPU utilization by priority, storage, leader regions, request latency, transaction aborts, lock wait, query latency, rows scanned, and backup status. Use change streams or exports for downstream analytics rather than broad scans on the transactional database.

#### Note

Spanner cost is driven by provisioned capacity, storage, backups, change streams, network traffic, and inefficient queries. Because capacity is always on, label instances and export billing data so global OLTP cost is visible by product, environment, and tenant.

## 3.8 Interview Q&A

### 3.8.1 Concepts and Trade-offs

1. **Q: What problem does TrueTime solve in Spanner?**

A: TrueTime gives Spanner a bounded uncertainty interval for real time. Spanner uses commit wait so transaction timestamps respect real-time ordering, enabling external consistency.

2. **Q: Why can a monotonically increasing primary key hurt Spanner performance?**

A: New writes concentrate at the end of one key range, creating a hot split. A higher-cardinality leading key, hash bucket, or tenant-aware design distributes writes better.

3. **Q: When should you use an interleaved table?**

A: Use interleaving when child rows are usually read with the parent and share the parent lifecycle. Avoid it when children are independently scanned or one parent can become extremely large and hot.

4. **Q: How is Spanner different from Cloud SQL read replicas?**

A: Cloud SQL replicas are asynchronous read copies of a primary database. Spanner synchronously replicates data with consensus and exposes strong consistency across distributed replicas.

5. **Q: What evidence would you review before adding an index?**

A: Query text, parameters, execution plan, rows scanned, rows returned, latency, CPU, lock wait, and current index usage. Add the smallest index that improves the measured access path.

## 3.9 Further Reading

Read the Cloud Spanner documentation for current instance, database, schema, query, backup, and operations guidance [12]. Review schema design best practices before choosing primary keys, interleaving, and indexes [14]. Study query execution plans and optimizer documentation for performance tuning [13]. Use Key Visualizer and monitoring documentation to identify hot spots and operational anomalies [26]. Read the original Spanner paper to understand the research foundations behind TrueTime, Paxos, and external consistency [2].

## Key Takeaways

- Spanner is a globally consistent relational database, not a larger single-node database.
- TrueTime and Paxos let Spanner provide externally consistent transactions across distributed replicas.
- Primary keys define physical locality, so schema design directly affects scalability and latency.
- Interleaved tables colocate parent-child rows, while secondary indexes provide alternate access paths at synchronous write cost.
- Query plans, Key Visualizer, metrics, and transaction statistics should guide performance tuning.
- Multi-region Spanner is a strong fit for ledgers, entitlements, and other global systems that require zero data loss and strong consistency.

### 3.10 Exercises

1. **(Conceptual)** Explain external consistency and how it differs from ordinary eventual consistency.
2. **(Conceptual)** Compare regional Spanner, multi-region Spanner, Cloud SQL HA, and a traditional sharded relational database.
3. **(Design)** Propose a primary key for a global payments table and explain how it avoids write hot spots.
4. **(Design)** Decide whether carts and cart items should be interleaved for three different e-commerce access patterns.
5. **(Hands-on)** Create a sandbox multi-region Spanner instance and verify its instance configuration.
6. **(Hands-on)** Load 10,000 synthetic cart records and inspect a query plan for a cart lookup.
7. **(Performance)** Identify two indexes that could hot spot and redesign their leading columns.
8. **(Architecture)** Extend the payment-ledger use case with monitoring, audit logging, backup, restore, and analytics export requirements.

### 3.11 Capstone Project: Globally Consistent Payment Ledger on Cloud Spanner

#### Capstone Project

**Mission:** Design and prototype a globally consistent payment ledger on multi-region Cloud Spanner. Your solution must preserve zero data loss for committed transactions, support idempotent payment requests, survive a continental outage scenario, and produce auditable evidence for finance, risk, and compliance teams.

**Estimated time:** 6–8 hours.

**What you'll practice:**

- Creating a multi-region Spanner instance and payment-ledger database.
- Designing account, payment request, ledger entry, and balance tables with scalable primary keys.
- Implementing idempotent read-write transactions in Python.
- Creating secondary indexes for operational lookup without hot-spotting writes.
- Defining monitoring, backup, restore, and analytics export evidence for zero data loss.
- Producing a reference architecture, implementation transcript, acceptance criteria, and residual-risk statement.

#### Scenario

Globex Payments processes merchant card and wallet transactions in the Americas, Europe, and Asia. The current platform keeps one primary relational database per continent and reconciles conflicts overnight. Duplicate authorizations, delayed refunds, and inconsistent account holds have become unacceptable. Leadership wants one globally consistent ledger that can prove every committed debit and credit survives regional failures.

## Requirements

- Deploy a multi-region Cloud Spanner instance appropriate for a production payment ledger.
- Design tables for accounts, payment requests, ledger entries, account balances, and audit checkpoints.
- Enforce idempotency so retried authorization requests cannot double-post.
- Preserve zero data loss for committed transactions through synchronous replication and tested backups.
- Support merchant, account, request, and time-range lookups with measured indexes and query plans.
- Export committed ledger facts to BigQuery or another analytical system without overloading OLTP transactions.
- Document IAM, database roles, CMEK, audit logging, monitoring, alert thresholds, and incident runbooks.
- Provide validation evidence for balance reconstruction, duplicate suppression, transaction ordering, and restore testing.

## Reference Architecture

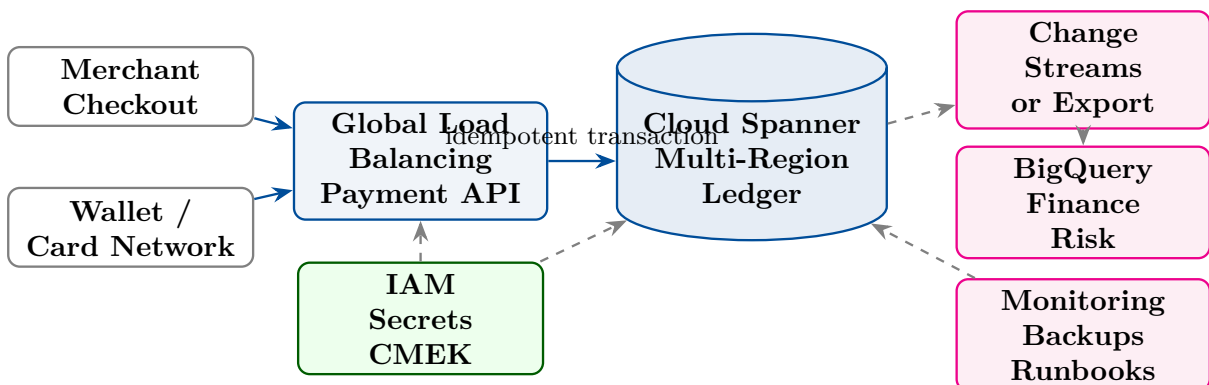


Figure 3.4: Capstone reference architecture for a globally consistent payment ledger on multi-region Spanner.

## Implementation Steps

1. **Create the Spanner instance, database, and ledger schema.** Replace all placeholders and use a sandbox project first.

```

1 $env:PROJECT_ID = "my-gcp-ledger-capstone"
2 $env:INSTANCE_ID = "global-ledger-prod"
3 $env:DATABASE_ID = "ledgerdb"
4 $env:INSTANCE_CONFIG = "nam-eur-asia1"
5 $env:PROCESSING_UNITS = "1000"
6
7 gcloud config set project $env:PROJECT_ID
8 gcloud services enable spanner.googleapis.com
9
10 gcloud spanner instances create $env:INSTANCE_ID '

```

```

11  --config=$env:INSTANCE_CONFIG '
12  --description="Globally consistent payment ledger" '
13  --processing-units=$env:PROCESSING_UNITS
14
15  gcloud spanner databases create $env:DATABASE_ID '
16  --instance=$env:INSTANCE_ID
17
18  $ddl = @'
19  CREATE TABLE Accounts (
20    AccountId STRING(36) NOT NULL,
21    MerchantId STRING(36) NOT NULL,
22    HomeRegion STRING(16) NOT NULL,
23    Currency STRING(3) NOT NULL,
24    CreatedAt TIMESTAMP NOT NULL OPTIONS (allow_commit_timestamp=true),
25  ) PRIMARY KEY (AccountId);
26
27  CREATE TABLE PaymentRequests (
28    RequestId STRING(64) NOT NULL,
29    AccountId STRING(36) NOT NULL,
30    MerchantId STRING(36) NOT NULL,
31    Amount NUMERIC NOT NULL,
32    Currency STRING(3) NOT NULL,
33    Status STRING(16) NOT NULL,
34    CreatedAt TIMESTAMP NOT NULL OPTIONS (allow_commit_timestamp=true),
35  ) PRIMARY KEY (RequestId);
36
37  CREATE TABLE LedgerEntries (
38    AccountId STRING(36) NOT NULL,
39    EntryId STRING(36) NOT NULL,
40    RequestId STRING(64) NOT NULL,
41    Direction STRING(6) NOT NULL,
42    Amount NUMERIC NOT NULL,
43    Currency STRING(3) NOT NULL,
44    CommittedAt TIMESTAMP NOT NULL OPTIONS (allow_commit_timestamp=true),
45  ) PRIMARY KEY (AccountId, EntryId),
46    INTERLEAVE IN PARENT Accounts ON DELETE NO ACTION;
47
48  CREATE TABLE AccountBalances (
49    AccountId STRING(36) NOT NULL,
50    Currency STRING(3) NOT NULL,
51    AvailableBalance NUMERIC NOT NULL,
52    UpdatedAt TIMESTAMP NOT NULL OPTIONS (allow_commit_timestamp=true),
53  ) PRIMARY KEY (AccountId, Currency);
54
55  CREATE INDEX PaymentRequestsByMerchant
56  ON PaymentRequests(MerchantId, CreatedAt DESC)
57  STORING (Status, Amount, Currency);
58  '@
59
60  $ddl | gcloud spanner databases ddl update $env:DATABASE_ID '
61  --instance=$env:INSTANCE_ID '
62  --ddl-file=-

```

Listing 3.5: Capstone Spanner payment-ledger database and schema creation.

2. **Prototype idempotent posting logic.** The transaction first checks the request identifier, then writes the request, ledger entry, and balance update atomically.

```

1  import os
2  import uuid
3  from decimal import Decimal
4
5  from google.cloud import spanner
6  from google.cloud.spanner_v1 import COMMIT_TIMESTAMP

```

```

7
8
9 PROJECT_ID = os.environ["PROJECT_ID"]
10 INSTANCE_ID = os.environ.get("INSTANCE_ID", "global-ledger-prod")
11 DATABASE_ID = os.environ.get("DATABASE_ID", "ledgerdb")
12
13
14 def post_payment(transaction, request_id, account_id, merchant_id, amount,
15 currency):
16     existing = list(transaction.execute_sql(
17         "SELECT Status FROM PaymentRequests WHERE RequestId = @request_id",
18         params={"request_id": request_id},
19         param_types={"request_id": spanner.param_types.STRING},
20     ))
21     if existing:
22         return {"request_id": request_id, "status": existing[0][0], "
23 duplicate": True}
24
25     entry_id = str(uuid.uuid4())
26     balance_rows = list(transaction.execute_sql(
27         """
28         SELECT AvailableBalance
29         FROM AccountBalances
30         WHERE AccountId = @account_id AND Currency = @currency
31         """,
32         params={"account_id": account_id, "currency": currency},
33         param_types={
34             "account_id": spanner.param_types.STRING,
35             "currency": spanner.param_types.STRING,
36         },
37     ))
38     current_balance = balance_rows[0][0] if balance_rows else Decimal("0")
39     new_balance = current_balance + amount
40
41     transaction.insert(
42         table="PaymentRequests",
43         columns=("RequestId", "AccountId", "MerchantId", "Amount", "
44 Currency", "Status", "CreatedAt"),
45         values=[(request_id, account_id, merchant_id, amount, currency, "
46 POSTED", COMMIT_TIMESTAMP)],
47     )
48     transaction.insert(
49         table="LedgerEntries",
50         columns=("AccountId", "EntryId", "RequestId", "Direction", "Amount"
51 , "Currency", "CommittedAt"),
52         values=[(account_id, entry_id, request_id, "CREDIT", amount,
53 currency, COMMIT_TIMESTAMP)],
54     )
55     transaction.insert_or_update(
56         table="AccountBalances",
57         columns=("AccountId", "Currency", "AvailableBalance", "UpdatedAt"),
58         values=[(account_id, currency, new_balance, COMMIT_TIMESTAMP)],
59     )
60     return {"request_id": request_id, "status": "POSTED", "duplicate":
61 False}
62
63
64 def main() -> None:
65     client = spanner.Client(project=PROJECT_ID)
66     database = client.instance(INSTANCE_ID).database(DATABASE_ID)
67     result = database.run_in_transaction(
68         post_payment,
69         request_id="merchant-123-order-456",

```

```

63     account_id="acct-001",
64     merchant_id="merchant-123",
65     amount=Decimal("42.50"),
66     currency="USD",
67 )
68 print(result)
69
70
71 if __name__ == "__main__":
72     main()

```

Listing 3.6: Idempotent payment posting with a Spanner read-write transaction.

3. **Capture operations evidence.** Record query plans, backup configuration, and restore-test notes as part of the project transcript.

```

1 gcloud spanner databases execute-sql $env:DATABASE_ID '
2   --instance=$env:INSTANCE_ID '
3   --query-mode=PLAN '
4   --sql="SELECT RequestId, Status FROM PaymentRequests WHERE MerchantId = '
5       merchant-123' ORDER BY CreatedAt DESC LIMIT 20"
6
6 gcloud spanner backups create ledgerdb-backup-$(Get-Date -Format
7   yyyyMMddHHmm) '
8   --instance=$env:INSTANCE_ID '
9   --database=$env:DATABASE_ID '
10  --retention-period=7d
11
11 gcloud spanner backups list --instance=$env:INSTANCE_ID

```

Listing 3.7: Operational evidence commands for the payment-ledger capstone.

## Deliverables

- A one-page architecture summary explaining why Spanner is required instead of Cloud SQL, AlloyDB, or a sharded RDBMS.
- The completed reference diagram or an equivalent diagram showing merchants, payment APIs, Spanner, security controls, analytics export, monitoring, and backups.
- DDL and command transcript proving instance creation, schema deployment, indexes, backup configuration, and cleanup plan.
- Python or pseudocode evidence for idempotent posting and duplicate-request suppression.
- Query-plan evidence for merchant lookup, account ledger lookup, and balance reconstruction.
- A zero-data-loss runbook covering regional failure, restore testing, alert response, and post-incident reconciliation.

## Acceptance Criteria

- The Spanner instance uses a multi-region configuration and an explicit capacity plan.
- Primary keys distribute writes while preserving efficient account-ledger and merchant-request access.
- Payment posting is idempotent and writes request, ledger, and balance records in one read-write transaction.
- The design documents how committed transactions survive replica or regional failures.

- Validation reconstructs balances from ledger entries and proves duplicate requests are not double-applied.
- Monitoring covers latency percentiles, abort rates, CPU, lock wait, storage, hot splits, backup status, and failed exports.
- Analytics uses change streams, export, or controlled replication rather than unbounded OLTP scans.
- Secrets, credentials, payment identifiers, and customer data are protected in commands, logs, and screenshots.

### Stretch Goals

- Add change streams from Spanner into Dataflow and BigQuery for near-real-time risk scoring.
- Implement double-entry accounting with debit and credit entries that must balance within each transaction.
- Add a reconciliation notebook that rebuilds account balances from immutable ledger entries.
- Create a synthetic load test that measures p50, p95, and p99 authorization latency by region.
- Add a backup restore rehearsal into a separate Spanner instance and validate row counts and ledger totals.
- Estimate monthly cost for capacity, storage, backups, change streams, network transfer, monitoring, and analytics export.

# Wide-Column NoSQL: Cloud Bigtable

## Learning Objectives

- Explain Cloud Bigtable architecture, including instances, clusters, nodes, tablets, column families, and app profiles.
- Design row keys that distribute high-throughput reads and writes while preserving efficient access patterns.
- Apply salting, hashing, reversed timestamps, and entity-prefix layouts to avoid hotspotting.
- Tune Bigtable performance with node sizing, schema choices, garbage collection policies, and Key Visualizer.
- Deploy a Bigtable cluster and ingest IoT temperature-sensor readings with a Python script and reversed-timestamp row key.
- Design an ingestion and serving layer for a financial trading platform processing 100,000 stock ticks per second.
- Complete the Part I capstone by integrating Cloud SQL, Cloud Spanner, Cloud Bigtable, and Cloud Storage behind secure private connectivity.

## Cross-Cloud Equivalence

Wide-column and high-throughput NoSQL systems expose different APIs, but the core design questions are consistent: partition key, row-key order, hot-range prevention, throughput scaling, consistency, and operational visibility.

| Capability                 | GCP Bigtable                                                                  | AWS                                                                        | Azure                                                                         | Apache HBase / Cassandra                                                       |
|----------------------------|-------------------------------------------------------------------------------|----------------------------------------------------------------------------|-------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Wide-column model          | Sparse rows, column families, cells, timestamps, Bigtable API                 | DynamoDB items and attributes; Keyspaces uses Cassandra tables             | Cosmos DB Cassandra API; Table Storage entities with partition and row keys   | HBase tables and column families; Cassandra wide rows and clustering columns   |
| Row-key / partition design | Row key controls tablet locality and scan order                               | DynamoDB partition key and sort key; Keyspaces partition key               | Cosmos partition key and clustering; Table Storage partition key plus row key | HBase row key regions; Cassandra partition key and clustering order            |
| Hotspotting mitigation     | Salting, hashing, bucketing, reversed timestamps, app-profile routing         | Write sharding, adaptive capacity, random prefixes, capacity planning      | Synthetic partition keys, partition spread, RU planning                       | Region pre-splitting, token distribution, bucketing, compaction-aware modeling |
| Tablets vs partitions      | Tablets are contiguous row-key ranges served by nodes and split automatically | DynamoDB partitions and Keyspaces storage partitions distribute load       | Logical partitions map to physical partitions                                 | HBase regions and Cassandra token ranges distribute ownership                  |
| Throughput scaling         | Add nodes per cluster; replicate clusters; route with app profiles            | Provisioned or on-demand capacity; table and global secondary index limits | Request units and partition throughput limits                                 | Add region servers or Cassandra nodes and rebalance tokens                     |

MongoDB can serve high-volume document workloads, but it is not a direct wide-column replacement because its shard key, document model, and secondary-index behavior differ. Snowflake is excellent for analytical serving, yet it is not designed as a single-digit-millisecond operational key-value store.

### Key Terms

**Cloud Bigtable:** Google Cloud’s managed, horizontally scalable wide-column database for low-latency, high-throughput operational and analytical workloads. **Tablet:** a contiguous range of row keys assigned to servers and split as data or traffic grows. **Row key:** the byte string that determines physical ordering, locality, and scan behavior in a Bigtable table. **Column family:** a grouping of columns that share storage, compression, and garbage collection policy. **App profile:** a routing and transaction policy that sends client traffic to one or more clusters in a Bigtable instance. **Hotspot:** a narrow row-key range or node that receives disproportionate reads or writes.

## 4.1 Week 4 Roadmap

Week 4 closes Part I by moving from relational infrastructure to high-throughput wide-column storage. Cloud Bigtable is the GCP service to evaluate when the workload needs predictable low latency at large scale, sparse data, very high write rates, and key-range access patterns rather than ad hoc relational joins [8]. It descends from the Bigtable system that influenced HBase, Cassandra, and many distributed storage systems [1].

Monday introduces Bigtable architecture: instances, clusters, nodes, tablets, column families, timestamps, replication, and app profiles. Tuesday focuses on row-key design, salting, hashing, and hotspot avoidance. Wednesday studies performance tuning, garbage collection, Key Visualizer, and operational evidence. Thursday builds a hands-on lab that ingests IoT temperature-

sensor data with reversed-timestamp row keys. Friday applies the same principles to a financial trading platform that must ingest and serve 100,000 stock ticks per second.

#### Definition

**Cloud Bigtable** is a managed wide-column store that sorts all rows lexicographically by row key. It is optimized for high-throughput point lookups, prefix scans, time-series patterns, feature stores, personalization, telemetry, and serving layers where query access is known before the schema is designed.

## 4.2 Monday: Bigtable Architecture, Tablets, Nodes, and App Profiles

### 4.2.1 Instances, Clusters, Nodes, Tables, and Column Families

A Bigtable instance is an administrative boundary that can contain one or more clusters. A cluster is a set of Bigtable nodes in one zone. Nodes provide CPU, memory, and network capacity for serving reads and writes. Tables live inside the instance, and each table stores rows ordered by row key. Columns are grouped into column families, and each cell can have multiple timestamped versions.

Unlike Cloud SQL or Spanner, Bigtable does not enforce relational constraints, joins, or multi-row SQL transactions. Its strength is serving known access patterns at massive scale. You model the table from the query first: identify the entity, time range, prefix scan, latest-state lookup, or reverse-time feed that the application must read quickly. Then choose a row key and column families that make that read path local and predictable.

#### Note

Bigtable is sparse. A row can have thousands of possible columns, but only stored cells consume space. This is useful for telemetry, feature vectors, metrics, and event attributes where different entities report different measurements.

### 4.2.2 Tablets and Node-Level Sharding

Bigtable stores each table as tablets. A tablet is a contiguous row-key range such as `sensor#03#...` through `sensor#04#...`. Tablets can split as the table grows, and Bigtable assigns tablets across nodes in a cluster. The row key therefore controls both scan locality and write distribution. If every new write lands at the same end of the keyspace, only a small number of tablets and nodes are busy even when the cluster has many nodes.

#### Definition

**Lexicographic row-key order** means that row keys are sorted byte by byte. Prefixes are powerful because they make range scans efficient, but low-cardinality or monotonically increasing prefixes can create hot tablets.

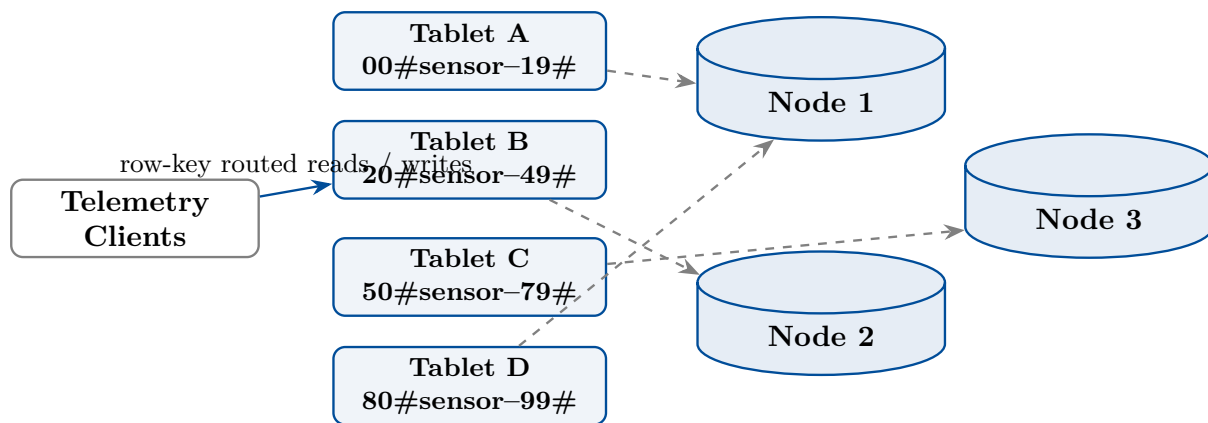


Figure 4.1: Bigtable splits a table into tablets, then serves those row-key ranges across cluster nodes.

### 4.2.3 Replication and App Profiles

A Bigtable instance can replicate data across clusters. Replication supports availability, locality, and workload isolation, but it also requires careful routing decisions. App profiles define whether a client routes to one cluster or multiple clusters and whether single-row transactions are allowed. For example, an ingestion app profile might send writes to a regional cluster near devices, while an analytics app profile might route batch reads to a separate cluster so scans do not compete with serving traffic.

#### Tip

Use app profiles as part of application design, not as an afterthought. Separate ingestion, serving, analytics, and backfill traffic when their latency and retry behavior differ.

## 4.3 Tuesday: Row-Key Design, Salting, Hashing, and Hotspot Avoidance

### 4.3.1 Design from Access Patterns

A Bigtable schema is successful when the row key makes the critical query a short point lookup or a bounded range scan. A row key such as `device#bucket#reverse_timestamp` works for recent events by device. A row key such as `symbol#bucket#reverse_timestamp` works for recent market ticks by symbol. A row key such as `tenant#hash#entity` works for multi-tenant entity lookup when tenants need isolation but large tenants must spread traffic.

The dangerous row key is often the obvious one. A key beginning with the current timestamp makes chronological scans simple, but it concentrates all new writes at the newest range. A key beginning with a single popular symbol, merchant, or device can also become hot. A good design balances three pressures: distributing writes, preserving read locality, and keeping the key understandable enough to operate.

#### Common Pitfall

Do not use an unbounded timestamp as the leading row-key component for high-volume ingestion. It usually creates a moving hot spot at the end of the table, even when the cluster has enough total nodes.

### 4.3.2 Salting, Hashing, Bucketing, and Reverse Time

Salting adds a small prefix such as 00 through 63 before the natural key. Hashing derives the prefix from stable fields such as device identifier or account identifier. Bucketing groups time into coarse windows such as hour or day. Reversed timestamps invert chronological order so the newest data appears first inside an entity prefix. These techniques are often combined: `hash#symbol#reversed_millis#trade_id` distributes writes while supporting recent-tick scans by symbol and bucket fan-out.

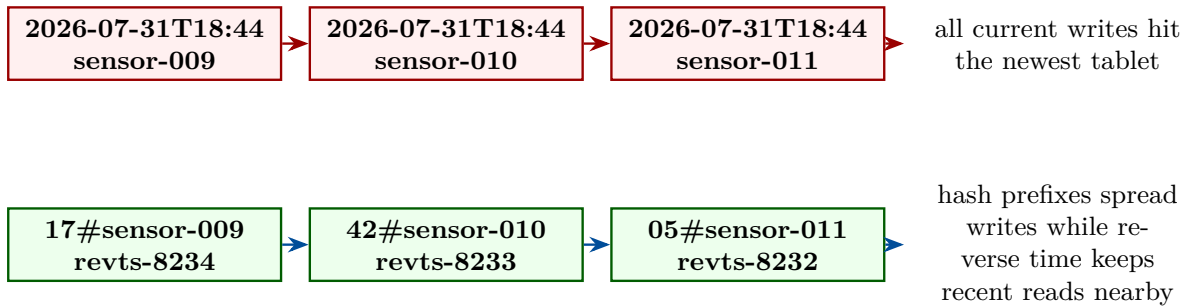


Figure 4.2: A timestamp-leading key hot spots current writes, while a salted reversed-timestamp key spreads ingestion and preserves recent-event scans.

### 4.3.3 Column Families and Cell Versions

Column families should group data with similar lifecycle, compression, and access behavior. A time-series table might place measurements in a `metrics` family and metadata in a `meta` family. A trading table might keep raw ticks in `tick`, derived indicators in `signal`, and audit ingestion markers in `audit`. Avoid creating many column families without a reason; each family has configuration and storage implications.

#### Note

Bigtable cells are timestamped. You can store multiple versions of a value, but garbage collection policy determines how many versions or how much history remains. Versioning is not a substitute for a deliberate event-history schema.

## 4.4 Wednesday: Performance Optimization, Garbage Collection, and Key Visualizer

### 4.4.1 Throughput, Nodes, and Workload Isolation

Bigtable performance depends on schema, cluster size, storage type, request mix, replication, and client behavior [9]. Adding nodes increases serving capacity, but it cannot fully compensate for a hot row key. Before scaling up, inspect CPU, latency, request count, error rate, storage utilization, and Key Visualizer heatmaps. If one range is hot while other nodes are idle, fix the row-key distribution first.

Workload isolation is equally important. A batch export or backfill that scans broad key ranges can affect latency for online serving. Use separate clusters, app profiles, scheduled windows, Dataflow connector throttling, and bounded scans. For high-throughput writes, batch mutations, reuse clients, use retry settings intentionally, and keep row mutations reasonably sized.

**Tip**

Tune in this order: verify the row key, verify the access path, isolate workload classes, tune client batching and retries, then add nodes. Scaling a poor key often hides the problem temporarily and raises cost permanently.

**4.4.2 Garbage Collection Policies**

Garbage collection policies control old cell versions. A max-age policy keeps cells younger than a specified duration. A max-versions policy keeps a fixed number of versions. Intersection and union policies combine those rules. Use them to align storage cost with retention requirements. For IoT readings, raw high-frequency telemetry may need only 30 days in Bigtable before being archived to Cloud Storage or BigQuery. For financial ticks, compliance may require immutable downstream storage even if the serving table retains only recent windows.

**Definition**

**Bigtable garbage collection** removes cells according to configured family policies during compaction. It is not an immediate delete guarantee for compliance workflows; design retention, export, and legal hold requirements explicitly.

**4.4.3 Key Visualizer and Heatmap Evidence**

Key Visualizer displays access patterns by row-key range over time [25]. Bright diagonal lines often indicate a moving timestamp hot spot. Vertical bands can indicate one hot prefix or tenant. Broad evenly distributed color suggests healthy spread. Use the heatmap together with Cloud Monitoring metrics because a quiet heatmap can still hide client errors, failed retries, or inefficient downstream processing.

**Common Pitfall**

A salted key requires read fan-out. If an application needs the latest 100 rows for a symbol and the table has 64 salt buckets, the service may need to query 64 prefixes and merge results. Use only as many buckets as the write rate requires.

| Signal                               | Likely meaning                       | Response                                                                           |
|--------------------------------------|--------------------------------------|------------------------------------------------------------------------------------|
| High latency and one hot key range   | Row-key hotspotting                  | Add hash or salt prefix, pre-split when appropriate, or re-design leading key      |
| High CPU on all nodes                | Cluster capacity pressure            | Add nodes, reduce broad scans, batch requests, or isolate workloads                |
| Storage growing faster than expected | Retention or version policy mismatch | Review garbage collection, archive raw history, and compress family design         |
| Many failed retries                  | Client backoff or quota issue        | Tune deadlines, retry policy, batching, and app-profile routing                    |
| Analytical scans hurting serving     | Workload contention                  | Use a separate replicated cluster or route batch jobs with a different app profile |

Table 4.1: Bigtable performance diagnosis patterns.

## 4.5 Thursday Hands-On Project: IoT Temperature Sensor Ingestion

This lab creates a single-cluster Bigtable instance, configures a table for temperature telemetry, and writes synthetic sensor events with reversed-timestamp row keys. Use a sandbox project, replace placeholders, and delete resources when finished. Commands are written for Windows PowerShell.

### Note

Bigtable clusters can incur cost while running. Use a small development cluster, label resources, and delete the instance immediately after the lab unless you are intentionally keeping it for later exercises.

### 4.5.1 PowerShell Preparation and Cluster Creation

```

1 $env:PROJECT_ID = "my-gcp-bigtable-lab"
2 $env:INSTANCE_ID = "iot-telemetry-lab"
3 $env:CLUSTER_ID = "iot-telemetry-c1"
4 $env:ZONE = "us-central1-b"
5 $env:TABLE_ID = "temperature_readings"
6
7 gcloud config set project $env:PROJECT_ID
8 gcloud services enable bigtable.googleapis.com bigtableadmin.googleapis.com
9
10 gcloud bigtable instances create $env:INSTANCE_ID '
11   --display-name="IoT telemetry lab" '
12   --cluster=$env:CLUSTER_ID '
13   --cluster-zone=$env:ZONE '
14   --cluster-num-nodes=1 '
15   --cluster-storage-type=SSD
16
17 cbt -project $env:PROJECT_ID -instance $env:INSTANCE_ID createtable $env:
18   TABLE_ID
19 cbt -project $env:PROJECT_ID -instance $env:INSTANCE_ID createfamily $env:
20   TABLE_ID metrics
21 cbt -project $env:PROJECT_ID -instance $env:INSTANCE_ID createfamily $env:
22   TABLE_ID meta
23 cbt -project $env:PROJECT_ID -instance $env:INSTANCE_ID setgcpcpolicy $env:
24   TABLE_ID metrics maxage=30d

```

Listing 4.1: Creating a Bigtable instance, cluster, and telemetry table from PowerShell.

### 4.5.2 Python Ingestion Script

The script uses a two-digit hash bucket, sensor identifier, and reversed millisecond timestamp. Recent readings for one sensor can be found by scanning that sensor's bucketed prefixes, while writes from many sensors spread across the keyspace.

```

1 import hashlib
2 import os
3 import random
4 import time
5 from datetime import datetime, timezone
6
7 from google.cloud import bigtable
8
9
10 PROJECT_ID = os.environ["PROJECT_ID"]
11 INSTANCE_ID = os.environ.get("INSTANCE_ID", "iot-telemetry-lab")

```

```

12 TABLE_ID = os.environ.get("TABLE_ID", "temperature_readings")
13 MAX_TS = 9999999999999
14 SENSOR_COUNT = 200
15 READINGS_PER_SENSOR = 25
16
17
18 def bucket_for(sensor_id: str, buckets: int = 64) -> str:
19     digest = hashlib.sha256(sensor_id.encode("utf-8")).hexdigest()
20     return f"{int(digest[:4], 16) % buckets:02d}"
21
22
23 def make_row_key(sensor_id: str, event_ms: int) -> bytes:
24     reverse_ms = MAX_TS - event_ms
25     return f"{bucket_for(sensor_id)}#{sensor_id}#{reverse_ms:013d}".encode("utf-8")
26
27
28 def main() -> None:
29     client = bigtable.Client(project=PROJECT_ID, admin=True)
30     instance = client.instance(INSTANCE_ID)
31     table = instance.table(TABLE_ID)
32     mutations = []
33
34     now_ms = int(time.time() * 1000)
35     for sensor_number in range(SENSOR_COUNT):
36         sensor_id = f"sensor-{sensor_number:04d}"
37         for offset in range(READINGS_PER_SENSOR):
38             event_ms = now_ms - offset * 60000 - random.randint(0, 5000)
39             row = table.direct_row(make_row_key(sensor_id, event_ms))
40             temperature = 18.0 + random.random() * 12.0
41             event_time = datetime.fromtimestamp(event_ms / 1000, tz=timezone.
42             utc)
43             row.set_cell("metrics", "temp_c", f"{temperature:.2f}")
44             row.set_cell("metrics", "humidity_pct", f"{random.uniform(30, 80)
45             :.1f}")
46             row.set_cell("meta", "sensor_type", "warehouse-thermistor")
47             row.set_cell("meta", "event_time", event_time.isoformat())
48             mutations.append(row)
49
50     table.mutate_rows(mutations)
51     print({"rows_written": len(mutations), "table": TABLE_ID})
52
53 if __name__ == "__main__":
54     main()

```

Listing 4.2: Ingesting IoT temperature readings with google-cloud-bigtable and reversed-timestamp row keys.

### 4.5.3 Read Check and Cleanup

```

1 cbt -project $env:PROJECT_ID -instance $env:INSTANCE_ID read $env:TABLE_ID
  count=10
2
3 python .\ingest-iot-bigtable.py
4
5 cbt -project $env:PROJECT_ID -instance $env:INSTANCE_ID read $env:TABLE_ID
  count=10
6
7 gcloud bigtable instances delete $env:INSTANCE_ID --quiet

```

Listing 4.3: Reading sample rows and deleting the Bigtable lab instance.

## 4.6 Friday Use Case: 100,000 Stock Ticks per Second

### 4.6.1 Scenario

A trading platform receives quotes and trades from multiple exchanges. The target rate is 100,000 stock ticks per second during market peaks. Downstream systems need immediate lookup of the latest ticks by symbol, short historical windows for charting, anomaly detection features, and replay into analytics. The platform must absorb bursts without putting the serving path or compliance archive at risk.

### 4.6.2 Architecture Pattern

Pub/Sub absorbs exchange events and decouples producers from consumers. Dataflow validates, normalizes, enriches, and writes tick rows to Bigtable using a salted symbol and reversed-timestamp key. Bigtable serves recent market data to APIs with single-digit-millisecond lookups and bounded prefix scans. Cloud Storage stores immutable raw feed files, while BigQuery receives curated analytical facts for surveillance, reporting, and model training.

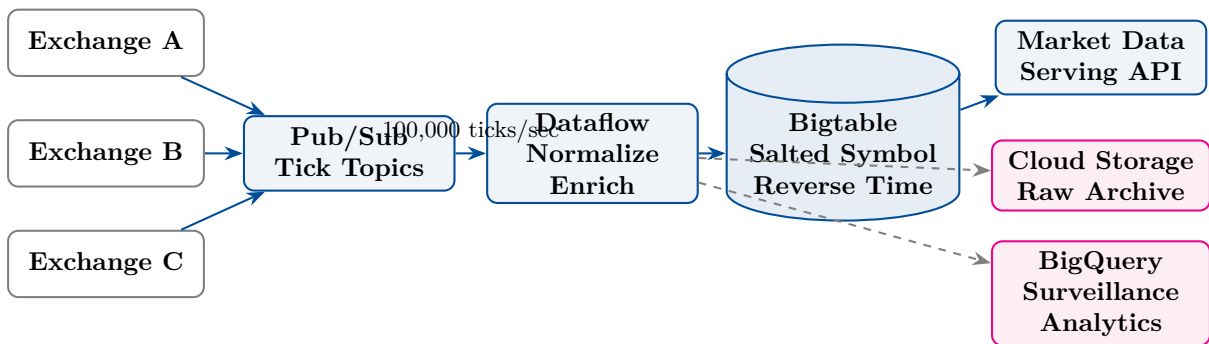


Figure 4.3: Trading ingestion uses Pub/Sub and Dataflow for burst absorption, Bigtable for low-latency serving, and analytical stores for archive and surveillance.

### 4.6.3 Serving and Correctness Controls

A practical tick row key is `bucket#symbol#reverse_event_nanos#exchange#sequence`. The bucket spreads writes, the symbol supports bounded reads, reverse time returns latest ticks first, and exchange sequence numbers support de-duplication. Use separate column families for raw tick fields, normalized fields, and derived indicators. Keep the serving table focused on recent windows; archive immutable raw feeds and replay checkpoints to Cloud Storage for audit and recovery.

#### Tip

For trading systems, prove both speed and replayability. Bigtable serves the current operational window, while Cloud Storage and BigQuery preserve raw history, reconstructed facts, and compliance evidence.

## 4.7 Security, Operations, and Cost Notes

Use least-privilege IAM, service accounts per workload, CMEK where required, VPC Service Controls for sensitive projects, audit logging, and private application paths where possible. Bigtable access is usually from application code, Dataflow, or GKE/GCE workloads using

IAM rather than database usernames. Monitor node CPU, latency percentiles, request errors, storage, replication lag, hot ranges, garbage collection policy, and client retry behavior.

#### Note

Bigtable cost is driven by nodes, storage type, stored bytes, backups, replication, network, and inefficient scans. A table that retains every raw version forever can become more expensive than the serving workload itself.

## 4.8 Interview Q&A

### 4.8.1 Concepts and Trade-offs

1. **Q: Why is row-key design the most important Bigtable schema decision?**

A: The row key determines physical sort order, tablet ranges, scan locality, and whether reads and writes distribute across nodes.

2. **Q: What is a tablet in Bigtable?**

A: A tablet is a contiguous row-key range that Bigtable can split, move, and serve independently across cluster nodes.

3. **Q: Why use a reversed timestamp in a time-series key?**

A: It places the newest events first inside an entity prefix, so recent-history reads can stop after a small range scan.

4. **Q: What is the downside of salting row keys?**

A: Salting spreads writes but can require read fan-out across buckets and result merging for entity-level queries.

5. **Q: How do app profiles help production systems?**

A: They route client traffic to specific clusters or multi-cluster policies, allowing workload isolation and locality-aware access.

6. **Q: When is Bigtable a poor fit?**

A: It is a poor fit for ad hoc joins, relational constraints, small departmental OLTP, and workloads without predictable key-based access paths.

## 4.9 Further Reading

Read the Cloud Bigtable documentation for current table, cluster, replication, backup, and administration guidance [8]. Study schema design guidance before committing to row keys and column families [10]. Review performance guidance for node sizing, workload isolation, and request patterns [9]. Use Key Visualizer documentation to interpret heatmaps and diagnose hot ranges [25]. Read the original Bigtable paper to understand the storage model and design lineage [1].

## Key Takeaways

- Bigtable is a managed wide-column database for predictable, key-based access at very high throughput.
- Row keys determine locality and distribution; poor leading keys create hot tablets regardless of total cluster capacity.

- Salting, hashing, bucketing, and reversed timestamps trade read simplicity for write distribution and recent-event access.
- Column families should reflect access and retention patterns because garbage collection is configured at the family level.
- Key Visualizer, Cloud Monitoring, and client metrics should guide tuning before adding nodes.
- Bigtable often pairs with Pub/Sub, Dataflow, Cloud Storage, and BigQuery to separate ingestion, serving, archive, and analytics responsibilities.

## 4.10 Exercises

1. **(Conceptual)** Explain how a Bigtable tablet differs from a Spanner split and from a DynamoDB partition.
2. **(Conceptual)** Describe why Bigtable is not a relational database even though it stores tables, rows, and columns.
3. **(Design)** Propose a row key for a fleet of 2 million vehicles reporting location every five seconds.
4. **(Design)** Redesign a timestamp-leading key that is creating a moving hotspot during ingestion.
5. **(Hands-on)** Create the IoT lab table and write at least 1,000 readings using the Python ingestion script.
6. **(Performance)** Interpret a Key Visualizer heatmap that shows one vertical red band for three hours.
7. **(Operations)** Define garbage collection policies for raw telemetry, latest device state, and derived alert flags.
8. **(Architecture)** Extend the trading use case with disaster recovery, replay, monitoring, and cost controls.

## 4.11 Milestone 1 Capstone: E-Commerce Multi-Tier Data Backend

### Capstone Project

**Mission:** Design and prototype the Part I e-commerce data backend. Integrate Cloud SQL for user accounts, Cloud Spanner for global inventory and checkout transactions, Cloud Bigtable for clickstream tracking, and Cloud Storage for product images with lifecycle management and CMEK. Secure the tiers with VPC peering, private services access, private IP, service accounts, and least-privilege IAM.

**Estimated time:** 8–10 hours.

#### What you'll practice:

- Mapping Week 1 Cloud Storage image buckets with lifecycle rules, uniform access, and CMEK.
- Using Week 2 Cloud SQL private IP for user-account and profile data.
- Using Week 3 Cloud Spanner for globally consistent inventory reservations and checkout transactions.

- Using Week 4 Cloud Bigtable for clickstream events, product impressions, and recent behavior serving.
- Designing VPC peering and private connectivity so application tiers do not depend on public database endpoints.
- Producing acceptance evidence across security, data modeling, operations, and cleanup.

## Scenario

Mercury Retail runs an international e-commerce platform. User profile updates are regional and relational. Inventory reservations and checkout must remain globally consistent. Product images need durable object storage with lifecycle transitions and customer-managed encryption keys. Clickstream events arrive at high volume and must support recent-session personalization. Leadership wants a single reference design that proves Part I services can work together securely.

## Requirements

- Store user accounts, hashed credentials, profile settings, and consent flags in Cloud SQL with private IP.
- Store global inventory, carts, checkout orders, and reservation transactions in Cloud Spanner.
- Store clickstream events in Cloud Bigtable using a salted user or session prefix and reversed timestamp.
- Store product images in Cloud Storage with uniform bucket-level access, lifecycle rules, versioning where appropriate, and CMEK.
- Use a shared application VPC with private services access or required service networking peering for Cloud SQL and other private managed-service paths.
- Use least-privilege service accounts for web API, checkout worker, image pipeline, and clickstream ingester.
- Document IAM, KMS keys, audit logs, monitoring dashboards, backup policies, and cleanup commands.
- Provide validation evidence for private connectivity, row-key distribution, Spanner transaction correctness, Cloud SQL backups, and Storage lifecycle configuration.

## Reference Architecture

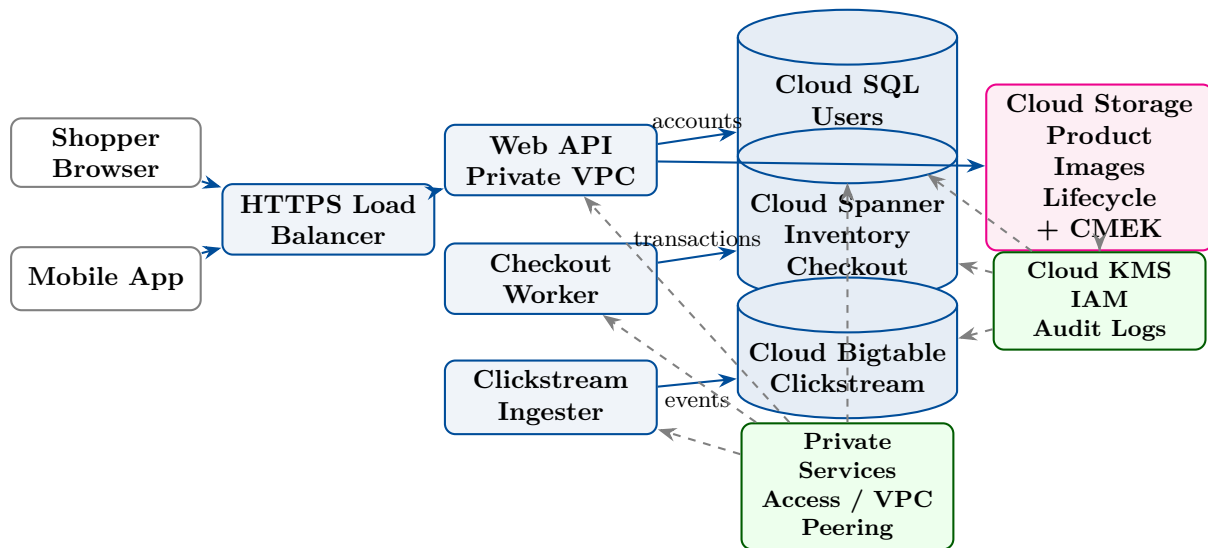


Figure 4.4: Milestone 1 reference architecture integrating Cloud Storage, Cloud SQL, Spanner, and Bigtable behind private application connectivity.

## Implementation Steps

1. Create the shared network, private services access range, KMS key, image bucket, and managed data services. Replace placeholders and use a sandbox project.

```

1 $env:PROJECT_ID = "my-gcp-ecommerce-capstone"
2 $env:REGION = "us-central1"
3 $env:ZONE = "us-central1-b"
4 $env:VPC = "ecommerce-private-vpc"
5 $env:RANGE = "google-managed-services-range"
6 $env:KEYRING = "ecommerce-keys"
7 $env:KEY = "data-backend-key"
8 $env:BUCKET = "my-ecommerce-product-images"
9
10 gcloud config set project $env:PROJECT_ID
11 gcloud services enable compute.googleapis.com servicenetworking.googleapis.com '
12   sqladmin.googleapis.com spanner.googleapis.com bigtable.googleapis.com '
13   bigtableadmin.googleapis.com storage.googleapis.com cloudkms.googleapis.com
14
15 gcloud compute networks create $env:VPC --subnet-mode=custom
16 gcloud compute networks subnets create app-subnet '
17   --network=$env:VPC --range=10.20.0.0/24 --region=$env:REGION
18
19 gcloud compute addresses create $env:RANGE '
20   --global --purpose=VPC_PEERING --prefix-length=20 --network=$env:VPC
21
22 gcloud services vpc-peerings connect '
23   --service=servicenetworking.googleapis.com '
24   --ranges=$env:RANGE '
25   --network=$env:VPC
26
27 gcloud kms keyrings create $env:KEYRING --location=$env:REGION
28 gcloud kms keys create $env:KEY --location=$env:REGION '
29   --keyring=$env:KEYRING --purpose=encryption

```

```

30
31 gcloud storage buckets create gs://$env:BUCKET '
32   --location=$env:REGION '
33   --uniform-bucket-level-access '
34   --default-encryption-key=projects/$env:PROJECT_ID/locations/$env:REGION/
    keyRings/$env:KEYRING/cryptoKeys/$env:KEY
35
36 gcloud storage buckets update gs://$env:BUCKET --versioning

```

Listing 4.4: Creating the Milestone 1 network, storage, and managed-service foundations.

2. **Deploy representative schemas and write one clickstream event.** This prototype shows the service boundaries; production code should add migrations, secret management, and CI validation.

```

1 import hashlib
2 import os
3 import time
4 import uuid
5
6 from google.cloud import bigtable
7
8
9 PROJECT_ID = os.environ["PROJECT_ID"]
10 INSTANCE_ID = os.environ.get("BIGTABLE_INSTANCE", "ecommerce-clicks")
11 TABLE_ID = os.environ.get("BIGTABLE_TABLE", "clickstream")
12 MAX_TS = 999999999999999
13
14
15 def bucket_for(user_id: str, buckets: int = 32) -> str:
16     digest = hashlib.sha256(user_id.encode("utf-8")).hexdigest()
17     return f"{int(digest[:4], 16) % buckets:02d}"
18
19
20 def row_key(user_id: str, session_id: str, event_ms: int) -> bytes:
21     reverse_ms = MAX_TS - event_ms
22     return f"{bucket_for(user_id)}#{user_id}#{session_id}#{reverse_ms:013d}"
    ".encode("utf-8")
23
24
25 def main() -> None:
26     client = bigtable.Client(project=PROJECT_ID, admin=True)
27     table = client.instance(INSTANCE_ID).table(TABLE_ID)
28     user_id = "user-123"
29     session_id = "session-456"
30     event_ms = int(time.time() * 1000)
31     row = table.direct_row(row_key(user_id, session_id, event_ms))
32     row.set_cell("event", "event_id", str(uuid.uuid4()))
33     row.set_cell("event", "type", "product_view")
34     row.set_cell("event", "product_id", "sku-00042")
35     row.set_cell("event", "event_ms", str(event_ms))
36     row.commit()
37     print({"status": "written", "user_id": user_id, "session_id":
    session_id})
38
39
40 if __name__ == "__main__":
41     main()

```

Listing 4.5: Writing a Bigtable clickstream event with a salted reversed-timestamp row key.

3. **Add database-specific setup.** Create a private-IP Cloud SQL instance for users, a Spanner instance for inventory and checkout, and a Bigtable instance for clickstream. Use

the same VPC and service accounts where applicable.

```

1 gcloud sql instances create ecommerce-users '
2   --database-version=POSTGRES_16 '
3   --region=$env:REGION '
4   --network=$env:VPC '
5   --no-assign-ip '
6   --backup-start-time=03:00
7
8 gcloud spanner instances create ecommerce-global '
9   --config=nam3 '
10  --description="E-commerce global inventory and checkout" '
11  --processing-units=1000
12
13 gcloud spanner databases create commerce --instance=ecommerce-global
14
15 gcloud bigtable instances create ecommerce-clicks '
16   --display-name="E-commerce clickstream" '
17   --cluster=ecommerce-clicks-c1 '
18   --cluster-zone=$env:ZONE '
19   --cluster-num-nodes=3 '
20   --cluster-storage-type=SSD
21
22 cbt -project $env:PROJECT_ID -instance ecommerce-clicks createtable
   clickstream
23 cbt -project $env:PROJECT_ID -instance ecommerce-clicks createfamily
   clickstream event
24 cbt -project $env:PROJECT_ID -instance ecommerce-clicks setgcpolicy
   clickstream event maxage=90d

```

Listing 4.6: Representative Cloud SQL, Spanner, and Bigtable creation commands for the capstone.

## Deliverables

- A one-page architecture summary explaining why each Part I service is used and what data it owns.
- A completed reference diagram showing browser, application VPC, private connectivity, Cloud SQL, Spanner, Bigtable, Storage, KMS, IAM, logging, and monitoring.
- Command transcripts for network peering, KMS, bucket creation, lifecycle configuration, Cloud SQL private IP, Spanner database, and Bigtable table setup.
- DDL or schema notes for user accounts, inventory reservations, checkout orders, and clickstream rows.
- Python or pseudocode evidence for one user lookup, one checkout transaction, one image object operation, and one clickstream write.
- A security and operations checklist covering IAM, service accounts, CMEK, backups, lifecycle, audit logs, alerting, and cleanup.

## Acceptance Criteria

- Cloud SQL is reachable through private IP from the application VPC and has automated backups enabled.
- Spanner schema supports globally consistent inventory reservation and checkout without duplicate order application.

- Bigtable row-key design distributes clickstream writes and supports recent-session reads with bounded scans.
- Cloud Storage product-image bucket uses uniform bucket-level access, CMEK, lifecycle policy, and appropriate versioning.
- VPC peering or private services access is documented with reserved ranges, routes, firewall assumptions, and service producer connectivity.
- IAM grants each workload only the roles required for its owned data service and no broad owner/editor permissions.
- Monitoring covers database latency, errors, backups, storage growth, hot ranges, lifecycle actions, and KMS key usage.
- The final report includes cleanup steps and a residual-risk statement for production hardening.

### Stretch Goals

- Add Pub/Sub and Dataflow between the web tier and Bigtable so clickstream ingestion can absorb traffic bursts.
- Export Spanner order facts and Bigtable clickstream summaries to BigQuery for product analytics.
- Add Cloud Armor, Identity-Aware Proxy, Secret Manager, and Binary Authorization to strengthen the application tier.
- Create Terraform or Deployment Manager equivalents for the manual commands.
- Add synthetic load tests for checkout latency, image-object throughput, and clickstream row-key distribution.
- Estimate monthly cost by service, environment, and workload owner using labels and billing export.

## Appendix A

# Cross-Cloud Service Equivalence

This appendix is a living reference that maps conceptually equivalent services and ideas across the six platforms studied in this book series: Amazon Web Services (AWS), Microsoft Azure, Microsoft Fabric, Google Cloud Platform (GCP), MongoDB, and Snowflake. The names differ, but the underlying data-engineering concepts—durable object storage, tiered lifecycle economics, immutability, managed relational engines, elastic compute, and disciplined data organization—recur everywhere.

### Note

This appendix grows with each chapter. Every new chapter contributes the equivalence tables introduced in its cross-cloud callout, so by the end of the book you will hold a single consolidated Rosetta Stone for moving fluently between clouds. Where a clean one-to-one mapping does not exist, the prose notes the closest analog and the caveat.

## A.1 Object and Data-Lake Storage

The foundational layer covered in Chapter 1. Every platform offers durable, HTTP-addressable object storage with tiered pricing, lifecycle transitions, versioning, and write-once-read-many (WORM) controls.

| Capability          | AWS                        | Azure                       | GCP                             |
|---------------------|----------------------------|-----------------------------|---------------------------------|
| Object storage      | Amazon S3                  | ADLS Gen2 / Blob            | Cloud Storage                   |
| Hot tier            | S3 Standard                | Hot                         | Standard                        |
| Infrequent access   | S3 Standard-IA             | Cool                        | Nearline                        |
| Archive (instant)   | Glacier Instant Re-trieval | Cold                        | Coldline                        |
| Deep archive        | Glacier Deep Archive       | Archive                     | Archive                         |
| Lifecycle rules     | Lifecycle policies         | Lifecycle management        | Object Lifecycle Mgmt           |
| Versioning          | S3 Versioning              | Blob versioning             | Object Versioning               |
| Immutability (WORM) | Object Lock                | Immutable blob stor-<br>age | Bucket Lock                     |
| Cross-region copy   | CRR / SRR                  | Object replication          | Dual/multi-region +<br>Transfer |
| Encryption at rest  | SSE-S3 / SSE-KMS           | SSE + CMK (Key<br>Vault)    | Google-managed /<br>CMEK        |

The three remaining platforms approach storage differently. **Microsoft Fabric** exposes a single logical lake, *OneLake*, and uses *shortcuts* to virtualize S3, ADLS Gen2, and Google Cloud Storage in place without copying. **Snowflake** abstracts storage entirely: table data lives in cloud object storage as immutable, columnar *micro-partitions* that the service manages on your

behalf. **MongoDB** persists documents through the *WiredTiger* storage engine, balancing an internal cache against the operating-system page cache rather than exposing storage tiers.

## A.2 Managed Relational Databases

The focus of Chapter 2 for the three hyperscalers. Each provides a fully managed relational engine, a cloud-native high-performance variant, high availability, read replicas, and a migration service.

| Capability          | AWS                     | Azure                        | GCP                    |
|---------------------|-------------------------|------------------------------|------------------------|
| Managed relational  | Amazon RDS              | Azure SQL Database           | Cloud SQL              |
| Cloud-native engine | Amazon Aurora           | SQL Hyperscale               | AlloyDB                |
| Managed PostgreSQL  | RDS / Aurora PostgreSQL | Azure DB for PostgreSQL      | Cloud SQL / AlloyDB    |
| High availability   | Multi-AZ                | Zone-redundant HA            | Regional HA            |
| Read scaling        | Read Replicas           | Read replicas / geo-replicas | Read Replicas          |
| Global / geo        | Aurora Database Global  | Active geo-replication       | Cross-region replicas  |
| Serverless compute  | Aurora Serverless v2    | Azure SQL Serverless         | AlloyDB (elastic)      |
| Migration service   | DMS + SCT               | Azure Database Migration Svc | Database Migration Svc |

**Snowflake**, **Fabric**, and **MongoDB** are not managed relational engines, but they solve overlapping problems: Snowflake and Fabric provide SQL analytics over warehouse/lakehouse storage, while MongoDB provides a distributed document store whose schema design (Chapter 2) replaces rigid relational normalization with intentional, access-driven modeling.

## A.3 Elastic Compute and Analytical Warehouses

Snowflake’s Chapter 2 topic—separating compute from storage and scaling it elastically—has a counterpart in every analytics platform.

| Concept             | Snowflake               | AWS                       | Azure / GCP                    |
|---------------------|-------------------------|---------------------------|--------------------------------|
| Compute unit        | Virtual Warehouse       | Redshift RA3 / Serverless | Synapse pool / Big-Query slots |
| Elastic concurrency | Multi-cluster warehouse | Concurrency scaling       | Autoscale / on-demand          |
| Pause when idle     | Auto-suspend / resume   | Pause / resume            | Auto-pause / serverless        |
| Billing unit        | Credits                 | RPU / node-hours          | DWU / slot-hours               |
| Result reuse        | Result cache            | Result cache              | Result / BI cache              |

**Microsoft Fabric** meters all workloads against a shared pool of Capacity Units (CU) on an F-SKU, an approach closer to a single elastic budget than to per-engine warehouses. **MongoDB Atlas** scales through cluster tiers with optional auto-scaling of compute and storage.

## A.4 Data Organization and the Medallion Pattern

Fabric’s Chapter 2 topic—the Bronze/Silver/Gold medallion—is a portable design pattern rather than a product. Every platform implements the same progressive-refinement idea with its own primitives.

| Layer              | Fabric           | Data<br>(AWS/Azure/GCP)        | lake   | Snowflake           |
|--------------------|------------------|--------------------------------|--------|---------------------|
| Bronze (raw)       | Bronze Lakehouse | raw/<br>(S3/ADLS/GCS)          | prefix | RAW schema / stage  |
| Silver (validated) | Silver Lakehouse | validated/ prefix              |        | STAGING schema      |
| Gold (curated)     | Gold Lakehouse   | curated/ prefix                |        | ANALYTICS<br>schema |
| Storage format     | Delta-Parquet    | Parquet / Delta / Ice-<br>berg |        | Micro-partitions    |

**MongoDB** realizes the same separation with distinct collections or databases (for example **raw\_events**, **validated**, **curated**) governed by schema-design patterns rather than a lake layout.

## A.5 Document Data Modeling

MongoDB’s Chapter 2 schema-design patterns generalize to every document and wide-column store.

| Concept              | MongoDB               | Amazon<br>nomoDB    | Dy- | Azure Cosmos DB               |
|----------------------|-----------------------|---------------------|-----|-------------------------------|
| Primary access unit  | Document / collection | Item / table        |     | Item / container              |
| Partitioning         | Shard key             | Partition key       |     | Partition key                 |
| One-table modeling   | Embedding + patterns  | Single-table design |     | Embedding + parti-<br>tioning |
| Time-series grouping | Bucket Pattern        | Composite sort keys |     | Bucketing by key              |
| Optional attributes  | Attribute Pattern     | Sparse attributes   |     | Sparse properties             |

Google Cloud’s closest analogs are **Firestore** and **Bigtable**. In **Snowflake** and **Fabric**, semi-structured documents are stored in **VARIANT** (Snowflake) or JSON columns and queried with path expressions rather than modeled as collections.

## A.6 Globally Distributed and NoSQL Databases

Chapter 3 branches by platform into high-scale NoSQL (AWS DynamoDB, Azure Cosmos DB), globally consistent NewSQL (GCP Cloud Spanner), and advanced document modeling (MongoDB). They all answer the same question: how to scale a database horizontally across partitions and regions.

| Concept           | DynamoDB          | Cosmos DB           | Cloud Spanner                   | MongoDB                          |
|-------------------|-------------------|---------------------|---------------------------------|----------------------------------|
| Model             | Key-value / doc   | Multi-model         | Distributed SQL                 | Document                         |
| Partitioning      | Partition key     | Partition key       | Split by key range              | Shard key                        |
| Throughput unit   | RCU / WCU         | Request Units (RU)  | Nodes / processing<br>units     | Cluster tier                     |
| Secondary indexes | GSI / LSI         | Automatic indexing  | Secondary indexes               | Indexes                          |
| Change stream     | DynamoDB Streams  | Change Feed         | Change streams                  | Change streams                   |
| Expiry            | TTL               | TTL                 | (manual)                        | TTL index                        |
| Multi-region      | Global Tables     | Multi-region writes | Multi-region (syn-<br>chronous) | Zones / global<br>clusters       |
| Consistency       | Eventual / strong | 5 tunable levels    | External (True-<br>Time)        | Tunable<br>read/write<br>concern |

**Snowflake** and **Fabric** are analytical platforms rather than OLTP key-value stores; they ingest from these operational databases and store semi-structured payloads in **VARIANT**/JSON columns. Google’s dedicated NoSQL analogs are **Firestore** (document) and **Bigtable** (wide-column).

## A.7 Managed Apache Spark

Microsoft Fabric’s Chapter 3 topic—managed Spark pools and notebooks—has a first-class equivalent on every cloud.

| Capability        | Fabric                 | AWS                         | Azure / GCP                             |
|-------------------|------------------------|-----------------------------|-----------------------------------------|
| Managed Spark     | Starter / Custom pools | Glue / EMR / EMR Serverless | Synapse Spark / Databricks; Dataproc    |
| Notebook UX       | Fabric Notebooks       | Glue Studio / EMR Studio    | Synapse / Databricks; Vertex AI         |
| Serverless option | Autoscaling pools      | Glue / EMR Serverless       | Synapse serverless; Dataproc Serverless |
| Lake format       | Delta on OneLake       | Delta / Iceberg / Hudi      | Delta; Delta / Iceberg                  |

**Snowflake** offers **Snowpark** for DataFrame-style processing, and **MongoDB** integrates through the Spark Connector.

## A.8 Query Acceleration and Materialized Views

Snowflake’s Chapter 3 acceleration tools—materialized views, the Search Optimization Service, and transient/temporary tables—map to comparable features in every warehouse.

| Feature             | Snowflake               | AWS Redshift          | Azure Synapse / GCP BigQuery |
|---------------------|-------------------------|-----------------------|------------------------------|
| Materialized views  | Materialized Views      | Materialized views    | Materialized views           |
| Point-lookup accel. | Search Optimization Svc | Sort keys / zone maps | (indexing) / search indexes  |
| Result reuse        | Result cache            | Result cache          | Result-set cache / BI Engine |
| Ephemeral tables    | Transient / temporary   | Temp tables           | Temp tables                  |

**Fabric** accelerates reads through Direct Lake and its Warehouse engine, while **MongoDB** relies on secondary and compound indexes plus the in-memory working set.

## A.9 In-Memory Caching and Hybrid Ingestion

The AWS and Azure Chapter 4 topic—low-latency caching in front of a durable store, fed by hybrid (edge-to-cloud) ingestion—has a managed Redis-compatible equivalent on every cloud.

| Capability          | AWS                               | Azure                    | GCP / others                  |
|---------------------|-----------------------------------|--------------------------|-------------------------------|
| Managed cache       | ElastiCache (Redis/Memcached)     | Cache for Redis          | Memorystore (Redis/Memcached) |
| Real-time API layer | AppSync / API Gateway             | API Management / SignalR | API Gateway / Apigee          |
| Hybrid ingestion    | DataSync / Snow family / IoT Core | Data Box / IoT Hub / Arc | Transfer Appliance / IoT Core |
| Streaming intake    | Kinesis / MSK                     | Event Hubs               | Pub/Sub                       |

**Snowflake** and **Fabric** rely on result and Direct Lake caches rather than a standalone Redis tier, while **MongoDB** serves hot data from its in-memory working set and Atlas can pair with an external cache.

## A.10 Wide-Column and Key-Value NoSQL

GCP’s Chapter 4 topic—Cloud Bigtable—is a wide-column, high-throughput, low-latency NoSQL store. Its peers span the managed and open-source ecosystems.

| Characteristic   | GCP Bigtable              | AWS                      | Azure / open source            |
|------------------|---------------------------|--------------------------|--------------------------------|
| Data model       | Wide-column               | DynamoDB (key-value/doc) | Cosmos DB (multi-model)        |
| HBase-compatible | HBase API                 | Keyspaces (Cassandra)    | Managed Instance for Cassandra |
| Scaling unit     | Nodes / clusters          | On-demand / provisioned  | RU/s (Cosmos)                  |
| Best fit         | Time-series, IoT, ad-tech | Serverless key-value     | Global multi-model             |

**Snowflake** and **Fabric** address these workloads analytically rather than as operational key-value stores; **MongoDB** covers the document end of the same NoSQL spectrum.

## A.11 Data Protection: Cloning, Time Travel, and Migration

Snowflake’s Chapter 4 topic—zero-copy cloning, Time Travel, and Fail-safe—plus MongoDB’s relational-to-document migration map to data-protection and migration tooling across the clouds.

| Capability             | Snowflake          | AWS / Azure                | GCP / Fabric                           |
|------------------------|--------------------|----------------------------|----------------------------------------|
| Instant clone          | Zero-copy clone    | Aurora clone / SQL DB copy | BigQuery table clone; OneLake shortcut |
| Point-in-time recovery | Time Travel        | PITR (RDS/SQL DB backups)  | PITR; Delta time travel                |
| Retention safety net   | Fail-safe (7 days) | Automated backups          | Backups; Delta VACUUM window           |
| Schema migration       | —                  | DMS / SCT                  | Database Migration Service             |

**MongoDB** migrations use **Relational Migrator** plus the embed-vs-reference patterns from Chapter 4, and **Delta Lake** exposes historical versions through `VERSION AS OF` time-travel queries.

### Tip

Use this appendix as a study aid for the book’s lockstep, cross-cloud approach: when you learn a concept on one platform, immediately locate its equivalent here and predict how the other five would express the same idea. That habit turns six separate vocabularies into one mental model.

# Glossary

This glossary grows with each chapter and collects the cross-cloud data-engineering terms introduced so far. Definitions are platform-neutral unless a platform is named explicitly.

**Access tier** A storage pricing class that trades retrieval cost and latency for lower at-rest price (for example Azure Hot, Cool, Cold, and Archive). See also *storage class*.

**AlloyDB** Google Cloud’s PostgreSQL-compatible engine with a columnar accelerator for analytical queries, positioned as the cloud-native tier above Cloud SQL.

**Apache Spark** A distributed data-processing engine; Microsoft Fabric runs it on Starter or Custom pools, with counterparts in AWS Glue/EMR, Azure Synapse/Databricks, and GCP Dataproc.

**Aurora** Amazon’s cloud-native relational engine with distributed storage, Global Databases, and a serverless (v2) compute option.

**Availability SLA** The contractual uptime a provider guarantees for a service, distinct from *durability*, which concerns data loss rather than reachability.

**Bigtable** Google Cloud’s wide-column, high-throughput, low-latency NoSQL store for time-series, IoT, and ad-tech workloads; exposes the HBase API and scales by nodes and clusters.

**Bucket (storage)** The top-level container for objects in AWS S3 and Google Cloud Storage; Azure calls the equivalent a *container*.

**Bucket Pattern** A MongoDB schema-design pattern that groups many small time-series readings into a single document to cut document count and metadata overhead.

**Capped collection** A fixed-size MongoDB collection that overwrites its oldest documents when full, useful for high-volume logs and bounded activity feeds.

**Change Feed / Streams** An ordered log of item-level changes used for event-driven processing; called Change Feed in Cosmos DB, DynamoDB Streams in DynamoDB, and change streams in MongoDB.

**Checkpoint** In WiredTiger, a periodic flush of a consistent snapshot of cached data to disk, bounding crash-recovery time together with the journal.

**Clustering key** A user-defined column set that Snowflake uses to co-locate related rows within micro-partitions, improving partition pruning on selective queries.

**CMEK / CMK** Customer-managed encryption keys (GCP CMEK; Azure customer-managed keys via Key Vault), giving the customer control over the key that encrypts data at rest.

**Cloud Spanner** Google Cloud’s horizontally scalable, globally distributed relational database offering external (strong) consistency backed by TrueTime.

**Consistency level** A tunable guarantee about read recency; Azure Cosmos DB offers five (Strong, Bounded Staleness, Session, Consistent Prefix, Eventual).

**Cosmos DB** Azure’s globally distributed multi-model NoSQL database, billed in Request Units (RU) and supporting multiple APIs (NoSQL, MongoDB, Cassandra).

**Cross-Region Replication (CRR)** Asynchronous copying of objects to a bucket in another region for disaster recovery; Azure’s equivalent user-configured feature is *object replication*.

**DynamoDB** Amazon’s managed key-value and document NoSQL database, with capacity measured in read/write capacity units (RCU/WCU) or on-demand, and Global Tables for multi-region.

**Durability** The probability that stored data is not lost, commonly quoted as “eleven nines” (99.999999999%) for cloud object storage.

**ElastiCache** Amazon’s managed in-memory cache (Redis- and Memcached-compatible) placed in front of a durable store to serve hot data at sub-millisecond latency; Azure’s analog is Cache for Redis and GCP’s is Memorystore.

**Embedding vs referencing** The MongoDB modeling choice between nesting related data in one document (embedding) or linking documents by reference (and joining with `lookup`).

**Fail-safe** Snowflake’s non-configurable seven-day recovery window, available only to Snowflake support, that follows the Time Travel retention period as a last-resort safety net.

**Global Secondary Index (GSI)** A DynamoDB index with its own partition/sort key enabling queries on non-primary attributes; a Local Secondary Index (LSI) shares the table’s partition key.

**Hierarchical namespace (HNS)** The Azure Data Lake Storage Gen2 feature that adds true directories and atomic directory operations on top of Blob storage.

**Journal (WAL)** The write-ahead log that records changes before they are checkpointed, enabling crash recovery in MongoDB’s WiredTiger engine.

**JSON Schema validation** MongoDB’s rule mechanism (via `jsonSchema` in `createCollection`) that enforces required fields, data types, and value constraints.

**Interleaved table** A Cloud Spanner layout that physically co-locates child rows with their parent row to make parent-child joins fast.

**Lakehouse** An architecture that stores open table formats (such as Delta) in a data lake while offering warehouse-like SQL and governance; the core storage abstraction in Microsoft Fabric.

**Lifecycle policy** A rule set that automatically transitions or expires objects across storage tiers based on age or other conditions.

**Medallion architecture** A progressive-refinement data layout with Bronze (raw), Silver (validated), and Gold (curated/enriched) layers.

**Materialized view** A precomputed, automatically maintained query result stored for fast reads, trading maintenance/storage cost for lower query latency (Snowflake, Redshift, Synapse, BigQuery).

**Memorystore** Google Cloud’s managed in-memory cache service (Redis and Memcached), the GCP counterpart to AWS ElastiCache and Azure Cache for Redis.

- Micro-partition** Snowflake's internal storage unit: an immutable, compressed, columnar file of roughly 50–500 MB of uncompressed data, annotated with per-column metadata for pruning.
- Multi-AZ** An AWS deployment spanning multiple Availability Zones for high availability; Azure's analog is zone-redundant HA and GCP's is regional HA.
- Multi-cluster warehouse** A Snowflake virtual warehouse that adds and removes compute clusters automatically to absorb concurrent query load.
- Object Lock** The AWS S3 WORM feature (governance mode, compliance mode, and legal hold) that prevents object deletion or overwrite for a retention period.
- OneLake** Microsoft Fabric's single, tenant-wide logical data lake shared by all workspaces.
- Partition pruning** Skipping storage units (micro-partitions, files, or partitions) that cannot match a query's predicates, reducing scanned data.
- Point-in-time recovery (PITR)** Restoring a database or table to any moment within a retention window, using continuous backups or transaction logs; realized as Snowflake Time Travel, RDS/SQL Database PITR, and Delta Lake time travel.
- Read replica** A read-only copy of a database that offloads read traffic and can serve as a failover or geo-local endpoint.
- Relational Migrator** MongoDB's tool for migrating relational schemas and data to document collections, applying embed-versus-reference decisions from schema-design patterns.
- Request Unit (RU)** Azure Cosmos DB's normalized currency for throughput, abstracting the CPU, memory, and IOPS consumed by a database operation.
- Search Optimization Service (SOS)** A Snowflake feature that builds a persistent search access path to accelerate selective point-lookup queries on high-cardinality columns.
- Shortcut (Fabric)** A OneLake reference that virtualizes external data (S3, ADLS Gen2, GCS) in place so it can be queried without copying.
- Storage class** The AWS/GCP term for a pricing tier of object storage (for example S3 Standard, Standard-IA, Glacier; GCP Standard, Nearline, Coldline, Archive).
- SSE-KMS** Server-side encryption in AWS S3 using keys managed in AWS Key Management Service.
- Time Travel** Snowflake's ability to query, clone, or restore data as it existed at an earlier point within a retention period (up to 90 days on higher editions); Delta Lake offers an analogous `VERSION AS OF / TIMESTAMP AS OF` capability.
- Transient table** A Snowflake table that persists across sessions but has no Fail-safe period, reducing storage cost for reproducible staging data; a temporary table lives only for one session.
- TrueTime** Google's globally synchronized clock (with bounded uncertainty) that lets Cloud Spanner assign commit timestamps and provide external consistency.
- TTL (Time to Live)** An expiry mechanism that automatically deletes items or documents after a set time (DynamoDB TTL, Cosmos DB TTL, MongoDB TTL index).
- Versioning** Retaining prior versions of an object or blob so that overwrites and deletes are recoverable.

**Virtual warehouse** Snowflake’s unit of elastic compute, sized XS–6XL and billed in credits, fully decoupled from storage.

**Wide-column store** A NoSQL data model that stores rows as sparse, dynamically wide sets of column families keyed by a row key (Bigtable, HBase, Cassandra), optimized for high write throughput and range scans.

**WiredTiger** MongoDB’s default storage engine, using a configurable internal cache alongside the operating-system page cache, with journaling and checkpointing.

**WORM** Write-once-read-many storage that prevents modification or deletion for a retention period; implemented as Object Lock (AWS), immutable blob storage (Azure), or Bucket Lock (GCP).

**Z-ordering** A Delta Lake optimization that co-locates related information in the same files by multi-dimensional clustering, improving data skipping; applied via `OPTIMIZE ... ZORDER BY` and complemented by `VACUUM` to reclaim obsolete files.

**Zero-copy clone** Snowflake’s instant, storage-free copy of a table, schema, or database that shares underlying micro-partitions until either copy is modified (copy-on-write).

# Bibliography

- [1] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E. Gruber. Bigtable: A distributed storage system for structured data. In *ACM Transactions on Computer Systems*, volume 26, pages 1–26, 2008.
- [2] James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, J. J. Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, et al. Spanner: Google’s globally-distributed database. In *Proceedings of the 10th USENIX Symposium on Operating Systems Design and Implementation*, pages 251–264, 2012.
- [3] Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung. The google file system. *ACM SIGOPS Operating Systems Review*, 37(5):29–43, 2003.
- [4] Google Cloud. Alloydb ai documentation. <https://cloud.google.com/alloydb/docs/ai>, 2026. AlloyDB AI, vector search, and model integration documentation.
- [5] Google Cloud. Alloydb columnar engine documentation. <https://cloud.google.com/alloydb/docs/columnar-engine>, 2026. Columnar engine concepts and query acceleration guidance for AlloyDB.
- [6] Google Cloud. Alloydb for postgresql documentation. <https://cloud.google.com/alloydb/docs>, 2026. AlloyDB clusters, instances, backups, high availability, read pools, and PostgreSQL compatibility.
- [7] Google Cloud. Bigquery documentation. <https://cloud.google.com/bigquery/docs>, 2026. Serverless data warehouse documentation.
- [8] Google Cloud. Cloud bigtable documentation. <https://cloud.google.com/bigtable/docs>, 2026. Cloud Bigtable product documentation for instances, clusters, tables, replication, backups, and operations.
- [9] Google Cloud. Cloud bigtable performance. <https://cloud.google.com/bigtable/docs/performance>, 2026. Performance guidance for nodes, workloads, latency, throughput, and client behavior.
- [10] Google Cloud. Cloud bigtable schema design. <https://cloud.google.com/bigtable/docs/schema-design>, 2026. Guidance for row keys, column families, time-series schemas, and avoiding hotspots.
- [11] Google Cloud. Cloud billing documentation. <https://cloud.google.com/billing/docs>, 2026. Billing accounts, budgets, alerts, billing export, and cost controls.
- [12] Google Cloud. Cloud spanner documentation. <https://cloud.google.com/spanner/docs>, 2026. Cloud Spanner product documentation for instances, databases, schemas, transactions, backups, and operations.

- [13] Google Cloud. Cloud spanner query execution plans. <https://cloud.google.com/spanner/docs/query-execution-plans>, 2026. Documentation for reading query plans and understanding optimizer behavior.
- [14] Google Cloud. Cloud spanner schema design best practices. <https://cloud.google.com/spanner/docs/schema-design>, 2026. Guidance for primary keys, interleaved tables, indexes, and avoiding hot spots.
- [15] Google Cloud. Cloud sql documentation. <https://cloud.google.com/sql/docs>, 2026. Managed PostgreSQL, MySQL, and SQL Server documentation for instances, high availability, backups, replicas, and operations.
- [16] Google Cloud. Cloud sql private ip documentation. <https://cloud.google.com/sql/docs/private-ip>, 2026. Private services access and private IP connectivity guidance for Cloud SQL.
- [17] Google Cloud. Cloud storage documentation. <https://cloud.google.com/storage/docs>, 2026. Product documentation for buckets, objects, locations, classes, lifecycle management, retention, and transfer options.
- [18] Google Cloud. Database migration service documentation. <https://cloud.google.com/database-migration/docs>, 2026. Connection profiles, migration jobs, continuous replication, validation, and cutover guidance.
- [19] Google Cloud. Dataflow documentation. <https://cloud.google.com/dataflow/docs>, 2026. Apache Beam managed service documentation.
- [20] Google Cloud. Dataplex documentation. <https://cloud.google.com/dataplex/docs>, 2026. Governance, metadata, data quality, and lake management documentation.
- [21] Google Cloud. Google cloud architecture framework. <https://cloud.google.com/architecture/framework>, 2026. Reliability, security, operational excellence, performance, and cost guidance.
- [22] Google Cloud. Google cloud cli documentation. <https://cloud.google.com/sdk/docs>, 2026. Installation, initialization, authentication, and command reference.
- [23] Google Cloud. Google cloud free program. <https://cloud.google.com/free/docs/free-cloud-features>, 2026. Free trial, credits, Always Free products, and account upgrade behavior.
- [24] Google Cloud. Identity and access management documentation. <https://cloud.google.com/iam/docs>, 2026. Cloud IAM roles, service accounts, policies, and least-privilege guidance.
- [25] Google Cloud. Key visualizer for cloud bigtable. <https://cloud.google.com/bigtable/docs/keyvis-overview>, 2026. Documentation for diagnosing Bigtable row-key access patterns, hot ranges, and heatmaps.
- [26] Google Cloud. Key visualizer for cloud spanner. <https://cloud.google.com/spanner/docs/key-visualizer>, 2026. Documentation for diagnosing key-range traffic, hot spots, and access patterns.
- [27] Google Cloud. Resource hierarchy documentation. <https://cloud.google.com/resource-manager/docs/cloud-platform-resource-hierarchy>, 2026. Organization, folder, project, and resource hierarchy guidance.
- [28] Google Cloud. Storage transfer service documentation. <https://cloud.google.com/storage-transfer/docs>, 2026. Online and scheduled data transfer guidance.

- [29] Google Cloud. Transfer appliance documentation. <https://cloud.google.com/transfer-appliance/docs>, 2026. Offline transfer appliance guidance for large data migrations.
- [30] Sergey Melnik, Andrey Gubarev, Jing Jing Long, Geoffrey Romer, Shiva Shivakumar, Matt Tolton, and Theo Vassilakis. Dremel: Interactive analysis of web-scale datasets. In *Proceedings of the VLDB Endowment*, volume 3, pages 330–339, 2010.

# Index

- 2-Step Verification, 4
- 99.999 percent availability, 59
- AlloyDB, 35
  - capstone, 45
  - columnar engine, 40
  - ML integration, 40
  - PostgreSQL, 39
- Always Free tier, 2
- Apache Spark
  - equivalence, 87
- app profiles, 68
- Archive, 18
- availability, 17
- Bigtable
  - app profiles, 70, 71
  - cbt, 74
  - clusters, 70
  - equivalence, 88
  - financial services, 76
  - garbage collection, 72
  - instances, 70
  - monitoring, 72
  - nodes, 70
  - Python client, 74
  - replication, 71
  - row keys, 68
  - row-key design, 71
  - tablets, 68, 70
- billing account, 2
- billing budgets, 1, 6
- billing export, 6
- bucket, 17
- Bucket Lock, 19, 29
- caching
  - equivalence, 87
- capstone project, 29, 45, 62
- change data capture, 43
- cloning
  - equivalence, 88
- Cloud Bigtable, 68
  - architecture, 70
  - capstone, 78
  - performance, 72
  - security, 76
- Cloud IAM, 1, 4
- Cloud KMS, 26
- Cloud Shell, 1, 5
- Cloud Spanner, 51
  - architecture, 53
  - capstone, 62, 78
  - indexes, 55
  - interleaving, 55
  - multi-region, 57
  - performance, 56
  - replicas, 53
  - schema design, 54
  - security, 60
- Cloud SQL, 35
  - architecture, 36
  - capstone, 45, 78
  - high availability, 37
  - PostgreSQL, 41
  - read replicas, 38
  - security, 44
- Cloud Storage, 15
  - capstone, 78
  - verification, 9
- CMEK, 25, 29, 60, 76
- Coldline, 18
- columnar engine, 39
- connection pooling, 44
- consistency, 17
- cost governance, 26, 44, 60, 76
- cross-cloud equivalence, 84
- cutover, 43
- data lake, 15
- data migration, 21
- database migration, 38
- Database Migration Service, 35, 38
- Dataplex, 26
- disaster recovery, 15, 25
- distributed databases

- equivalence, 86
- document modeling
  - equivalence, 86
- dual-region bucket, 18
- durability, 17
- e-commerce backend, 78
- elastic compute
  - equivalence, 85
- ERP migration, 43
  - capstone, 45
- external consistency, 51, 53
- failover, 36
- failover test, 41
- financial trading, 76
- financial transactions, 59
- Folder, 3
- foreign keys, 54
- free trial, 2
- garbage collection
  - Bigtable, 73
- gcloud auth list, 9
- gcloud config list, 9
- gcloud init, 5
- gcloud storage, 5, 21
- glossary, 89
- Google Cloud
  - free trial, 1
  - getting started, 1
- Google Cloud account, 2
- Google Cloud CLI, 1, 5
- google-cloud-storage, 22
- gsutil, 5, 21
- hands-on project, 22, 41, 57, 74
- hashing, 71
- high availability, 35
- hot spots, 56
- hotspotting, 68, 71
- IAM, 26, 60, 76
- in-memory cache
  - equivalence, 87
- interleaved schema, 57
- interleaved tables, 51, 54
- IoT telemetry, 74
- Key Visualizer, 56, 68, 73
  - Bigtable, 72
- labels, 6, 8
- leader replica, 53
- least privilege, 4
- lifecycle JSON, 22
- location types, 18
- machine learning, 39
- materialized views
  - equivalence, 87
- medallion architecture
  - equivalence, 85
- media asset store, 29
- media streaming, 25
- metadata, 17
- Milestone 1 capstone, 78
- monitoring, 26
- multi-cloud, 84
- multi-region architecture, 25
- multi-region bucket, 18
- multi-region databases, 51
- naming conventions, 8
- Nearline, 18
- NewSQL, 51
- NoSQL
  - equivalence, 86
  - wide-column, 68
- Object Lifecycle Management, 19
- object storage, 15
  - equivalence, 84
  - fundamentals, 17
- Object Versioning, 19
- offline transfer, 21
- online transfer, 21
- operations, 26
- Organization, 3
- Paxos, 51, 53
- payment ledger, 59
  - capstone, 62
- point-in-time recovery
  - equivalence, 88
- PostgreSQL, 35
- PowerShell, 1
- primary key, 54
- private connectivity, 78
- private IP, 35, 38
- private services access, 38, 39
- Project, 3
- public access prevention, 26
- query acceleration
  - equivalence, 87

- query execution plan, 56
- query optimizer, 56
- quotas, 6
- read replica, 36
- read replicas, 35
- regional bucket, 18
- regional HA, 36
- regions, 8
- regions and zones, 1
- relational databases
  - equivalence, 85
- replication lag, 38
- Resource, 3
- resource hierarchy, 3
- Retention Policies, 19
- reversed timestamp, 71, 74
- row key
  - bucketing, 72
  - hashing, 72
  - salting, 72
- runbook, 26
- salting, 71
- secondary indexes, 54
- Secret Manager, 44
- service accounts, 4
- service mapping, 84
- serving layer, 76
- setup verification, 9
- soft delete, 19
- Standard, 18
- stock ticks, 76
- Storage Classes, 18
- storage classes
  - equivalence, 84
- Storage Transfer Service, 15
  - capstone, 29
- strong consistency, 53
- synthetic data, 57
- time travel
  - equivalence, 88
- Transfer Appliance, 15
- TrueTime, 51, 53
  - external consistency, 54
- uniform bucket-level access, 26
- virtual warehouse
  - equivalence, 85
- VPC, 39
- VPC Service Controls, 76
- wide-column
  - equivalence, 88
- wide-column NoSQL, 68
- zero data loss, 59, 62
- zero-downtime migration, 43
- zones, 8